



UNIVERSITÀ DEGLI STUDI DEL SANNIO
DIPARTIMENTO DI INGEGNERIA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Community Smell in Open Source un'analisi qualitativa e temporale

Project Report Evoluzione e Qualità del Software 2024/2025

Studenti:
Rita Lamparelli 399000607
Marco Rossi 399000605

Docente:
Damian Andrew Tamburri

Sommario

INTRODUZIONE	3
DESCRIZIONE DEL DATASET	4
STRUTTURA DEL DATASET	4
<i>Esempio di un Progetto</i>	5
TABELLA DESCRITTIVA DEL DATASET	5
CLASSIFICAZIONE QUALITATIVA DEI PROGETTI	8
ANALISI DELLE RELAZIONI DI CAUSALITÀ TRA I COMMUNITY SMELL TRAMITE IL TEST DI GRANGER.....	9
RQ1: UN COMMUNITY SMELL PUÒ CAUSARNE UN ALTRO?	9
<i>Risultati RQ1</i>	10
<i>Risultati per categoria di progetto</i>	12
RQ 1.1: ESISTE UNA RELAZIONE CAUSALE TRA LE METRICHE DI TURNOVER E I COMMUNITY SMELL?	14
<i>Risultati RQ1.1</i>	14
<i>Riepilogo dei Pattern Identificati</i>	16
<i>Risultati per categoria di progetto</i>	16
ANDAMENTO TEMPORALE DELLE OCCORRENZE DEI COMMUNITY SMELL	18
RQ2: ESISTONO PATTERN RICORRENTI NELL'ANDAMENTO TEMPORALE DELLE OCCORRENZE DEI COMMUNITY SMELL NEI PROGETTI OPEN SOURCE ANCHE IN RELAZIONE ALLE CATEGORIE QUALITATIVE DEFINITE?	19
<i>Osservazioni generali</i>	22
<i>Interpretazioni e Implicazioni RQ2</i>	23
ANALISI DELLA CATEGORIZZAZIONE DEI COMMIT NEI PROGETTI OPEN SOURCE (PROOF OF CONCEPT)	24
RQ3: ESISTE UNA RELAZIONE TRA LA TIPOLOGIA DI COMMIT E L'EMERGERE DI DETERMINATI COMMUNITY SMELLS IN PROGETTI OPEN SOURCE?	24
<i>Considerazioni generali RQ3</i>	32
<i>Limiti dell'approccio e sviluppi futuri RQ3</i>	32
CONCLUSIONI E SVILUPPI FUTURI.....	33
IMPATTO SU EVOLUZIONE E QUALITÀ SOFTWARE	33
APPENDICE 1: RISULTATI TEST DI GRANGER RQ1	34
APPENDICE 2: RISULTATI TEST DI GRANGER RQ2	39
APPENDICE 3: GRAFICI ANDAMENTO TEMPORALE OCCORRENZE COMMUNITY SMELL	48
APPENDICE 4: ANDAMENTO TEMPORALE DELLE CATEGORIE DI COMMIT PER PROGETTO	59
BIBLIOGRAFIA.....	70

Introduzione

Lo sviluppo del software è intrinsecamente un'attività sociale, in cui le modalità di interazione tra i membri di un team influiscono non solo sulla qualità del codice prodotto, ma anche sulla dinamica organizzativa e relazionale della comunità. Le difficoltà di comunicazione e collaborazione, derivanti da differenze culturali, di esperienza o di gerarchia, possono generare situazioni socio-tecniche sub-ottimali, note come community smells. Questi fenomeni ostacolano la produzione, l'operatività e l'evoluzione del software, creando un impatto negativo sia a livello tecnico che sociale. Negli ultimi anni, i community smells hanno attirato l'attenzione della ricerca, con studi che ne hanno esplorato l'origine, i fattori influenti e le strategie per il loro superamento. Tuttavia, la letteratura si è concentrata prevalentemente su analisi quantitative, mentre rimane ancora poco esplorata una comprensione qualitativa di questi fenomeni. Per colmare questa lacuna, il nostro progetto si propone di analizzare qualitativamente i progetti coinvolti nei community smells e i commit utilizzati per il calcolo degli stessi. Attraverso questa prospettiva, miriamo a identificare i contesti e le dinamiche specifiche che favoriscono l'insorgenza e la persistenza di tali fenomeni. Questo approccio permette di arricchire la comprensione delle cause e degli effetti dei community smells, contribuendo a sviluppare interventi mirati per migliorare le interazioni e la qualità complessiva nei contesti collaborativi, con particolare attenzione alle comunità open-source.

I community smell considerati in questo lavoro sono: organizational silos, black cloud, radio silence, missing links, prima donnas.

Organizational silo: Questa forma di "social debt" si riferisce alla presenza di aree compartimentate all'interno della comunità di sviluppatori che non comunicano tra loro, se non attraverso uno o due membri rispettivi. (G. Catolino, 2021)

Black cloud: Questo community smell riflette un sovraccarico di informazioni dovuto alla mancanza di comunicazioni strutturate o di una governance della cooperazione. (G. Catolino, 2021)

Radio-silence: Questo smell si manifesta quando un membro si inserisce in ogni interazione formale tra due o più sotto-comunità, con poca o nessuna flessibilità per introdurre altri canali paralleli.
(G. Catolino, 2021)

Prima-donnas: L'effetto prima-donnas si verifica quando un team di persone non è disposto ad accettare cambiamenti esterni proposti da altri membri del team, a causa di una collaborazione inefficace all'interno della comunità. La presenza di questo smell può generare problemi di isolamento, senso di superiorità, disaccordi costanti, mancanza di cooperazione e promuovere comportamenti egoistici nel team, noti anche come "prima-donnas". (Nuri Almarimi, 2020)

Missing link: Questo smell si manifesta quando gli sviluppatori collaborano sul codice sorgente senza comunicare tra loro. Ciò può influire sulla manutenzione del software a causa della mancanza di consapevolezza reciproca, della sfiducia e del divario di conoscenze. (Ahammed, 2020)

In questo progetto, abbiamo analizzato i community smell utilizzando progetti open source di grandi dimensioni. Il nostro obiettivo principale è stato quello di comprendere meglio la distribuzione e la dinamica dei community smell nel tempo, nonché la loro relazione con metriche organizzative e tecniche. In particolare, ci siamo concentrati su:

- Analisi delle relazioni di causalità tra i Community Smell tramite il Test di Granger

- Analisi delle relazioni di causalità tra i Community Smell e le metriche di turnover tramite il Test di Granger
- Analisi dell'andamento temporale delle occorrenze dei Community Smell in relazione alla categoria del progetto
- Analisi della relazione tra la categorizzazione dei commit e le occorrenze di Community Smell (proof of concept (PoC))

Descrizione del Dataset

Le analisi condotte in questo progetto utilizzano il Dataset dello studio di Simone Magnoni, (Magnoni, 2016). Il dataset considerato in questa tesi magistrale consiste in 60 progetti Open Source Software. Sono state considerate comunità FLOSS (Free/Libre Open Source Software) di diverse dimensioni, popolarità, abitudini di sviluppo, livelli di apertura e contesti applicativi.

In conclusione, le 60 comunità di sviluppo software considerate sono risultate diversificate rispetto a:

- **Dimensione del codice base:** media, grande (500-850 KLOC) e molto grande (>850 KLOC).
- **Linguaggio di programmazione principale:** Java, C#, C, Python, YAML e altri.
- **Dimensione della comunità:** 20 progetti medi (<50 membri), 19 progetti grandi (50-150 membri) e 21 progetti molto grandi (>150 membri).
- **Età:** giovani (<24 mesi), consolidate (24-32 mesi) e popolari (>32 mesi). (Magnoni, 2016).

Ai fini dell'analisi effettuata sono stati utilizzati i 21 progetti (molto grandi), con dati relativi a metriche socio-tecniche e alle occorrenze dei community smell per ciascun progetto. Ogni riga rappresenta un periodo trimestrale, durante il quale sono stati analizzati tutti i commit effettuati nel relativo intervallo temporale. Per ogni progetto, sono considerati 12 trimestri consecutivi. Complessivamente, il dataset offre una visione dettagliata delle caratteristiche e delle evoluzioni dei progetti nel tempo.

Struttura del Dataset

Le colonne principali includono:

- **range (intervallo temporale):** Identifica in modo univoco l'intervallo di commit analizzato.
- **range.date (date intervallo temporale):** Specifica il periodo temporale corrispondente (ad esempio, "2013-05 -- 2013-08").
- **Community Smell:** Numero di occorrenze per *organisational silo*, *prima donnas*, *radio silence*, *black cloud* e *missing links*.
- **Metriche Socio-Tecniche:** Inclusi indicatori come *socio-technical congruence* e *communicability*.
- **Metriche di Turnover:** Turnover globale e turnover dei core developer, calcolati tra finestre temporali consecutive.
- **Metriche di Rete Sociale:** Dati su densità, modularità e centralità della rete dei developer.

Esempio di un Progetto

Il progetto AngularJS comprende 12 finestre temporali trimestrali che vanno da maggio 2013 a maggio 2016. Di seguito alcune caratteristiche del progetto:

- Sviluppatori coinvolti: Variabile tra 396 e 949, con una percentuale di sviluppatori sponsorizzati che varia dallo 0.027 al 0.113.
- Community Smell principali: I *missing links* si distinguono come smell più frequente, con valori che oscillano tra 114 e 539 occorrenze.
- Metriche di rete sociale: La modularità della rete globale varia tra 0.1385 e 0.5095, indicando cambiamenti significativi nella struttura della rete collaborativa nel tempo

Tabella descrittiva del dataset

Category	Metric ID	Socio-technical Quality Metric Description
Developer Social Network metrics	devs	Number of developers present in the global Developers Social Network
	ml.only.devs	Number of developers present only in the communication Developers Social Network
	code.only.devs	Number of developers present only in the collaboration Developers Social Network
	ml.code.devs	Number of developers present both in the collaboration and in the communication DSNs
	perc.ml.only.devs	Percentage of developers present only in the communication Developers Social Network
	perc.code.only.devs	Percentage of developers present only in the collaboration Developers Social Network
	perc.ml.code.devs	Percentage of developers present both in the collaboration and in the communication DSNs
	sponsored.devs	Number of sponsored developers (95% of their commits are done in working hours)
	ratio.sponsored	Ratio of sponsored developers with respect to developers present in the collaboration DSN
Socio-technical metrics	st.congruence	Estimation of socio-technical congruence

Category	Metric ID	Socio-technical Quality Metric Description
	communicability	Estimation of information communicability (decisions diffusion)
	num.tz	Number of timezones involved in the software development
	ratio.smelly.devs	Ratio of developers involved in at least one Community Smell
Core community members metrics	core.global.devs	Number of core developers of the global Developers Social Network
	core.mail.devs	Number of core developers of the communication Developers Social Network
	core.code.devs	Number of core developers of the collaboration Developers Social Network
	sponsored.core.devs	Number of core sponsored developers
	ratio.sponsored.core	Ratio of core sponsored developers with respect to core developers of the collaboration DSN
	global.truck	Ratio of non-core developers of the global Developers Social Network
	mail.truck	Ratio of non-core developers of the communication Developers Social Network
	code.truck	Ratio of non-core developers of the collaboration Developers Social Network
	mail.only.core.devs	Number of core developers present only in the communication DSN
	code.only.core.devs	Number of core developers present only in the collaboration DSN
	ml.code.core.devs	Number of core developers present both in the communication and in the collaboration DSNs
	ratio.mail.only.core	Ratio of core developers present only in the communication DSN

Category	Metric ID	Socio-technical Quality Metric Description
	ratio.code.only.core	Ratio of core developers present only in the collaboration DSN
	ratio.ml.code.core	Ratio of core developers present both in the communication and in the collaboration DSNs
Turnover	global.turnover	Global developers turnover with respect to the previous temporal window
	code.turnover	Collaboration developers turnover with respect to the previous temporal window
	core.global.turnover	Core global developers turnover with respect to the previous temporal window
	core.mail.turnover	Core communication developers turnover with respect to the previous temporal window
	core.code.turnover	Core collaboration developers turnover with respect to the previous temporal window
	ratio.smelly.quitters	Ratio of developers previously involved in any Community Smell that left the community
Social Network Analysis metrics	closeness.centr	SNA degree metric of the global DSN computed using closeness
	betweenness.centr	SNA degree metric of the global DSN computed using betweenness
	degree.centr	SNA degree metric of the global DSN computed using degree
	global.mod	SNA modularity metric of the global DSN
	mail.mod	SNA modularity metric of the communication Developers Social Network
	code.mod	SNA modularity metric of the collaboration Developers Social Network
	density	SNA density metric of the global Developers Social Network

Classificazione qualitativa dei progetti

La classificazione qualitativa dei progetti open source in base all'ambito di utilizzo e al linguaggio di programmazione rappresenta un'opportunità per comprendere meglio le dinamiche delle comunità di sviluppo e il loro impatto sull'insorgenza dei *community smell*. I progetti open source spaziano tra una vasta gamma di applicazioni, dai sistemi operativi agli strumenti per lo sviluppo software, fino alle applicazioni scientifiche. Allo stesso modo, i linguaggi di programmazione utilizzati variano in base alle esigenze specifiche: R è ampiamente adottato nell'analisi statistica, Java è prevalente nelle applicazioni enterprise, mentre C++ è scelto per software che richiedono alte prestazioni. Classificare i progetti secondo queste dimensioni permette di evidenziare eventuali correlazioni tra determinati contesti applicativi o linguaggi e l'occorrenza di *community smell*. Ad esempio, potrebbe emergere che alcune tipologie di progetti o linguaggi tendono a favorire la formazione di particolari *smell* a causa di dinamiche tecniche o sociali peculiari. Dal punto di vista dello stato dell'arte, l'attenzione ai *community smell* nei progetti open source è crescente, con studi che hanno già messo in luce il loro impatto negativo sull'evoluzione del software. Tuttavia, l'approccio che combina la classificazione dei progetti per ambito di utilizzo e linguaggio con l'analisi delle occorrenze di *community smell* rappresenta una novità. Questa metodologia integra prospettive tecniche e organizzative, offrendo un punto di vista innovativo per indagare le interazioni tra fattori sociali e tecnologici nelle comunità open source. In conclusione, una classificazione qualitativa dei progetti open source basata sull'ambito e sul linguaggio di programmazione offre una chiave di lettura nuova e preziosa per affrontare i *community smell*. Questo approccio non solo contribuisce a colmare una lacuna nella letteratura esistente, ma guida anche verso interventi più efficaci, promuovendo la salute e la sostenibilità delle comunità di sviluppo software.

La classificazione dei progetti 21 progetti è la seguente:

Progetto	Ambito di Utilizzo	Linguaggio di Programmazione
Django	Sviluppo Web	Python
Rails	Sviluppo Web	Ruby
AngularJS	Sviluppo Web	JavaScript
Node.js	Sviluppo Web	JavaScript
Firefox	Sviluppo Web	C++, JavaScript
Python	Programmazione generale	Python
Emacs	Sviluppo Software	Lisp
IPython	Sviluppo Software	Python
Git	Gestione Versioni, CI/CD	C
GitLab	Gestione Versioni, CI/CD	Ruby, Go
Vagrant	Gestione Ambienti	Ruby
QEMU	Virtualizzazione e Testing	C
LLVM	Compilazione	C, C++
FFmpeg	Multimedia	C
Gstreamer	Multimedia	C
Blender	Grafica 3D	C, C++
Mesa	Grafica 3D	C, C++
LibreOffice	Produttività	Python

Progetto	Ambito di Utilizzo	Linguaggio di Programmazione
Bitcoin	Blockchain e Criptovalute	C++, Python
Salt	Automazione	Python, C
U-Boot	Avvio Sistemi Embedded	C

Questa classificazione ha permesso di osservare che i progetti sono distribuiti tra diversi ambiti, principalmente legati a sviluppo web, gestione versioni, virtualizzazione e testing, multimedia, grafica 3D, produttività e blockchain. Inoltre, la varietà nei linguaggi di programmazione, tra cui Python, Ruby, JavaScript, C, C++, Go e Lisp, indica una notevole diversità nelle tecnologie utilizzate.

Analisi delle relazioni di causalità tra i Community Smell tramite il Test di Granger

I *community smell* sono segnali di problematiche organizzative e comunicative nei team di sviluppo software. Comprendere le relazioni tra di essi è fondamentale per intervenire in modo efficace. Spesso, infatti, uno *smell* può causarne altri, creando una catena di problemi. Identificare queste relazioni causali consente di agire sulle cause principali, eliminando uno *smell* che potrebbe generare o amplificare altri due o più problemi. Il test di Granger è uno strumento statistico utile per rilevare relazioni di causalità tra serie temporali. A differenza della semplice correlazione, questo test permette di verificare se i cambiamenti in uno *community smell* anticipano e "causano" cambiamenti in un altro. Il processo parte dall'analisi delle serie temporali associate agli *smell*, verificando requisiti come la stazionarietà, per poi applicare il test e identificare legami causali. Il beneficio principale è la possibilità di ottimizzare le azioni correttive: se uno *smell* causa altri due, intervenire su di esso riduce indirettamente l'insorgenza degli altri. Questo approccio migliora la gestione delle dinamiche della comunità, prevenendo problemi a cascata e supportando un ambiente di sviluppo più sano ed efficiente.

La prima analisi ha l'obiettivo di esplorare le possibili relazioni causali tra diversi community smell all'interno di un dataset costituito da 21 progetti open source di grandi dimensioni.

RQ1: Un community smell può causarne un altro?

Lo studio verte sull'identificazione di potenziali relazioni causali tra i community smell riscontrati, esaminando se la presenza di uno possa essere un indicatore per la futura comparsa o aggravamento di un altro.

Il processo analitico è stato suddiviso nelle seguenti fasi:

1. Preparazione del dataset:
Ogni progetto è stato rappresentato tramite un file CSV che contiene le metriche trimestrali (12 periodi), focalizzandosi sui cinque community smell di interesse. La struttura temporale dei dati (periodo trimestrale) permette di esplorare le dinamiche evolutive di ciascun community smell nel tempo.
2. Verifica della stazionarietà:
Poiché il test di causalità di Granger richiede che le serie temporali siano stazionarie, è stato necessario verificare ed eventualmente trasformare i dati in modo da soddisfare questa

condizione. La stazionarietà di una serie temporale implica che le sue proprietà statistiche, come media e varianza, siano costanti nel tempo. Se le serie non fossero stazionarie, il test di Granger potrebbe fornire risultati fuorvianti. Per valutare la stazionarietà, è stato utilizzato il test di Dickey-Fuller aumentato (ADF). Questo test verifica la presenza di una radice unitaria in una serie temporale, cioè se la serie è stazionaria o meno. La motivazione dietro l'uso di questo test è che, senza stazionarietà, i modelli di previsione basati sulla causalità potrebbero non essere validi, poiché le caratteristiche dei dati cambiano nel tempo.

- Se il test ADF indica che la serie non è stazionaria, la serie deve essere differenziata sottraendo il valore precedente da ogni dato. Questo processo di differenziazione aiuta a stabilizzare la varianza e rende la serie stazionaria.
- Nel caso in cui una serie avesse varianza nulla (cioè costante nel tempo), è stata esclusa dall'analisi, in quanto non avrebbe avuto alcuna informazione utile per il test di Granger.

3. Applicazione del test di Granger:

Una volta verificate le condizioni di stazionarietà, è stato eseguito il test di causalità di Granger su ogni coppia di community smell per ciascun progetto. Sono state testate tutte le coppie di community smell per verificare se le occorrenze di uno (smell1) potessero predire quelle di un altro (smell2) nel periodo successivo. Il massimo ritardo (lag) è stato impostato pari a 1 periodo, per evitare una complessità eccessiva nel modello e garantire che l'effetto della causa fosse rilevabile nel breve termine. Il test di Granger in sostanza verifica se l'inclusione di una variabile (smell1) migliora significativamente la previsione di un'altra variabile (smell2). Se il p-value associato al test è inferiore a una soglia di significatività (di solito 0.05), si rifiuta l'ipotesi nulla (smell1 non causa smell2) e si conclude che c'è evidenza che uno smell può causare l'altro.

4. Report dei risultati:

I risultati significativi sono stati identificati tramite un valore di p-value inferiore a 0.05. Per ogni progetto, i risultati sono stati salvati in un file di output denominato `granger_results.txt`. Ogni record nel file include la coppia di smell analizzata, il p-value associato e una breve descrizione del test. I risultati hanno permesso di identificare le relazioni causali più forti tra i community smell in ciascun progetto.

Risultati RQ1

Un test con campioni molto piccoli tende ad avere una bassa potenza, cioè una bassa probabilità di rilevare un effetto reale, ma potrebbe comunque produrre risultati falsi positivi. Anche se il p-value risulta valido, una bassa frequenza di occorrenze può portare a una stima poco affidabile delle relazioni di causalità. Alla luce di queste considerazioni abbiamo deciso di escludere dai risultati validi le relazioni di causalità che interessavano *Prima Donnas* e *Black Cloud* le cui occorrenze risultavano quasi nulle nel dataset. Inoltre, sono state ritenute rilevanti solo le relazioni individuate in almeno 2 progetti. I risultati dei test di Granger mostrano quali community smell possono influenzarsi a vicenda. Analizzando le relazioni più ricorrenti e significative, possiamo identificare alcuni pattern interessanti.

1. Relazione **Radio Silence** → **Organizational Silo**

Descrizione: In 4 progetti il test suggerisce che *Radio Silence* causa *Organizational Silo*. Ciò potrebbe indicare che il silenzio comunicativo (assenza di interazione tra gruppi o persone) favorisce l'isolamento organizzativo (Org. silo).

Interpretazione: La mancanza di comunicazione tra gruppi o individui in un progetto open source tende a creare compartimenti stagni (silos), dove i team lavorano isolati l'uno dall'altro.

2. Relazione **Radio.silence** → **Missing.links**

Descrizione: In 4 progetti è presente questa relazione. Radio.silence causa Missing.links indica che il silenzio comunicativo contribuisce direttamente alla mancanza di collegamenti tra componenti o team.

Interpretazione: Senza comunicazione, i team non riescono a creare connessioni significative o a collaborare efficacemente, aumentando il rischio di frammentazione.

3. Relazione **Org.silo** → **Missing.links**

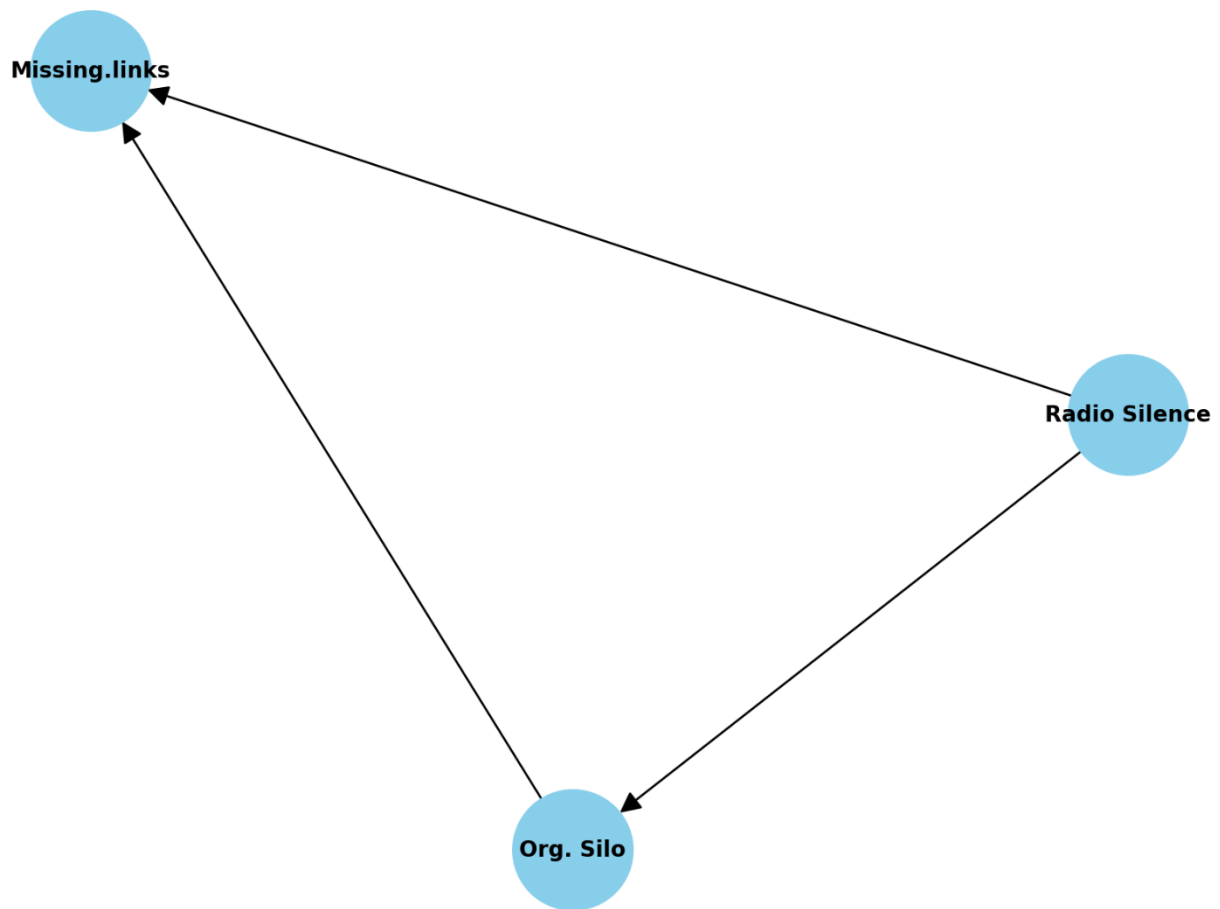
Descrizione: In 2 progetti analizzati, la presenza di "Organizational Silo" (Org.silo) è stata identificata come causa di "Missing Links". La frammentazione dei team e la formazione di silo organizzativi potrebbero favorire l'assenza di comunicazioni essenziali tra i contributori software.

Interpretazione: Questa relazione evidenzia come l'isolamento organizzativo contribuisca all'insorgere di *Missing Links*, compromettendo la coesione tra moduli e aumentando il rischio di difetti e difficoltà di manutenzione. Mitigare *Organizational Silo* potrebbe prevenire tali effetti e migliorare la qualità del progetto.

Implicazioni

Questi risultati suggeriscono che per ridurre i community smell in un progetto, potrebbe essere strategico iniziare migliorando la comunicazione (riducendo Radio.silence) e quindi di conseguenza garantire collegamenti tra team o componenti evitando Missing.links e Organizational.silo che risultano essere una conseguenza di Radio Silence.

Grafo delle relazioni tra Community Smells



Risultati per categoria di progetto

Un ulteriore contributo innovativo all'analisi deriva dalla classificazione qualitativa dei progetti open source utilizzati nello studio, basata sull'ambito di utilizzo e sul linguaggio di programmazione. Questo tipo di classificazione permette di contestualizzare i risultati dell'analisi di Granger, evidenziando come l'insorgenza e le relazioni causali tra i *community smell* possano variare in base alla natura dei progetti. Ad esempio, i progetti scientifici sviluppati in Python potrebbero mostrare dinamiche causali diverse rispetto a quelli enterprise in Java, a causa di fattori tecnici e sociali legati ai rispettivi contesti applicativi. Questa integrazione offre diversi benefici. In primo luogo, consente di identificare pattern specifici per determinate categorie di progetti, rendendo i risultati dell'analisi più mirati e utili. Inoltre, aiuta a progettare interventi personalizzati per mitigare gli effetti dei *community smell* in base alle caratteristiche del progetto.

Sviluppo Web

Rails:

radio.silence causa missing.links (p-value 0.013)

radio.silence causa org.silo (p-value 0.014)

Node.js:

radio.silence causa missing.links (p-value 0.029)

radio.silence causa org.silo (p-value 0.028)

Programmazione

Python:

org.silo causa missing.links (p-value=0.000)

Sviluppo Software

IPython:

org.silo causa missing.links (p-value=0.046)

Grafica 3D

Blender:

radio.silence causa missing.links (p-value=0.000)

radio.silence causa org.silo (p-value=0.000)

Avvio Sistemi Embedded

U-Boot:

radio.silence causa org.silo (p-value 0.026)

Gestione Versioni, CI/CD

Git:

radio.silence causa missing.links (p-value 0.020)

Considerazioni: un risultato rilevante che possa mettere in relazione la categoria di appartenenza con i legami di causalità rilevati, è quello relativo alla categoria *Sviluppo Web* in cui troviamo una relazione causale tra *Radio Silence* e *Missing Links* e una tra *Radio Silence* e *Organizational Silo* per 2 progetti appartenenti a questa categoria. Un ulteriore risultato degno di nota è quello riguardante i progetti *Python* e *iPython* accomunati dallo stesso tipo di linguaggio di programmazione e che presentano una relazione causale tra *Organizational Silo* e *Missing Links*.

RQ 1.1: Esiste una relazione causale tra le metriche di turnover e i Community Smell?

Nei progetti open-source, le comunità di sviluppo sono dinamiche e soggette a continui cambiamenti, gli sviluppatori entrano ed escono, e il livello di coinvolgimento individuale varia nel tempo. Questi cambiamenti, definiti collettivamente come turnover, possono influenzare in modo significativo le interazioni all'interno della comunità, contribuendo all'emergere di community smells. L'obiettivo di questa RQ è indagare se la presenza di metriche di turnover dei membri del team possa fungere da indicatore predittivo per l'evoluzione dei community smell in periodi successivi. In particolare, individuare singolarmente quale metrica di turnover possa causare uno specifico smell. Applicando il test di Granger, è possibile identificare se variazioni in specifiche metriche di turnover, come il turnover globale degli sviluppatori (*global.turnover*) o il turnover degli sviluppatori core (*core.global.turnover*), precedono cambiamenti nei *community smell*. Questo tipo di analisi consente di individuare pattern temporali e potenziali relazioni di causa-effetto, fornendo una comprensione più profonda delle dinamiche che influenzano la qualità e l'efficienza dei progetti open source.

I benefici derivanti dall'utilizzo del test di Granger in questo contesto includono:

- **Prevenzione proattiva:** Comprendere le relazioni causali consente di anticipare e mitigare l'insorgenza di *community smell*, migliorando la salute complessiva del progetto.
- **Ottimizzazione delle risorse:** Intervenire su metriche di turnover chiave può prevenire effetti negativi a catena, rendendo più efficiente la gestione della comunità.

Nell'ambito della ricerca accademica, l'applicazione del test di Granger per analizzare le relazioni tra turnover degli sviluppatori e *community smell* rappresenta un approccio innovativo. Sebbene studi precedenti abbiano esplorato l'impatto del turnover sugli aspetti organizzativi dei progetti open source, l'utilizzo specifico del test di Granger per determinare relazioni causali direzionali tra queste variabili è ancora relativamente nuovo. Questo approccio offre una prospettiva più dettagliata sulle dinamiche temporali e causali all'interno delle comunità di sviluppo, contribuendo a colmare una lacuna nella letteratura esistente.

Risultati Attesi:

Sappiamo dalla letteratura che le metriche di turnover sono indicatori di community smell, lo studio effettuato mira a trovare le relazioni causali tra questi fattori e se queste possano essere accomunate dall'appartenenza alla stessa categoria di progetti coinvolti.

Risultati RQ1.1

1. Relazione *global.turnover* → *radio.silence*

Descrizione:

In 6 progetti (Mesa, Nodejs, Rails, LibreOffice, iPython, LLVM) il test suggerisce che il *global.turnover* causa il community smell *Radio Silence*. Ciò potrebbe indicare che un'elevata sostituzione dei devs, probabilmente legata a instabilità o cambiamenti frequenti nella struttura del team, favorisce la creazione di periodi di inattività nelle comunicazioni del progetto.

Interpretazione:

Questa relazione evidenzia l'importanza della stabilità del team nel mantenere una comunicazione costante. Ridurre la frequenza di turnover potrebbe aiutare a prevenire episodi di *Radio Silence*, migliorando la collaborazione e il flusso comunicativo all'interno della comunità del progetto.

2. Relazione *code.turnover* → *radio.silence*

Descrizione:

In 5 progetti (GitLab, FFmpeg, Nodejs, Rails, iPython) il test suggerisce che il *code turnover* causa *Radio Silence*. Ciò potrebbe indicare che la frequente sostituzione dei contributori principali che lavorano sul codice può portare a lacune comunicative all'interno del progetto.

Interpretazione:

L'attenzione alla gestione delle competenze chiave e alla documentazione del codice può aiutare a ridurre le discontinuità comunicative, mitigando l'insorgere di *Radio Silence* anche in presenza di turnover del codice.

3. Relazione ratio.smelly.quitters → org.silo**Descrizione:**

In 5 progetti (U-boot, Django, Nodejs, Rails, Bitcoin) il test suggerisce che il rapporto di contributori "smelly" che abbandonano il progetto causa *Organizational Silo*. Ciò potrebbe indicare che l'uscita di figure problematiche dal progetto porta alla frammentazione organizzativa.

Interpretazione:

La relazione suggerisce che i contributori con comportamenti negativi potrebbero aver già contribuito alla creazione di *Organizational Silo*. La loro uscita, pur essendo positiva a lungo termine, potrebbe lasciare un'eredità strutturale di isolamento tra le sottocomunità del progetto.

4. Relazione global.turnover → org.silo**Descrizione:**

In 6 progetti (GitLab, FFmpeg, Nodejs, Rails, LibreOffice, Gstreamer) il test suggerisce che il *global.turnover* causa *Organizational Silo*. Ciò potrebbe indicare che i frequenti cambiamenti nella composizione del team contribuiscono all'insorgere di isolamenti organizzativi tra i vari gruppi del progetto.

Interpretazione:

L'implementazione di processi di integrazione più solidi potrebbe ridurre l'impatto del turnover globale sullo sviluppo di *Organizational Silo*, favorendo una maggiore coesione e comunicazione tra le sottocomunità.

5. Relazione ratio.smelly.quitters → missing.links**Descrizione:**

In 4 progetti (Django, Emacs, Nodejs, Bitcoin) il test suggerisce che il rapporto di contributori "smelly" che abbandonano il progetto causa il community smell *Missing Links*. Ciò potrebbe indicare che questi abbandoni contribuiscono a un indebolimento della rete di relazioni tra i contributori.

Interpretazione:

Per mitigare questo effetto, potrebbe essere utile rafforzare le connessioni tra i membri rimasti nel progetto, favorendo pratiche di team-building e collaborazione per ricostruire una rete comunicativa robusta.

6. Relazione code.turnover → missing.links**Descrizione:**

In 4 progetti (GitLab, FFmpeg, LLVM, Bitcoin) il test suggerisce che il *code turnover* causa il community smell *Missing Links*. Ciò potrebbe indicare che i cambiamenti frequenti nei contributori del codice riducono le interazioni e la coesione all'interno del team.

Interpretazione:

Un migliore passaggio delle conoscenze e la promozione di una cultura di collaborazione tecnica potrebbero aiutare a prevenire la perdita di connessioni significative tra i contributori, riducendo così l'insorgenza di *Missing Links*.

Riepilogo dei Pattern Identificati

1. Impatto del Turnover: Il turnover, sia globale che specifico al codice, è frequentemente associato a community smells come *radio.silence* e *org.silo*. Questo suggerisce che un turnover elevato possa frammentare la comunicazione e creare divisioni all'interno del team.
 2. "Smelly Quitters" e Isolamento: I contributori problematici che abbandonano il progetto sono legati a *missing.links* e *org.silo*, indicando che la loro uscita può intensificare l'isolamento tra i membri del team e ridurre le connessioni tra i gruppi.
-

Risultati per categoria di progetto

Tale classificazione consente di contestualizzare le metriche di turnover in relazione al tipo di progetto open source analizzato, evidenziando come fattori come il linguaggio di programmazione o l'ambito di utilizzo possano influenzare le dinamiche di turnover e la comparsa di *community smell*. Ad esempio, i progetti sviluppati in linguaggi come Python, comunemente usati per progetti scientifici, potrebbero manifestare un turnover diverso rispetto a progetti in Java o C++ orientati a soluzioni enterprise. Questa integrazione offre vantaggi significativi, in quanto permette di identificare pattern specifici e relazioni causali più precise, legate alle caratteristiche dei progetti. In conclusione, l'analisi delle metriche di turnover tramite il test di Granger, arricchita dalla classificazione qualitativa dei progetti, offre una visione più completa e mirata delle relazioni causali tra turnover e *community smell*. Questo approccio innovativo fornisce strumenti utili per migliorare la gestione dei progetti open source e promuovere la stabilità e la sostenibilità delle comunità di sviluppo.

Sviluppo Web

Django:

core.global.turnover causa radio.silence (p-value=0.000)
core.code.turnover causa radio.silence (p-value=0.014)
ratio.smelly.quitters causa radio.silence (p-value=0.000)
ratio.smelly.quitters causa missing.links (p-value=0.005)
ratio.smelly.quitters causa org.silo (p-value=0.046)

Rails:

global.turnover causa radio.silence (p-value=0.006)
code.turnover causa radio.silence (p-value=0.010)
core.code.turnover causa radio.silence (p-value=0.000)
ratio.smelly.quitters causa missing.links (p-value=0.000)
global.turnover causa org.silo (p-value=0.048)
ratio.smelly.quitters causa org.silo (p-value=0.000)

Node.js:

global.turnover causa radio.silence (p-value=0.000)
code.turnover causa radio.silence (p-value=0.000)
core.code.turnover causa radio.silence (p-value=0.001)
global.turnover causa missing.links (p-value=0.011)
ratio.smelly.quitters causa missing.links (p-value=0.005)
global.turnover causa org.silo (p-value=0.014)
ratio.smelly.quitters causa org.silo (p-value=0.005)

Firefox:

core.mail.turnover causa radio.silence (p-value=0.035)

Sviluppo Software

Emacs:

ratio.smelly.quitters causa missing.links (p-value=0.040)

IPython:

global.turnover causa radio.silence (p-value=0.018)

code.turnover causa radio.silence (p-value=0.009)

core.global.turnover causa radio.silence (p-value=0.004)

core.mail.turnover causa radio.silence (p-value=0.007)

Gestione Versioni CI/CD

GitLab:

global.turnover causa missing.links (p-value=0.045)

code.turnover causa missing.links (p-value=0.021)

global.turnover causa org.silo (p-value=0.036)

code.turnover causa org.silo (p-value=0.015)

Multimedia

FFMpeg:

code.turnover causa radio.silence (p-value=0.011)

code.turnover causa missing.links (p-value=0.004)

global.turnover causa org.silo (p-value=0.045)

code.turnover causa org.silo (p-value=0.001)

GStreamer:

ratio.smelly.quitters causa radio.silence (p-value=0.000)

global.turnover causa missing.links (p-value=0.026)

global.turnover causa org.silo (p-value=0.011)

code.turnover causa org.silo (p-value=0.014)

core.mail.turnover causa org.silo (p-value=0.039)

Grafica 3D

Blender:

core.code.turnover causa radio.silence (p-value=0.013)

Mesa:

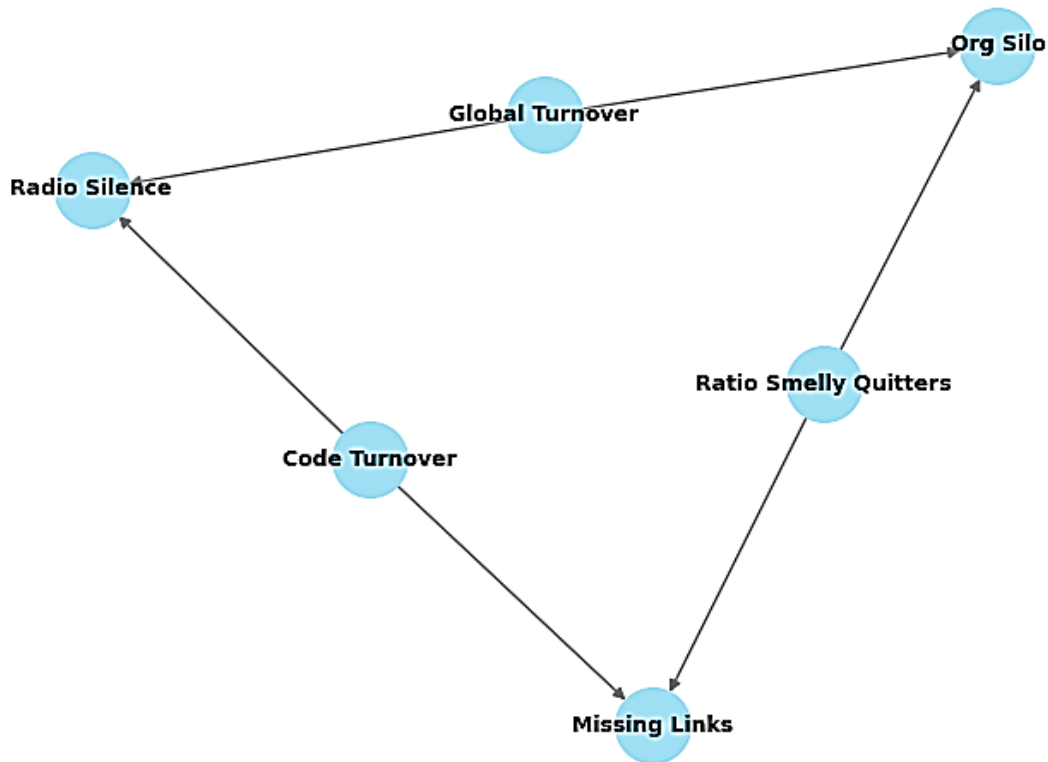
global.turnover causa radio.silence (p-value=0.005)

core.global.turnover causa radio.silence (p-value=0.030)

core.mail.turnover causa radio.silence (p-value=0.018)

ratio.smelly.quitters causa radio.silence (p-value=0.033)

Considerazioni: la categoria che presenta più occorrenze comuni di relazioni causali, come nella RQ precedente, risulta essere *Sviluppo Web*. Troviamo due occorrenze comuni anche per la categoria di progetto *Multimedia*. Risulta dunque di particolare interesse soprattutto la categoria *Sviluppo Web* in cui abbiamo ritrovato comportamenti comuni tra i progetti di appartenenza sia nella RQ1 che nella RQ1.1.



Andamento Temporale delle Occorrenze dei Community Smell

Il processo di analisi temporale si concentra sull'osservazione di come le metriche relative ai community smell variano nel tempo, cercando pattern, picchi o correlazioni con eventi o cambiamenti significativi nel progetto. Questo tipo di analisi consente di individuare quando e come si manifestano determinati smell, aiutando a prevenire effetti negativi più ampi che potrebbero compromettere il progetto nel lungo termine. La classificazione dei progetti open source permette di contestualizzare meglio l'evoluzione dei community smell, in quanto progetti con obiettivi e caratteristiche diverse possono presentare dinamiche temporali differenti. Ad esempio, i progetti di sviluppo web potrebbero mostrare pattern di smell differenti rispetto a progetti di Multimedia, a causa delle specifiche esigenze, risorse e pratiche di sviluppo legate a ciascun ambito. I benefici di un'analisi temporale contestualizzata sono molteplici: in primo luogo, consente di identificare con maggiore precisione le fasi di un progetto in cui i community smell sono più problematici, permettendo di intervenire in modo mirato e tempestivo. In secondo luogo, la combinazione con la classificazione qualitativa dei progetti consente di sviluppare strategie personalizzate per mitigare i smell, a seconda delle caratteristiche specifiche del progetto. Questo approccio facilita la gestione delle dinamiche sociali e organizzative, migliorando la collaborazione e la qualità complessiva del software.

RQ2: Esistono pattern ricorrenti nell'andamento temporale delle occorrenze dei community smell nei progetti open source anche in relazione alle categorie qualitative definite?

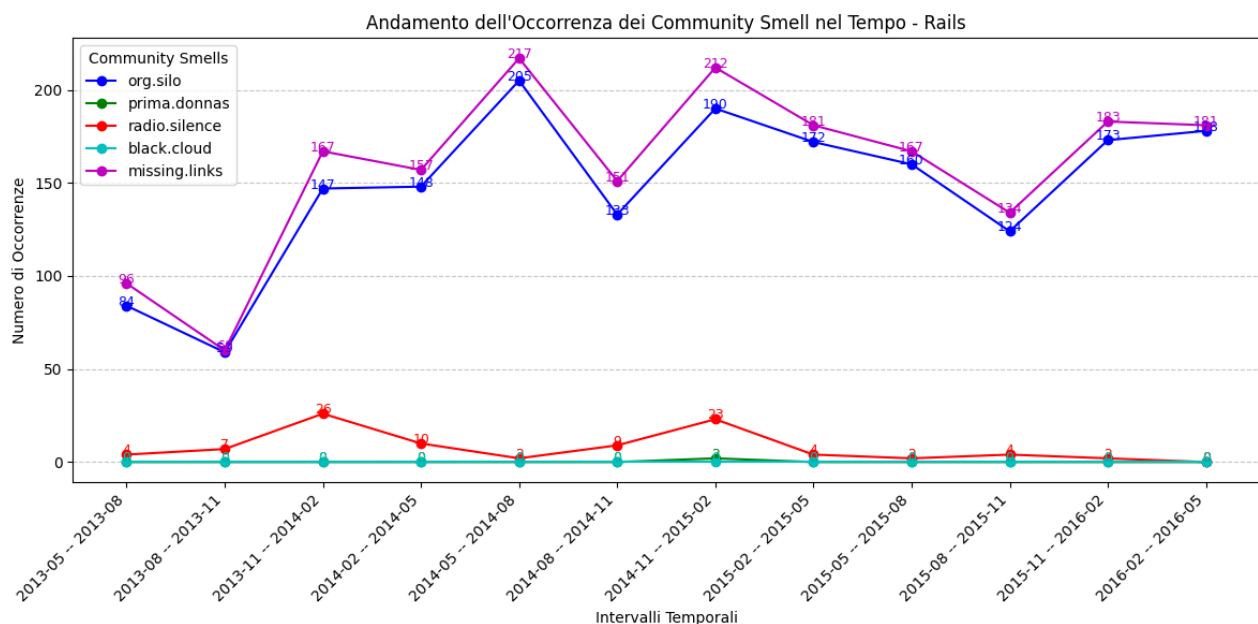
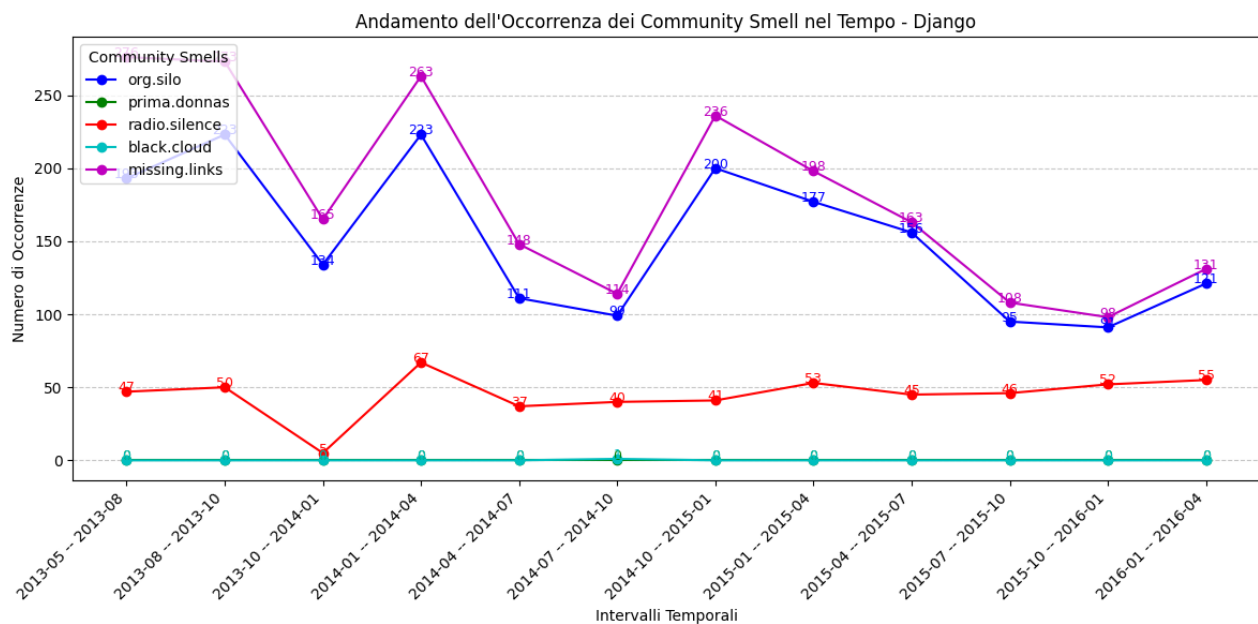
Metodologia:

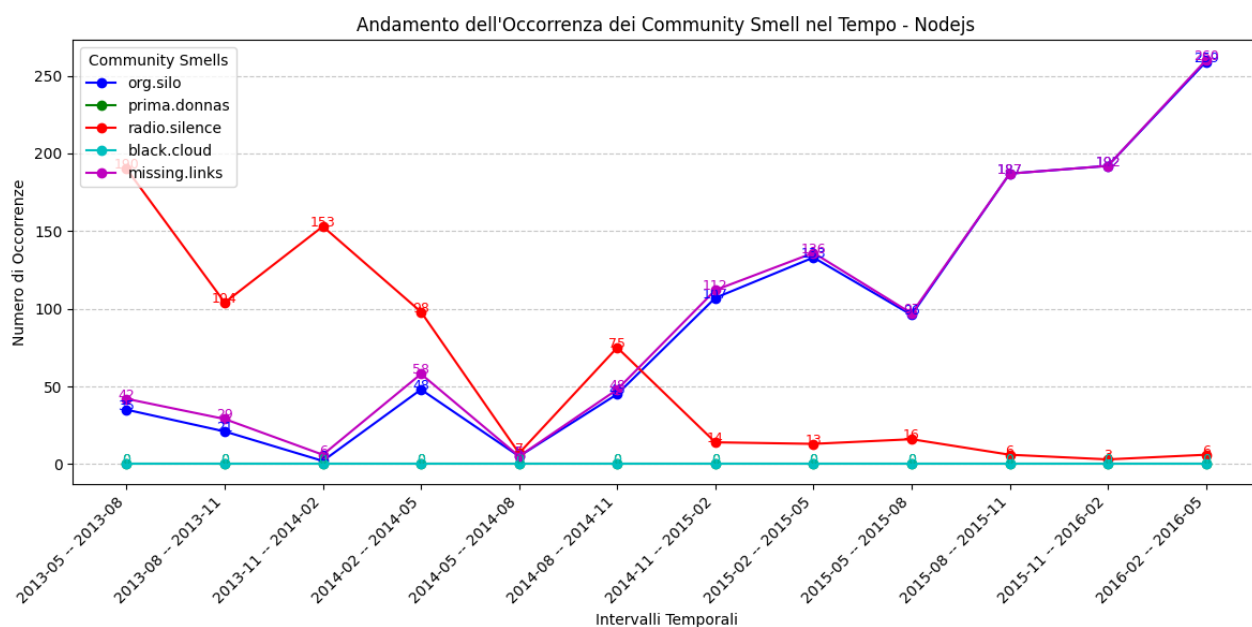
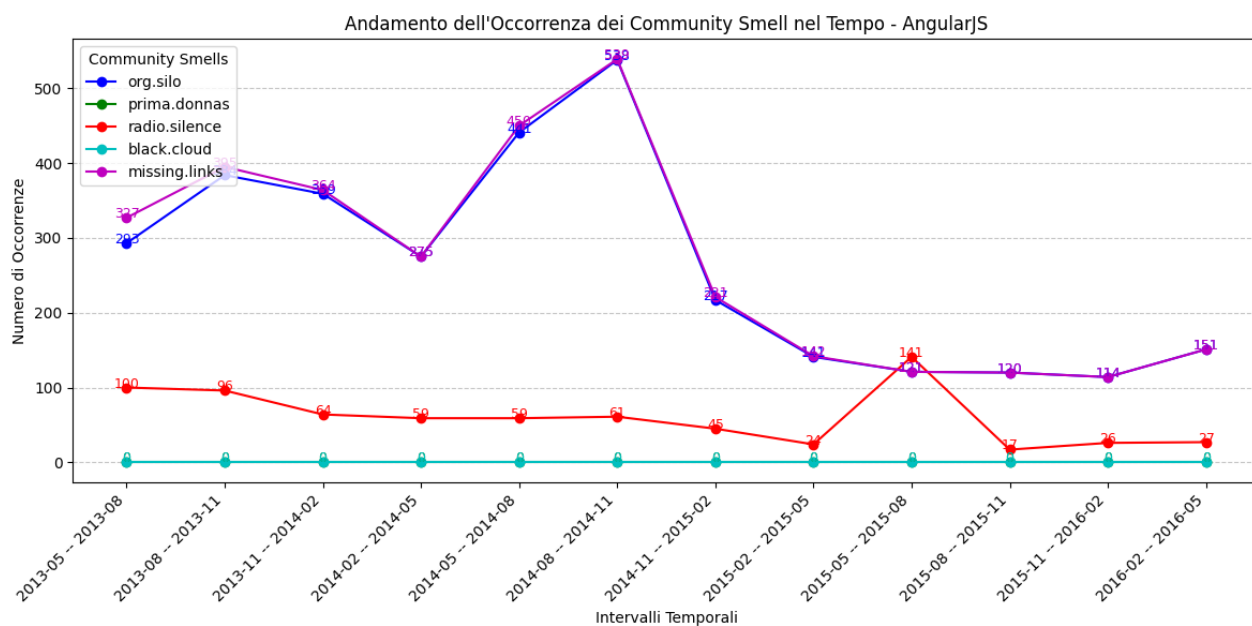
- **Analisi Temporale:** Per ciascun progetto, sono state tracciate le occorrenze dei community smell in 12 periodi temporali (ogni periodo corrisponde a un trimestre), per esaminare come queste variano nel tempo.
- **Strumento di visualizzazione:** È stato utilizzato uno script Python che raccoglie i dati dai file CSV dei report e visualizza l'andamento delle occorrenze nel tempo per ciascun community smell.

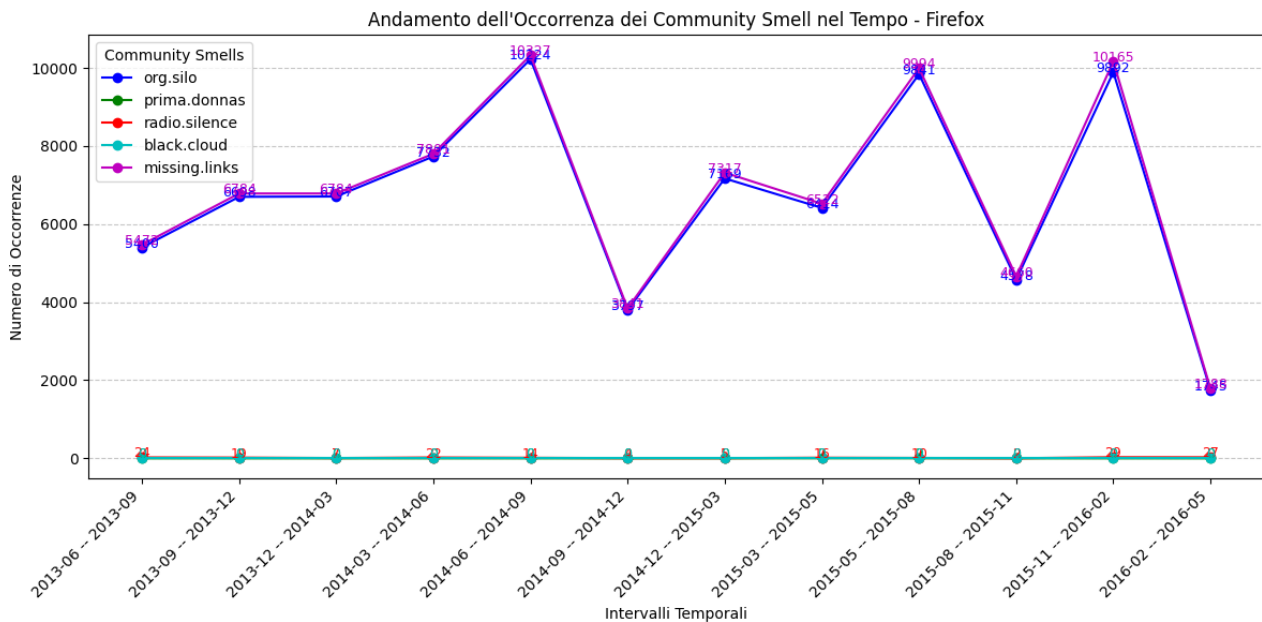
Risultati Attesi RQ2

- **Identificazione di Pattern:** Ci si aspetta che possano emergere determinati pattern, come l'aumento o la diminuzione costante di alcune problematiche (ad esempio, Radio.silence o Missing.links) durante i periodi esaminati. Questi pattern potrebbero indicare dinamiche specifiche all'interno dei team di sviluppo, come miglioramenti o regressioni nelle pratiche comunicative e di collaborazione.

Di seguito sono presentati i grafici che illustrano l'andamento temporale delle occorrenze dei community smell per i progetti appartenenti alla categoria “Sviluppo Web”. Ogni grafico rappresenta l'evoluzione dei community smell in 12 periodi temporali. L'analisi si concentra su come gli smell considerati variano nel tempo, fornendo una visione delle dinamiche di ciascun progetto open source nel corso dei periodi esaminati. I grafici sono utili per osservare fluttuazioni, miglioramenti o regressioni nelle occorrenze dei community smell, che potrebbero essere indicative di modifiche nel processo di sviluppo, nelle pratiche comunicative o nelle modifiche strutturali interne al progetto. La presenza di picchi o periodi di stabilità potrebbe fornire importanti informazioni sulla salute del progetto e sull'efficacia delle pratiche adottate dai team di sviluppo.







Osservazioni generali

Dall'analisi qualitativa dei grafici che mostrano l'andamento delle occorrenze dei community smell nel tempo emerge in molti progetti una relazione tra *Organizational silo* e *Missing links*. È possibile, quindi, definire 3 categorie:

1. Sovrapposizione Perfetta (P):

- Sviluppo Web: I progetti appartenenti a questa categoria (Django, Rails, AngularJS, Node.js, Firefox) presentano un andamento temporale perfettamente uguale (P) tra **org.silo** e **missing.links**. Ciò implica che, per questi progetti, la presenza di silos organizzativi sembra andare di pari passo con la mancanza di connessioni tra i vari componenti o team.
- Sviluppo Software: anche i progetti *Emacs* e *iPython* appartenenti alla stessa categoria presentano lo stesso tipo di andamento temporale tra *Organizational Silo* e *Missing Links*.
- *GitLab*, *Vagrant*, *LibreOffice*, *Salt*: questi progetti non appartengono alla stessa categoria ma presentano lo stesso una sovrapposizione dei due andamenti temporali.

2. Andamento Simile (X):

- LLVM, FFmpeg, Gstreamer, Blender, Bitcoin, U-boot, IPython: Per questi progetti, l'andamento temporale tra **org.silo** e **missing.links** è simile ma non uguale (X), suggerendo che c'è una relazione tra questi due smell, ma non sono esattamente sovrapponibili anche se l'andamento in termini qualitativi rimane il medesimo.

3. Nessuna Sovrapposizione:

- Python, Git, QEMU, Mesa: Questi progetti non presentano una correlazione evidente tra **org.silo** e **missing.links**. In questi casi, i due smell si sviluppano separatamente, suggerendo che i problemi organizzativi (come la separazione dei team) non sono strettamente legati alla mancanza di connessioni tra le diverse componenti del progetto.

Interpretazioni e Implicazioni RQ2

1. **Sovrapposizione nei progetti di sviluppo web:** La sovrapposizione perfetta nei progetti di Sviluppo Web (Django, Rails, AngularJS, Node.js, Firefox) potrebbe suggerire che i progetti web soffrano particolarmente di problematiche legate alla divisione delle responsabilità tra team, dove ciascun gruppo si occupa di una parte isolata (come front-end, back-end, e API) senza una connessione sufficiente tra di loro. Questo fenomeno potrebbe ridurre la qualità della comunicazione e dell'integrazione tra i vari team, aumentando il rischio di missing links. I progetti web potrebbero trarre vantaggio da una maggiore attenzione alla comunicazione inter-team e alla riduzione dei silos organizzativi per migliorare l'integrazione tra i componenti del progetto.
2. **Progetti con andamento simile:** in questi progetti l'analogia tra i due smell potrebbe suggerire che la mancanza di comunicazione o l'isolamento tra i team possano, in modo indiretto, influenzare la creazione di dipendenze o mancanze di connessioni tra le diverse aree del progetto, senza essere direttamente correlati.
3. **Progetti senza sovrapposizione:** un andamento temporale diverso tra org.silo e missing.links potrebbe suggerire che la struttura organizzativa e il contesto in cui operano tali progetti incidono sulla relazione tra questi due smell.

Di seguito è riportata la tabella aggiornata con la colonna “Andamento temporale **org.silo – missing links**”

Progetto	Ambito di Utilizzo	Linguaggio di Programmazione	Andamento temporale org.silo - missing.links
Django	Sviluppo Web	Python	P
Rails	Sviluppo Web	Ruby	P
AngularJS	Sviluppo Web	JavaScript	P
Node.js	Sviluppo Web	JavaScript	P
Firefox	Sviluppo Web	C++, JavaScript	P
Python	Programmazione generale	Python	
Emacs	Sviluppo Software	Lisp	P
IPython	Sviluppo Software	Python	P
Git	Gestione Versioni, CI/CD	C	
GitLab	Gestione Versioni, CI/CD	Ruby, Go	P
Vagrant	Gestione Ambienti	Ruby	P
QEMU	Virtualizzazione e Testing	C	
LLVM	Compilazione	C, C++	X
FFmpeg	Multimedia	C	X
Gstreamer	Multimedia	C	X
Blender	Grafica 3D	C, C++	X
Mesa	Grafica 3D	C, C++	
LibreOffice	Produttività	Python	P
Bitcoin	Blockchain e Criptovalute	C++, Python	X
Salt	Automazione	Python, C	P
U-Boot	Avvio Sistemi Embedded	C	X

Analisi della Categorizzazione dei Commit nei Progetti Open Source (Proof of Concept)

L'analisi temporale delle occorrenze dei *community smell* in relazione alla tipologia di commit nei progetti open source rappresenta un approccio avanzato per esplorare come le dinamiche sociali e tecniche si intrecciano nel corso del tempo. Studiando come le diverse tipologie di commit (ad esempio, commit di bugfix, commit di nuove funzionalità o commit di refactoring) si distribuiscono nel tempo insieme agli *community smell*, possiamo ottenere informazioni più precise su come le attività di sviluppo influenzano la salute sociale del progetto. Studi recenti hanno cercato di mappare i *community smell* all'interno di comunità open source, ma l'integrazione con le dinamiche temporali dei commit e l'esame dettagliato di diverse tipologie di commit rappresentano un'area di ricerca ancora poco esplorata. La combinazione di questi due aspetti permette di identificare pattern ricorrenti e di comprendere come il tipo di attività tecnica (ad esempio, l'aggiunta di una nuova funzionalità o il refactoring del codice) possa influire direttamente sulle dinamiche sociali e viceversa. Nonostante non siano emersi risultati chiari riguardo a una relazione diretta tra tipologia di commit e specifici *community smell*, l'approccio offre una base per approfondire la comprensione di queste dinamiche. Questo studio, sebbene presentato come un proof of concept, evidenzia il valore di una classificazione qualitativa dei commit, che può rivelarsi utile per identificare pattern nascosti e migliorare l'analisi delle problematiche delle comunità.

RQ3: Esiste una relazione tra la tipologia di commit e l'emergere di determinati *community smells* in progetti open source?

Questa domanda si propone di esplorare se la tipologia di modifiche al codice, come l'aggiunta di nuove funzionalità, la correzione di bug o il refactoring, abbia un impatto sulla comparsa di determinati *community smells*, e se alcune tipologie di commit siano più propense a generare o risolvere tali problemi nel lungo periodo.

Metodologia

Per rispondere a questa domanda è stato implementato uno script che per ogni progetto analizza i commit di ogni trimestre presente del dataset e ne effettua una categorizzazione. Lo script categorizza automaticamente i commit in base alle parole chiave presenti nei *commit message*. Le categorie definite per l'analisi sono:

- **Security:** Modifiche mirate a migliorare la sicurezza del codice, come patch a vulnerabilità o implementazione di misure di protezione.
- **Experimental/Prototype:** Commit relativi a tentativi iniziali, esperimenti o prototipi di funzionalità non definitive.
- **Rollback:** Ripristino di versioni precedenti del codice a causa di bug o problemi imprevisti nelle modifiche recenti.
- **Performance Improvements:** Ottimizzazioni per migliorare le prestazioni del sistema, come ridurre i tempi di risposta o l'uso delle risorse.
- **Dependencies:** Aggiornamenti o modifiche nelle librerie, pacchetti o altre dipendenze utilizzate nel progetto.
- **Style/Formatting:** Modifiche al formato del codice, come la correzione di indentazioni, spaziatura o altri aspetti stilistici.
- **Build/Configuration:** Modifiche ai file di configurazione o ai processi di build, inclusi script di compilazione e configurazioni ambientali.

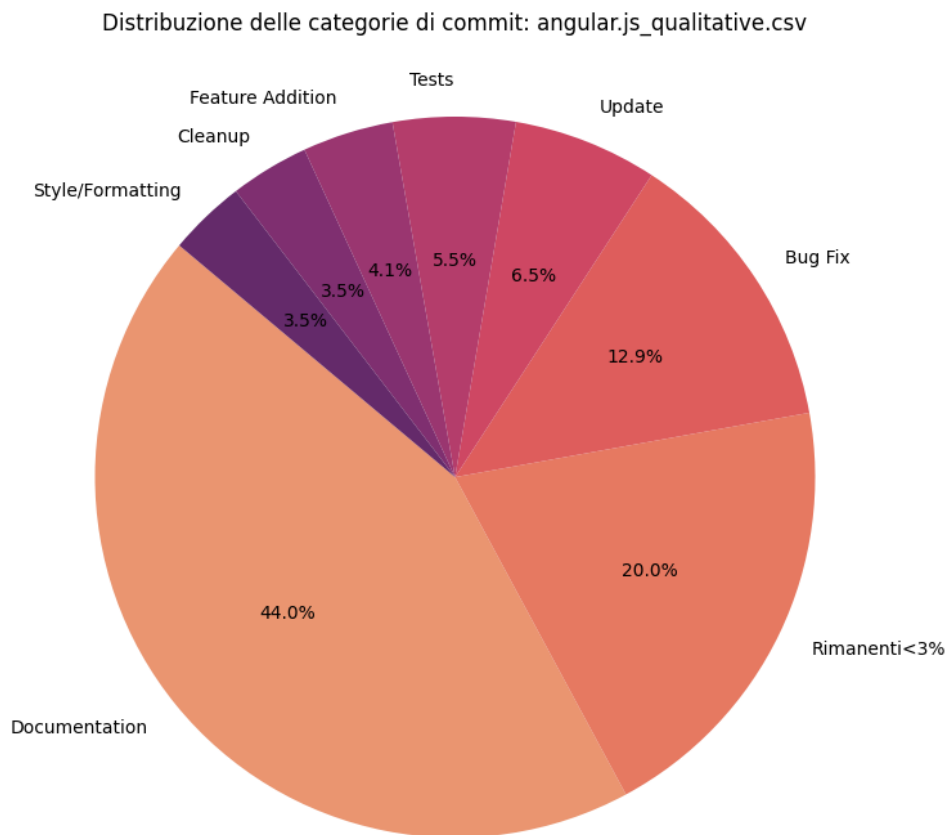
- Merge/Conflict Resolution: Risoluzione di conflitti durante il processo di merge tra rami di sviluppo diversi.
- Tests: Aggiunta, modifica o rimozione di test automatizzati o manuali per verificare la correttezza del codice.
- Cleanup: Rimozione di codice obsoleto o non più necessario, migliorando la leggibilità e la manutenzione del progetto.
- Refactoring: Ristrutturazione del codice esistente per migliorarne la qualità, la leggibilità o la manutenibilità, senza modificarne il comportamento.
- Major Release: Aggiornamenti significativi che introducono nuove funzionalità o cambiamenti incompatibili con le versioni precedenti.
- Minor Release: Modifiche che aggiungono funzionalità compatibili con le versioni precedenti, senza interrompere il comportamento esistente.
- Update: Aggiornamenti generali, che possono includere modifiche a funzionalità esistenti o miglioramenti minori.
- Documentation: Modifiche ai documenti di progetto, come README, guide o commenti nel codice.
- Feature Addition: Aggiunta di nuove funzionalità al progetto.
- Bug Fix: Risoluzione di errori o malfunzionamenti nel codice esistente.
- Architecture: Modifiche alla struttura o all'architettura del sistema per migliorare la progettazione complessiva.
- Deprecation: Indicazioni o rimozione di funzionalità obsolete che non sono più supportate.
- Production: Modifiche destinate alla produzione, che possono riguardare la stabilità, l'affidabilità o la sicurezza del sistema in ambiente live.
- Monitoring: Implementazione o miglioramento delle funzionalità di monitoraggio per raccogliere dati su performance, errori o altri parametri.
- Data Management: Modifiche legate alla gestione dei dati, come la creazione, la lettura, l'aggiornamento e la cancellazione dei dati.
- Git: Operazioni legate al sistema di controllo versione, come modifiche ai branch, configurazione di Git, o operazioni di fusione.
- User Interface: Modifiche all'interfaccia utente, comprese nuove funzionalità o miglioramenti all'esperienza utente.
- Quality Assurance: Attività volte a garantire che il codice rispetti gli standard di qualità, come la revisione del codice o il miglioramento dei processi di testing.
- Machine Learning: Modifiche o miglioramenti al codice che riguarda l'implementazione o il miglioramento di modelli di machine learning.
- Frontend: Modifiche al codice che riguarda la parte visibile dell'applicazione, come il design e l'interazione con l'utente.
- Backend: Modifiche al codice lato server, che riguardano la logica di business, le API o la gestione dei dati.
- Service: Modifiche ai servizi offerti dal sistema, incluse nuove funzionalità, miglioramenti o correzioni a servizi esistenti.
- Other: Categoria non identificata.

L'approccio adottato è il seguente:

1. **Recupero dei Commit:** Per ogni intervallo di commit definito nel file CSV, vengono recuperati i commit dal repository GitHub utilizzando l'API di GitHub. Viene effettuata una ricerca tra la data di inizio e fine trimestre, con gestione della paginazione per i commit numerosi.

2. **Categorizzazione dei Commit:** Ogni messaggio di commit viene analizzato per verificare la presenza di parole chiave specifiche associate a ciascuna categoria. La categorizzazione è effettuata utilizzando una serie di espressioni regolari per identificare parole chiave. Per ogni parola chiave appartenente ad una categoria viene assegnato un punto a quest'ultima. Alla fine del processo di analisi del messaggio, la categoria che totalizza il punteggio più alto viene assegnata a quel commit.
3. **Analisi dei Risultati:** Una volta categorizzati i commit, vengono analizzate le possibili relazioni tra il tipo di commit (es. bug fix, feature addition) e le occorrenze dei *community smells*, confrontando il numero di commit per ciascuna categoria con l'emergere di determinati smell.

Di seguito è riportato un esempio della categorizzazione dei commit per il progetto *AngularJS*

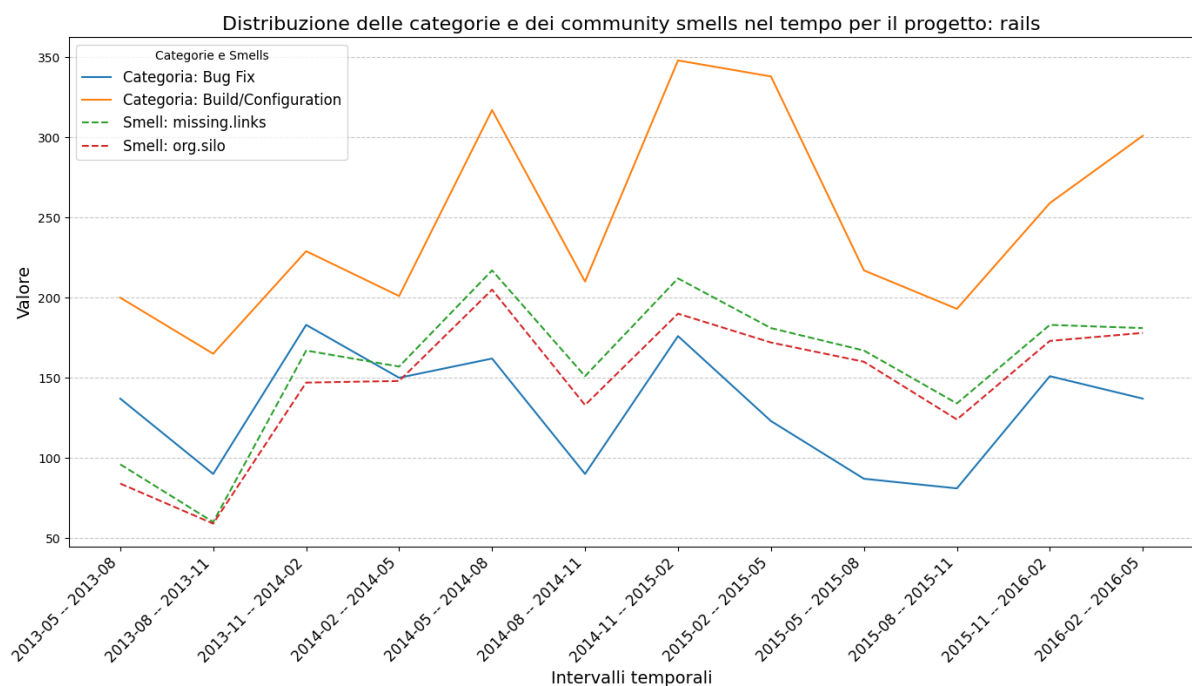


Risultati Attesi RQ3

I risultati di questa analisi permetteranno di comprendere se esiste una relazione significativa tra la tipologia di commit e l'emergere dei *community smells*. In particolare, ci si aspetta di identificare se alcune tipologie di commit, come ad esempio quelli legati alla correzione di bug o al refactoring del codice, abbiano una relazione più forte con l'insorgenza di determinati *community smells*. I risultati potrebbero essere utili per sviluppatori e maintainer di progetti open source, suggerendo pratiche di commit che possano minimizzare l'introduzione di *community smells* e migliorare la qualità complessiva del codice.

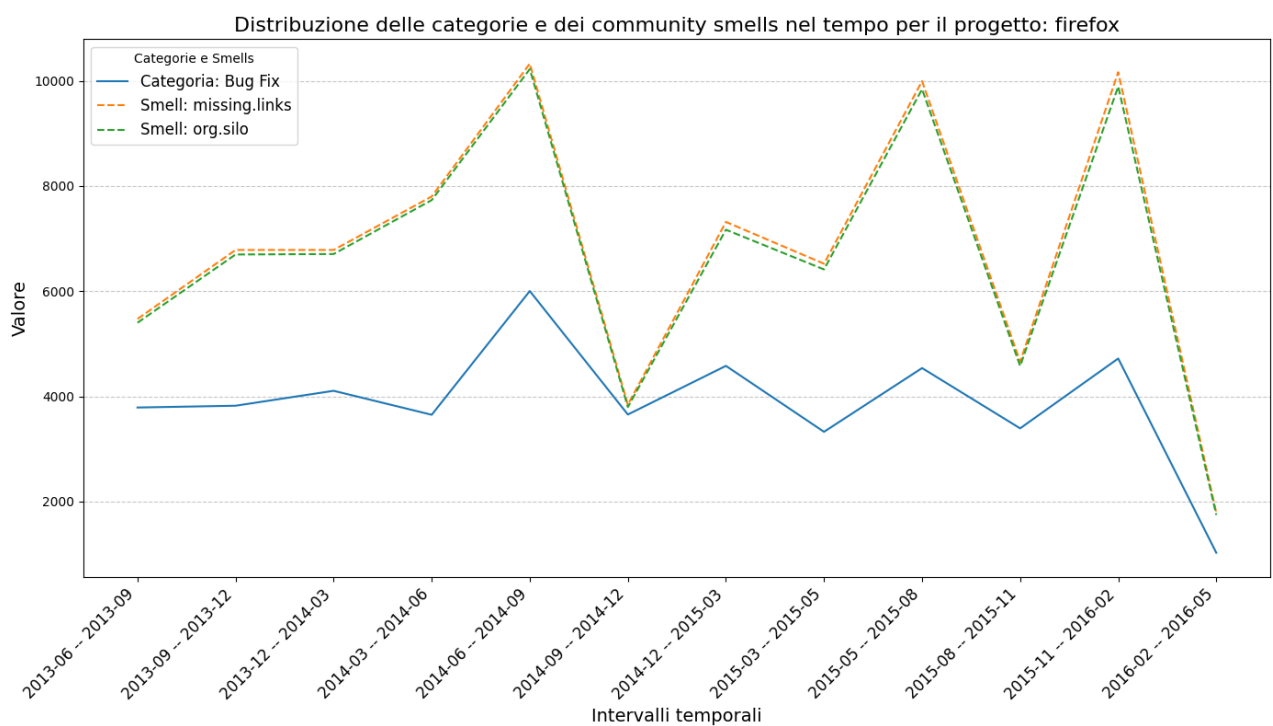
Progetto Rails

Gli smells *Missing Links* e *Organizational Silo* seguono un andamento relativamente simile a quello della categoria *Build/Configuration*. Entrambi presentano un picco tra il 2014 e il 2015, periodo in cui si registra anche un aumento delle modifiche di configurazione e build. Questo potrebbe indicare che le modifiche a livello di configurazione abbiano influito sulla struttura della comunità, portando all'emergere di questi smells. Il calo dei *Bug Fix* verso la fine del periodo sembra coincidere con una stabilizzazione degli smells, suggerendo che una riduzione delle attività di correzione potrebbe aver contribuito a migliorare l'organizzazione del progetto.



Progetto Firefox

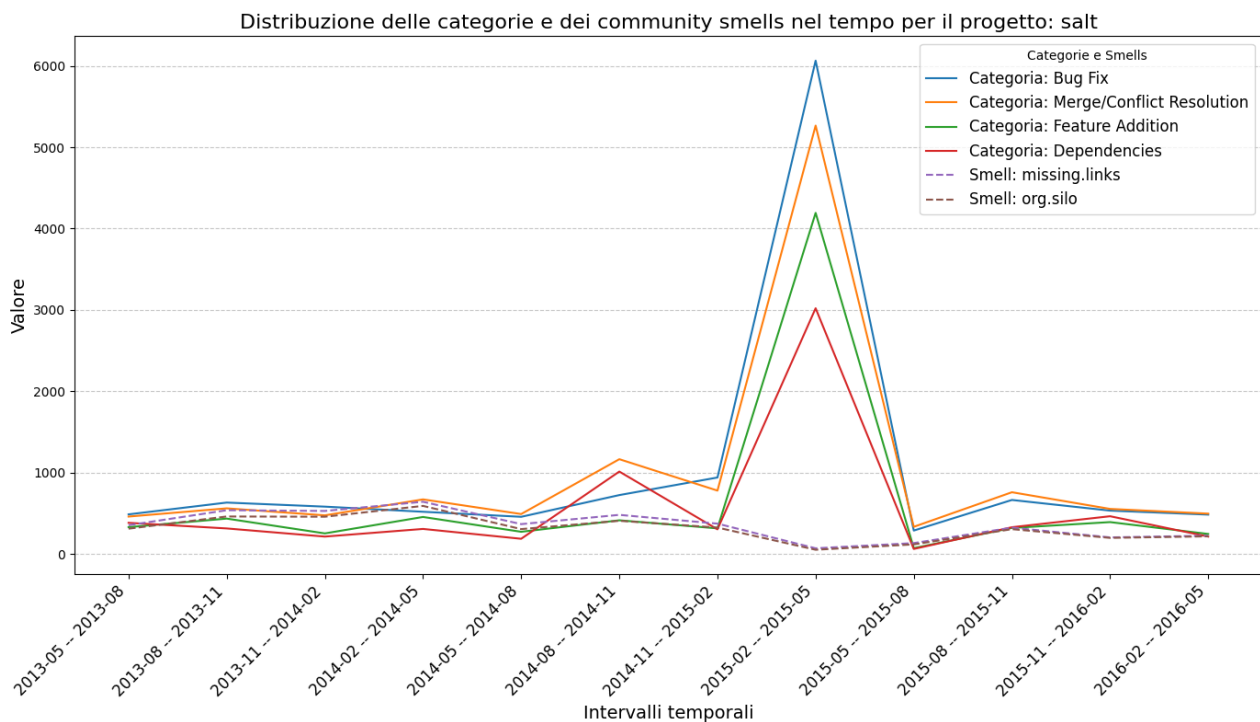
Se si osserva il periodo tra il 2014 e il 2015, c'è una certa coincidenza tra il leggero aumento dei *Bug Fix* e i picchi degli smells (*missing.links* e *org.silo*). In particolare, il calo significativo degli smells verso la fine del 2015 sembra riflettersi in un calo parallelo dei *Bug Fix*. Questo potrebbe indicare che una maggiore attenzione alla correzione dei bug abbia indirettamente ridotto i problemi strutturali nella comunità, oppure che una stabilizzazione della comunità abbia permesso di ridurre la necessità di *Bug Fix*. Sebbene non vi sia un allineamento perfetto tra *Bug Fix* e smells, alcuni picchi e cali coincidono, specialmente nei periodi di maggiore attività (2014-2015). Ciò suggerisce che, almeno in alcuni periodi, l'intensità del lavoro sui bug e i problemi di struttura della comunità potrebbero essere correlati.



Progetto Salt

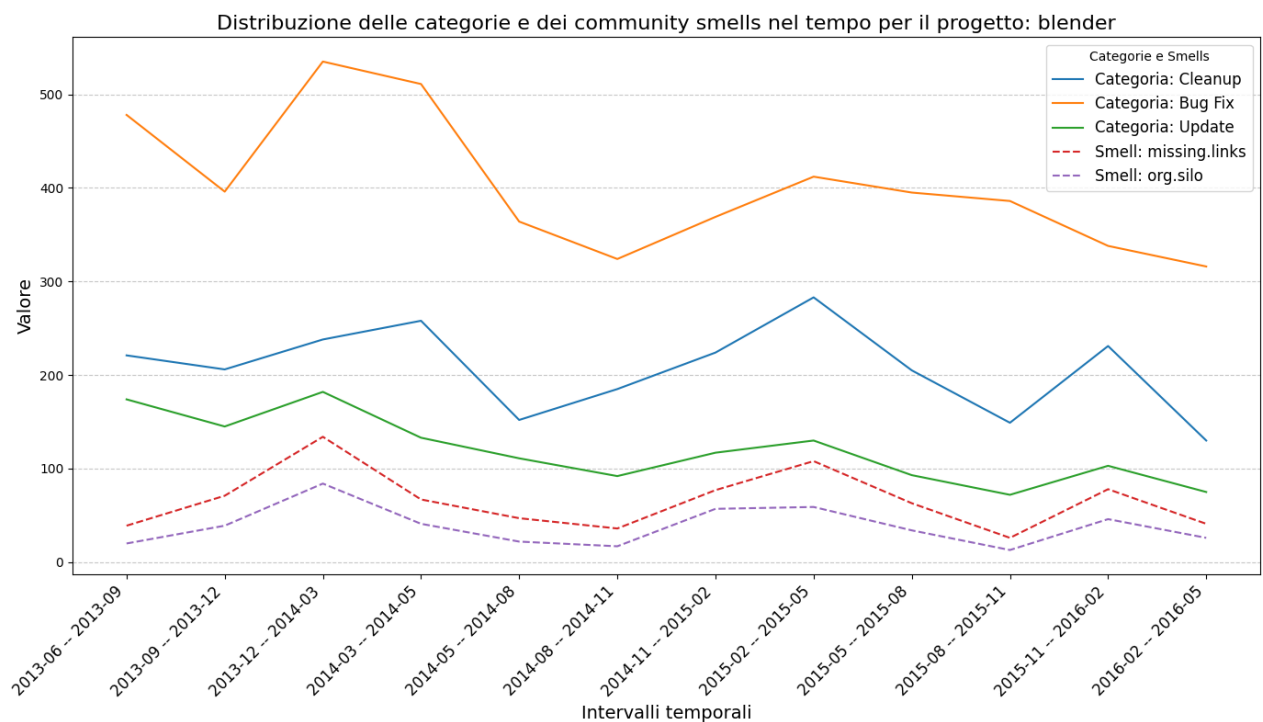
Nel periodo 2015-02/2015-08, c'è un picco massiccio per tutte le categorie di commit: Bug Fix, Merge/Conflict Resolution, Feature Addition e Dependencies. Questo periodo coincide con una riduzione drastica dei community smells (missing links e org. silo). Entrambi gli smell mostrano una netta diminuzione durante il picco di attività, segnalando che l'aumento dei commit potrebbe aver affrontato indirettamente o direttamente i problemi organizzativi o tecnici.

L'alto numero di commit di bug fix e merge resolution suggerisce una fase di stabilizzazione o consolidamento del progetto. Durante questa fase, potrebbero essere stati risolti problemi che contribuivano ai community smells, come dipendenze non gestite o team disallineati. L'aggiunta di feature e l'aggiornamento delle dipendenze possono aver migliorato la struttura del codice e ridotto il debito tecnico. Questo potrebbe aver contribuito a eliminare gli smell organizzativi. Un picco di merge conflict resolution indica una maggiore interazione tra team o membri del team, riducendo missing links e org. silo.



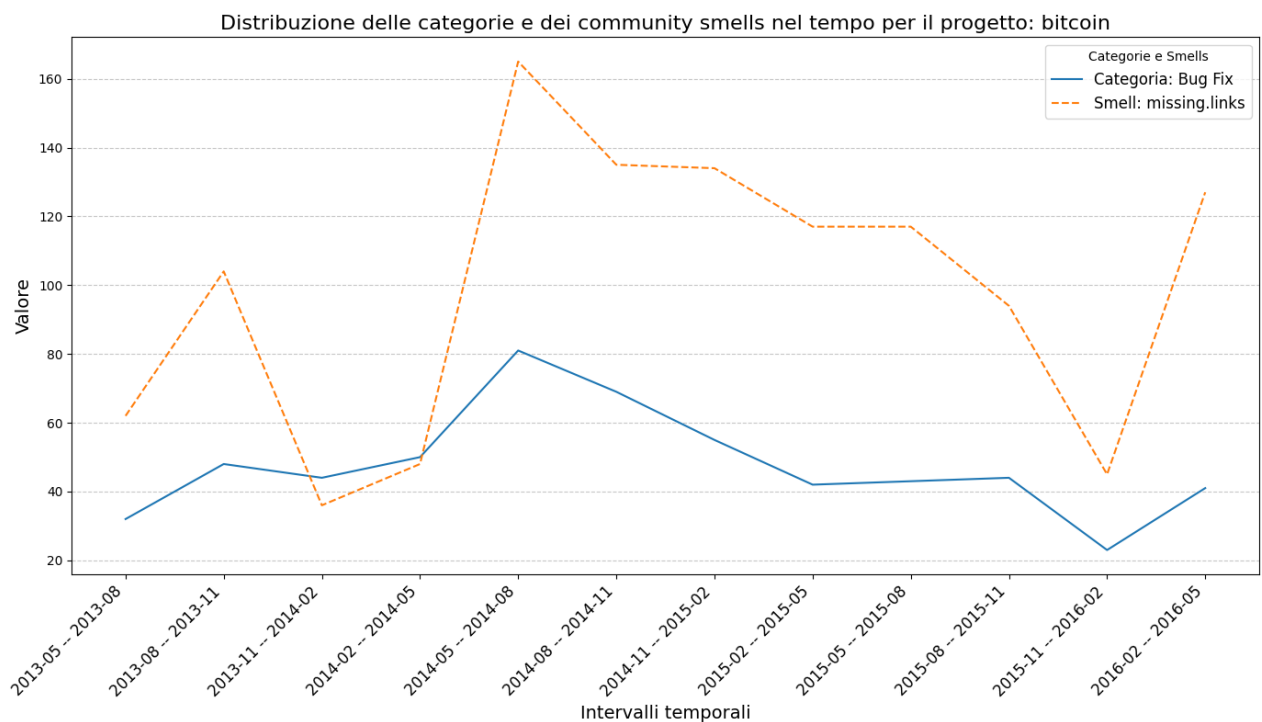
Blender

Il grafico evidenzia una leggera somiglianza nell'andamento tra "Bug Fix" e i community smells (specialmente "missing.links"). Entrambi mostrano una tendenza decrescente negli intervalli temporali finali. Questa relazione potrebbe suggerire che la riduzione dei commit dedicati ai bug abbia portato a una diminuzione dei problemi strutturali nella comunità, riducendo così l'insorgenza di determinati community smells. "Cleanup" mostra fluttuazioni che si allineano in parte a "org.silo". Questo può indicare che attività di pulizia del codice e miglioramento organizzativo siano legate alla gestione di questo smell, con una possibile causa-effetto indiretta: i team potrebbero concentrarsi sul miglioramento del codice per affrontare problemi organizzativi. "Update" presenta un andamento più stabile, ma le sue fluttuazioni sembrano talvolta allinearsi con quelle dei community smells. Ad esempio, picchi più alti nei commit di aggiornamento (intorno al 2014) coincidono con aumenti di alcuni smells, suggerendo che cambiamenti significativi nel progetto possano portare a disfunzioni nella comunicazione o nell'organizzazione.



Progetto Bitcoin

L'andamento di "Bug Fix" segue un pattern simile al community smell "missing.links", soprattutto nella fase centrale del grafico (2014-05). Questo potrebbe indicare che problemi tecnici rilevati nei bug abbiano avuto un impatto sulla struttura comunicativa del progetto, o viceversa. La riduzione congiunta di "Bug Fix" e "missing.links" verso la parte finale del periodo potrebbe riflettere un miglioramento sia sul piano tecnico sia su quello organizzativo, con minore necessità di interventi. Il picco di "missing.links" (2014-05) coincide con un aumento dei commit di "Bug Fix", suggerendo che un periodo di alta attività tecnica potrebbe essere stato accompagnato da problemi di coordinamento o disconnessioni tra i membri della comunità.



Considerazioni generali RQ3

L'analisi dei progetti rivela tendenze comuni legate all'andamento dei Bug Fix e alla presenza di community smells. Nei progetti Firefox, Blender e Bitcoin, ad esempio, si osservano picchi e cali paralleli tra l'attività di correzione di bug e il tipo di smells come *missing.links*, suggerendo una forte interazione tra la correzione di codice e la struttura del progetto. In particolare, nel caso dei progetti Rails e Bitcoin, la riduzione dei Bug Fix verso la fine di alcuni periodi coincide con un miglioramento degli smells, il che sembra indicare un processo di stabilizzazione a livello sia tecnico che organizzativo. Un altro punto interessante riguarda l'impatto delle *build/configuration* e delle attività di *cleanup* del codice nei progetti Rails e Blender, che seguono l'andamento degli smells, come *org.silo*. In questo contesto, il progetto Salt si distingue per un periodo di alta attività, con un elevato volume di commit legati a categorie diverse, come Bug Fix, Merge Resolution e Feature Addition, che porta a una significativa riduzione degli smells. Questo fenomeno suggerisce un miglioramento complessivo dei processi tecnici e organizzativi.

Limiti dell'approccio e sviluppi futuri RQ3

Lo studio effettuato presenta dei limiti relativi alla parte di categorizzazione dei commit in quanto utilizzare un approccio orientato all'utilizzo di parole chiave per definire le categorie aumenta la probabilità di un'assegnazione scorretta. Tutto questo avviene perché le singole keyword non presentano contezza del contesto relativo alla frase. Un approccio più efficace potrebbe essere quello basato sull'utilizzo di Large Language Model (LLM) che permettono una categorizzazione contestualizzata e quindi più precisa, riducendo significativamente l'errore.

(Nel contesto di questo progetto è stata intrapresa anche questa strategia che purtroppo è stata abbandonata per limitazioni dovute al numero di richieste associate all'account free)

Conclusioni e sviluppi futuri

In conclusione, il nostro progetto ha evidenziato come i community smells rappresentino un'importante sfida nei progetti open source, influenzando negativamente le dinamiche organizzative e la qualità del software. Le analisi hanno permesso di identificare pattern ricorrenti e relazioni causali tra diversi tipi di smells, nonché il loro legame con le metriche di turnover e le attività di commit.

Inoltre, l'analisi dei commit ha mostrato che attività come il bug fixing, il cleanup e la gestione delle configurazioni possono influire sull'andamento degli smells, suggerendo possibili relazioni tra l'intensità delle attività tecniche e la riduzione delle problematiche socio-tecniche. Per quanto riguarda gli sviluppi futuri, proponiamo di adottare Large Language Models (LLM) per migliorare la categorizzazione dei commit, superando le limitazioni legate all'approccio basato su parole chiave, che non coglie il contesto semantico. Inoltre, un'estensione dell'analisi temporale per periodi più lunghi e una maggiore integrazione di metriche sociali e tecniche potrebbero offrire una comprensione più completa delle dinamiche di evoluzione dei smells. Infine, estendere l'analisi a progetti di medie e piccole dimensioni per individuare possibili cambiamenti nei risultati.

Impatto su evoluzione e qualità Software

Il progetto apporta un contributo all'evoluzione e alla qualità del software grazie ai risultati dell'analisi condotte. Per la RQ1, che esplora le relazioni causali tra i community smells, l'approccio qualitativo ha permesso di identificare pattern ricorrenti, come la relazione causale tra Radio Silence e altri due smell (Organizational Silo e Missing Links), evidenziando come la mancanza di comunicazione formale in una sottocomunità possa isolare i team e creare divisioni organizzative. Questi risultati consentono di sviluppare interventi mirati al miglioramento della comunicazione (Riduzione Radio Silence) che permette di conseguenza di prevenire indirettamente l'insorgenza di altri due smell, Organizational Silo e Missing Links. Il turnover del personale, sia generale che specifico al codice, ha un impatto significativo sulla qualità e sull'evoluzione del software, favorendo *community smells* come *radio.silence* e *org.silo*, che frammentano la comunicazione e creano silos organizzativi. L'abbandono di contributori problematici (*smelly quitters*) amplifica ulteriormente l'isolamento e riduce le connessioni tra i gruppi, aumentando il rischio di debito tecnico e compromettendo la sostenibilità del progetto. Per affrontare queste problematiche, è fondamentale migliorare la comunicazione attraverso strumenti collaborativi, garantire una solida documentazione per preservare le conoscenze, monitorare e ridurre i *community smells*, gestire proattivamente il turnover con strategie di onboarding e offboarding, e rafforzare l'integrazione tra i team per evitare divisioni e promuovere una maggiore resilienza organizzativa. Per la RQ2, che analizza l'andamento temporale dei community smells, l'approccio qualitativo ha messo in luce pattern evolutivi distinti, come la sovrapposizione temporale tra Organizational Silo e Missing Links nei progetti di sviluppo web. Quindi quando si ha a che fare con questo tipo di progetti si dovrebbe prevenire la comparsa di questi due smell che sembrano andare di pari passo. Questo risultato evidenzia dinamiche organizzative specifiche, fornendo una base per pianificare interventi mirati durante le fasi critiche del ciclo di vita del progetto, come momenti di rilascio o ristrutturazioni del team. Questi risultati qualitativi arricchiscono la comprensione delle cause e degli effetti dei community smells, supportando il miglioramento delle pratiche collaborative e la stabilità dei progetti, con un impatto positivo sia sulla qualità del codice che sulle dinamiche organizzative nei contesti open source. Particolare attenzione è stata data alla RQ3, che esplora il legame tra tipologie di commit e community smells. Tuttavia, i risultati di questa domanda, essendo parte di un Proof of Concept (PoC), devono essere interpretati con cautela, in quanto non completamente generalizzabili. Nonostante ciò, alcune evidenze portano a porre l'attenzione a categorie di commit come Bug fix che si è visto essere una potenziale causa di mancanza di comunicazione e formazione di silos organizzativi. Queste evidenze consentono di sviluppare strategie mirate per migliorare la comunicazione, ridurre l'impatto del turnover e

promuovere pratiche che minimizzano l'insorgenza di smells, contribuendo alla creazione di software di qualità più elevata nei progetti open source.

Appendice 1: Risultati Test di Granger RQ1

=== Risultati per il progetto: U-boot_report ===

prima.donnas: Serie stazionaria (p-value=0.009).

black.cloud: Serie non stazionaria, differenziata.

radio.silence: Serie stazionaria (p-value=0.001).

missing.links: Serie stazionaria (p-value=0.031).

org.silo: Serie non stazionaria, differenziata.

Test di Granger: black.cloud causa radio.silence

Significativo: black.cloud causa radio.silence (p-value=0.040)

Test di Granger: black.cloud causa missing.links

Significativo: black.cloud causa missing.links (p-value=0.013)

Test di Granger: radio.silence causa org.silo

Significativo: radio.silence causa org.silo (p-value=0.026)

Test di Granger: org.silo causa prima.donnas

Significativo: org.silo causa prima.donnas (p-value=0.002)

=== Risultati per il progetto: Mesa_report ===

prima.donnas: Serie non stazionaria, differenziata.

black.cloud: Serie stazionaria (p-value=0.014).

radio.silence: Serie non stazionaria, differenziata.

missing.links: Serie stazionaria (p-value=0.000).

org.silo: Serie stazionaria (p-value=0.000).

Test di Granger: prima.donnas causa radio.silence

Significativo: prima.donnas causa radio.silence (p-value=0.048)

Test di Granger: prima.donnas causa missing.links

Significativo: prima.donnas causa missing.links (p-value=0.036)

Test di Granger: black.cloud causa missing.links

Significativo: black.cloud causa missing.links (p-value=0.009)

Test di Granger: black.cloud causa org.silo

Significativo: black.cloud causa org.silo (p-value=0.004)

Test di Granger: missing.links causa prima.donnas

Significativo: missing.links causa prima.donnas (p-value=0.038)

=== Risultati per il progetto: Django_report ===

Serie costante per prima.donnas. Test ADF non eseguito.

black.cloud: Serie stazionaria (p-value=0.014).

radio.silence: Serie non stazionaria, differenziata.

missing.links: Serie non stazionaria, differenziata.

org.silo: Serie non stazionaria, differenziata.

Test di Granger: missing.links causa black.cloud
Significativo: missing.links causa black.cloud (p-value=0.006)

Test di Granger: org.silo causa black.cloud
Significativo: org.silo causa black.cloud (p-value=0.007)

=== Risultati per il progetto: Emacs_report ===
prima.donnas: Serie stazionaria (p-value=0.006).
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie stazionaria (p-value=0.000).
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa radio.silence
Significativo: prima.donnas causa radio.silence (p-value=0.029)

Test di Granger: black.cloud causa missing.links
Significativo: black.cloud causa missing.links (p-value=0.000)

Test di Granger: black.cloud causa org.silo
Significativo: black.cloud causa org.silo (p-value=0.000)

Test di Granger: radio.silence causa black.cloud
Significativo: radio.silence causa black.cloud (p-value=0.037)

=== Risultati per il progetto: FFmpeg_report ===
prima.donnas: Serie non stazionaria, differenziata.
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa radio.silence
Significativo: prima.donnas causa radio.silence (p-value=0.045)

Test di Granger: missing.links causa black.cloud
Significativo: missing.links causa black.cloud (p-value=0.010)

Test di Granger: org.silo causa black.cloud
Significativo: org.silo causa black.cloud (p-value=0.012)

=== Risultati per il progetto: Nodejs_report ===
Serie costante per prima.donnas. Test ADF non eseguito.
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie stazionaria (p-value=0.000).
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: radio.silence causa missing.links
Significativo: radio.silence causa missing.links (p-value=0.029)

Test di Granger: radio.silence causa org.silo
Significativo: radio.silence causa org.silo (p-value=0.038)

=== Risultati per il progetto: Rails_report ===

prima.donnas: Serie stazionaria (p-value=0.014).
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa org.silo
Significativo: prima.donnas causa org.silo (p-value=0.035)

Test di Granger: radio.silence causa missing.links
Significativo: radio.silence causa missing.links (p-value=0.013)

Test di Granger: radio.silence causa org.silo
Significativo: radio.silence causa org.silo (p-value=0.014)

=== Risultati per il progetto: Git_report ===

prima.donnas: Serie non stazionaria, differenziata.
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie stazionaria (p-value=0.033).
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie stazionaria (p-value=0.009).

Test di Granger: black.cloud causa org.silo
Significativo: black.cloud causa org.silo (p-value=0.000)

Test di Granger: radio.silence causa black.cloud
Significativo: radio.silence causa black.cloud (p-value=0.008)

Test di Granger: radio.silence causa missing.links
Significativo: radio.silence causa missing.links (p-value=0.020)

Test di Granger: missing.links causa black.cloud
Significativo: missing.links causa black.cloud (p-value=0.025)

=== Risultati per il progetto: iPython_report ===

Serie costante per prima.donnas. Test ADF non eseguito.
black.cloud: Serie stazionaria (p-value=0.000).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: black.cloud causa missing.links
Significativo: black.cloud causa missing.links (p-value=0.047)

Test di Granger: black.cloud causa org.silo
Significativo: black.cloud causa org.silo (p-value=0.041)

Test di Granger: org.silo causa missing.links
Significativo: org.silo causa missing.links (p-value=0.046)

=== Risultati per il progetto: LLVM_report ===

prima.donnas: Serie stazionaria (p-value=0.003).
black.cloud: Serie stazionaria (p-value=0.000).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa missing.links
Significativo: prima.donnas causa missing.links (p-value=0.050)

Test di Granger: missing.links causa radio.silence
Significativo: missing.links causa radio.silence (p-value=0.029)

=== Risultati per il progetto: Gstreamer_report ===
prima.donnas: Serie stazionaria (p-value=0.014).
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.026).
org.silo: Serie stazionaria (p-value=0.035).

Test di Granger: black.cloud causa radio.silence
Significativo: black.cloud causa radio.silence (p-value=0.002)

Test di Granger: radio.silence causa prima.donnas
Significativo: radio.silence causa prima.donnas (p-value=0.041)

Test di Granger: radio.silence causa black.cloud
Significativo: radio.silence causa black.cloud (p-value=0.001)

=== Risultati per il progetto: Python_report ===
prima.donnas: Serie stazionaria (p-value=nan).
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie non stazionaria, differenziata.

Test di Granger: org.silo causa missing.links
Significativo: org.silo causa missing.links (p-value=0.000)

=== Risultati per il progetto: Blender_report ===
Serie costante per prima.donnas. Test ADF non eseguito.
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: radio.silence causa missing.links
Significativo: radio.silence causa missing.links (p-value=0.000)

Test di Granger: radio.silence causa org.silo
Significativo: radio.silence causa org.silo (p-value=0.000)

=== Risultati per il progetto: Firefox_report ===
prima.donnas: Serie non stazionaria, differenziata.
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie stazionaria (p-value=0.001).

Test di Granger: radio.silence causa prima.donnas
Significativo: radio.silence causa prima.donnas (p-value=0.032)

=== Risultati per il progetto: QEMU_report ===
prima.donnas: Serie stazionaria (p-value=0.003).
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie stazionaria (p-value=0.042).

missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa org.silo
Significativo: prima.donnas causa org.silo (p-value=0.003)

Test di Granger: black.cloud causa org.silo
Significativo: black.cloud causa org.silo (p-value=0.009)

Test di Granger: missing.links causa prima.donnas
Significativo: missing.links causa prima.donnas (p-value=0.027)

Test di Granger: missing.links causa radio.silence
Significativo: missing.links causa radio.silence (p-value=0.000)

Test di Granger: org.silo causa black.cloud
Significativo: org.silo causa black.cloud (p-value=0.005)

=== Risultati per il progetto: Salt_report ===
prima.donnas: Serie stazionaria (p-value=0.014).
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.

Test di Granger: prima.donnas causa radio.silence
Significativo: prima.donnas causa radio.silence (p-value=0.014)

Test di Granger: radio.silence causa prima.donnas
Significativo: radio.silence causa prima.donnas (p-value=0.000)

Appendice 2: Risultati Test di Granger RQ2

=== Risultati Test di Granger ===

=== Risultati per il progetto: GitLab_report ===

Serie costante per prima.donnas. Test ADF non eseguito.

Serie costante per black.cloud. Test ADF non eseguito.

radio.silence: Serie stazionaria (p-value=0.005).

missing.links: Serie non stazionaria, differenziata.

org.silo: Serie non stazionaria, differenziata.

global.turnover: Serie non stazionaria, differenziata.

code.turnover: Serie non stazionaria, differenziata.

core.global.turnover: Serie stazionaria (p-value=0.000).

core.mail.turnover: Serie stazionaria (p-value=0.000).

core.code.turnover: Serie non stazionaria, differenziata.

ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: global.turnover causa missing.links

Significativo: global.turnover causa missing.links (p-value=0.045)

Test di Granger: code.turnover causa missing.links

Significativo: code.turnover causa missing.links (p-value=0.021)

Test di Granger: global.turnover causa org.silo

Significativo: global.turnover causa org.silo (p-value=0.036)

Test di Granger: code.turnover causa org.silo

Significativo: code.turnover causa org.silo (p-value=0.015)

=== Risultati per il progetto: U-boot_report ===

prima.donnas: Serie stazionaria (p-value=0.009).

black.cloud: Serie non stazionaria, differenziata.

radio.silence: Serie stazionaria (p-value=0.001).

missing.links: Serie stazionaria (p-value=0.031).

org.silo: Serie non stazionaria, differenziata.

global.turnover: Serie stazionaria (p-value=0.000).

code.turnover: Serie stazionaria (p-value=0.000).

core.global.turnover: Serie non stazionaria, differenziata.

core.mail.turnover: Serie non stazionaria, differenziata.

core.code.turnover: Serie stazionaria (p-value=0.000).

ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: global.turnover causa black.cloud

Significativo: global.turnover causa black.cloud (p-value=0.006)

Test di Granger: code.turnover causa black.cloud

Significativo: code.turnover causa black.cloud (p-value=0.002)

Test di Granger: core.code.turnover causa black.cloud

Significativo: core.code.turnover causa black.cloud (p-value=0.004)

Test di Granger: core.global.turnover causa radio.silence

Significativo: core.global.turnover causa radio.silence (p-value=0.014)

Test di Granger: core.mail.turnover causa radio.silence

Significativo: core.mail.turnover causa radio.silence (p-value=0.003)

Test di Granger: ratio.smelly.quitters causa org.silo

Significativo: ratio.smelly.quitters causa org.silo (p-value=0.001)

=== Risultati per il progetto: Mesa_report ===

prima.donnas: Serie non stazionaria, differenziata.
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie stazionaria (p-value=0.000).
global.turnover: Serie stazionaria (p-value=0.043).
code.turnover: Serie stazionaria (p-value=0.011).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie stazionaria (p-value=0.007).

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.005)

Test di Granger: core.global.turnover causa radio.silence
Significativo: core.global.turnover causa radio.silence (p-value=0.030)

Test di Granger: core.mail.turnover causa radio.silence
Significativo: core.mail.turnover causa radio.silence (p-value=0.018)

Test di Granger: ratio.smelly.quitters causa radio.silence
Significativo: ratio.smelly.quitters causa radio.silence (p-value=0.033)

=== Risultati per il progetto: Django_report ===

Serie costante per prima.donnas. Test ADF non eseguito.
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: ratio.smelly.quitters causa black.cloud
Significativo: ratio.smelly.quitters causa black.cloud (p-value=0.037)

Test di Granger: core.global.turnover causa radio.silence
Significativo: core.global.turnover causa radio.silence (p-value=0.000)

Test di Granger: core.code.turnover causa radio.silence
Significativo: core.code.turnover causa radio.silence (p-value=0.014)

Test di Granger: ratio.smelly.quitters causa radio.silence
Significativo: ratio.smelly.quitters causa radio.silence (p-value=0.000)

Test di Granger: ratio.smelly.quitters causa missing.links
Significativo: ratio.smelly.quitters causa missing.links (p-value=0.005)

Test di Granger: ratio.smelly.quitters causa org.silo
Significativo: ratio.smelly.quitters causa org.silo (p-value=0.046)

=== Risultati per il progetto: Emacs_report ===

prima.donnas: Serie stazionaria (p-value=0.006).

black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie stazionaria (p-value=0.000).
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie stazionaria (p-value=0.000).
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: code.turnover causa prima.donnas
Significativo: code.turnover causa prima.donnas (p-value=0.037)

Test di Granger: ratio.smelly.quitters causa black.cloud
Significativo: ratio.smelly.quitters causa black.cloud (p-value=0.019)

Test di Granger: ratio.smelly.quitters causa missing.links
Significativo: ratio.smelly.quitters causa missing.links (p-value=0.040)

=== Risultati per il progetto: FFmpeg_report ===

prima.donnas: Serie non stazionaria, differenziata.
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie non stazionaria, differenziata.
code.turnover: Serie stazionaria (p-value=0.034).
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie stazionaria (p-value=0.001).

Test di Granger: ratio.smelly.quitters causa prima.donnas
Significativo: ratio.smelly.quitters causa prima.donnas (p-value=0.036)

Test di Granger: core.code.turnover causa black.cloud
Significativo: core.code.turnover causa black.cloud (p-value=0.029)

Test di Granger: code.turnover causa radio.silence
Significativo: code.turnover causa radio.silence (p-value=0.011)

Test di Granger: code.turnover causa missing.links
Significativo: code.turnover causa missing.links (p-value=0.004)

Test di Granger: global.turnover causa org.silo
Significativo: global.turnover causa org.silo (p-value=0.045)

Test di Granger: code.turnover causa org.silo
Significativo: code.turnover causa org.silo (p-value=0.001)

=== Risultati per il progetto: Nodejs_report ===

Serie costante per prima.donnas. Test ADF non eseguito.
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie stazionaria (p-value=0.000).
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie non stazionaria, differenziata.
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie stazionaria (p-value=0.000).

core.mail.turnover: Serie stazionaria (p-value=0.000).
core.code.turnover: Serie stazionaria (p-value=0.000).
ratio.smelly.quitters: Serie stazionaria (p-value=0.012).

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.000)

Test di Granger: code.turnover causa radio.silence
Significativo: code.turnover causa radio.silence (p-value=0.000)

Test di Granger: core.code.turnover causa radio.silence
Significativo: core.code.turnover causa radio.silence (p-value=0.001)

Test di Granger: global.turnover causa missing.links
Significativo: global.turnover causa missing.links (p-value=0.011)

Test di Granger: ratio.smelly.quitters causa missing.links
Significativo: ratio.smelly.quitters causa missing.links (p-value=0.005)

Test di Granger: global.turnover causa org.silo
Significativo: global.turnover causa org.silo (p-value=0.014)

Test di Granger: ratio.smelly.quitters causa org.silo
Significativo: ratio.smelly.quitters causa org.silo (p-value=0.005)

=== Risultati per il progetto: Rails_report ===
prima.donnas: Serie stazionaria (p-value=0.014).
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie stazionaria (p-value=0.005).
code.turnover: Serie non stazionaria, differenziata.
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie stazionaria (p-value=0.003).
ratio.smelly.quitters: Serie stazionaria (p-value=0.014).

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.006)

Test di Granger: code.turnover causa radio.silence
Significativo: code.turnover causa radio.silence (p-value=0.010)

Test di Granger: core.code.turnover causa radio.silence
Significativo: core.code.turnover causa radio.silence (p-value=0.000)

Test di Granger: ratio.smelly.quitters causa missing.links
Significativo: ratio.smelly.quitters causa missing.links (p-value=0.000)

Test di Granger: global.turnover causa org.silo
Significativo: global.turnover causa org.silo (p-value=0.048)

Test di Granger: ratio.smelly.quitters causa org.silo
Significativo: ratio.smelly.quitters causa org.silo (p-value=0.000)

=== Risultati per il progetto: LibreOffice_report ===
prima.donnas: Serie stazionaria (p-value=0.005).
black.cloud: Serie non stazionaria, differenziata.
radio.silence: Serie non stazionaria, differenziata.

missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie stazionaria (p-value=0.018).
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie stazionaria (p-value=0.000).
ratio.smelly.quitters: Serie stazionaria (p-value=0.003).

Test di Granger: code.turnover causa prima.donnas
Significativo: code.turnover causa prima.donnas (p-value=0.045)

Test di Granger: core.code.turnover causa prima.donnas
Significativo: core.code.turnover causa prima.donnas (p-value=0.001)

Test di Granger: global.turnover causa black.cloud
Significativo: global.turnover causa black.cloud (p-value=0.025)

Test di Granger: code.turnover causa black.cloud
Significativo: code.turnover causa black.cloud (p-value=0.022)

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.000)

Test di Granger: code.turnover causa radio.silence
Significativo: code.turnover causa radio.silence (p-value=0.001)

Test di Granger: core.mail.turnover causa missing.links
Significativo: core.mail.turnover causa missing.links (p-value=0.003)

Test di Granger: core.global.turnover causa org.silo
Significativo: core.global.turnover causa org.silo (p-value=0.039)

Test di Granger: core.mail.turnover causa org.silo
Significativo: core.mail.turnover causa org.silo (p-value=0.010)

=== Risultati per il progetto: Git_report ===

prima.donnas: Serie non stazionaria, differenziata.
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie stazionaria (p-value=0.033).
missing.links: Serie stazionaria (p-value=0.000).
org.silo: Serie stazionaria (p-value=0.009).
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie stazionaria (p-value=0.000).
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie stazionaria (p-value=0.000).

Test di Granger: core.global.turnover causa prima.donnas
Significativo: core.global.turnover causa prima.donnas (p-value=0.008)

Test di Granger: core.mail.turnover causa prima.donnas
Significativo: core.mail.turnover causa prima.donnas (p-value=0.005)

=== Risultati per il progetto: iPython_report ===

Serie costante per prima.donnas. Test ADF non eseguito.
black.cloud: Serie stazionaria (p-value=0.000).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.

org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie non stazionaria, differenziata.
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.018)

Test di Granger: code.turnover causa radio.silence
Significativo: code.turnover causa radio.silence (p-value=0.009)

Test di Granger: core.global.turnover causa radio.silence
Significativo: core.global.turnover causa radio.silence (p-value=0.004)

Test di Granger: core.mail.turnover causa radio.silence
Significativo: core.mail.turnover causa radio.silence (p-value=0.007)

=== Risultati per il progetto: LLVM_report ===

prima.donnas: Serie stazionaria (p-value=0.003).
black.cloud: Serie stazionaria (p-value=0.000).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie non stazionaria, differenziata.
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie stazionaria (p-value=0.000).
core.code.turnover: Serie stazionaria (p-value=0.021).
ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: code.turnover causa prima.donnas
Significativo: code.turnover causa prima.donnas (p-value=0.014)

Test di Granger: core.code.turnover causa prima.donnas
Significativo: core.code.turnover causa prima.donnas (p-value=0.005)

Test di Granger: global.turnover causa radio.silence
Significativo: global.turnover causa radio.silence (p-value=0.005)

Test di Granger: core.global.turnover causa radio.silence
Significativo: core.global.turnover causa radio.silence (p-value=0.000)

Test di Granger: ratio.smelly.quitters causa radio.silence
Significativo: ratio.smelly.quitters causa radio.silence (p-value=0.001)

Test di Granger: code.turnover causa missing.links
Significativo: code.turnover causa missing.links (p-value=0.043)

Test di Granger: code.turnover causa org.silo
Significativo: code.turnover causa org.silo (p-value=0.005)

Test di Granger: core.mail.turnover causa org.silo
Significativo: core.mail.turnover causa org.silo (p-value=0.008)

=== Risultati per il progetto: Gstreamer_report ===

prima.donnas: Serie stazionaria (p-value=0.014).
black.cloud: Serie non stazionaria, differenziata.

radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie stazionaria (p-value=0.026).
org.silo: Serie stazionaria (p-value=0.035).
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.004).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie stazionaria (p-value=0.007).
core.code.turnover: Serie stazionaria (p-value=0.046).
ratio.smelly.quitters: Serie stazionaria (p-value=0.025).

Test di Granger: global.turnover causa prima.donnas
Significativo: global.turnover causa prima.donnas (p-value=0.000)

Test di Granger: code.turnover causa prima.donnas
Significativo: code.turnover causa prima.donnas (p-value=0.000)

Test di Granger: core.mail.turnover causa prima.donnas
Significativo: core.mail.turnover causa prima.donnas (p-value=0.000)

Test di Granger: core.code.turnover causa prima.donnas
Significativo: core.code.turnover causa prima.donnas (p-value=0.004)

Test di Granger: ratio.smelly.quitters causa black.cloud
Significativo: ratio.smelly.quitters causa black.cloud (p-value=0.003)

Test di Granger: ratio.smelly.quitters causa radio.silence
Significativo: ratio.smelly.quitters causa radio.silence (p-value=0.000)

Test di Granger: global.turnover causa missing.links
Significativo: global.turnover causa missing.links (p-value=0.026)

Test di Granger: global.turnover causa org.silo
Significativo: global.turnover causa org.silo (p-value=0.011)

Test di Granger: code.turnover causa org.silo
Significativo: code.turnover causa org.silo (p-value=0.014)

Test di Granger: core.mail.turnover causa org.silo
Significativo: core.mail.turnover causa org.silo (p-value=0.039)

=== Risultati per il progetto: Blender_report ===

Serie costante per prima.donnas. Test ADF non eseguito.

black.cloud: Serie stazionaria (p-value=0.014).

radio.silence: Serie non stazionaria, differenziata.

missing.links: Serie non stazionaria, differenziata.

org.silo: Serie non stazionaria, differenziata.

global.turnover: Serie non stazionaria, differenziata.

code.turnover: Serie non stazionaria, differenziata.

core.global.turnover: Serie non stazionaria, differenziata.

core.mail.turnover: Serie stazionaria (p-value=0.000).

core.code.turnover: Serie stazionaria (p-value=0.000).

ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: core.code.turnover causa radio.silence
Significativo: core.code.turnover causa radio.silence (p-value=0.013)

=== Risultati per il progetto: Firefox_report ===

prima.donnas: Serie non stazionaria, differenziata.

Serie costante per black.cloud. Test ADF non eseguito.

radio.silence: Serie non stazionaria, differenziata.

missing.links: Serie stazionaria (p-value=0.000).

org.silo: Serie stazionaria (p-value=0.001).
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie non stazionaria, differenziata.
core.mail.turnover: Serie stazionaria (p-value=0.000).
core.code.turnover: Serie stazionaria (p-value=0.000).
ratio.smelly.quitters: Serie stazionaria (p-value=0.002).

Test di Granger: core.mail.turnover causa radio.silence
Significativo: core.mail.turnover causa radio.silence (p-value=0.035)

=== Risultati per il progetto: Salt_report ===

prima.donnas: Serie stazionaria (p-value=0.014).
black.cloud: Serie stazionaria (p-value=0.014).
radio.silence: Serie non stazionaria, differenziata.
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie non stazionaria, differenziata.
code.turnover: Serie non stazionaria, differenziata.
core.global.turnover: Serie stazionaria (p-value=0.000).
core.mail.turnover: Serie stazionaria (p-value=0.018).
core.code.turnover: Serie non stazionaria, differenziata.
ratio.smelly.quitters: Serie non stazionaria, differenziata.

Test di Granger: global.turnover causa prima.donnas
Significativo: global.turnover causa prima.donnas (p-value=0.000)

Test di Granger: code.turnover causa prima.donnas
Significativo: code.turnover causa prima.donnas (p-value=0.008)

Test di Granger: core.code.turnover causa prima.donnas
Significativo: core.code.turnover causa prima.donnas (p-value=0.002)

Test di Granger: ratio.smelly.quitters causa prima.donnas
Significativo: ratio.smelly.quitters causa prima.donnas (p-value=0.000)

Test di Granger: core.global.turnover causa black.cloud
Significativo: core.global.turnover causa black.cloud (p-value=0.000)

Test di Granger: core.mail.turnover causa black.cloud
Significativo: core.mail.turnover causa black.cloud (p-value=0.000)

Test di Granger: core.global.turnover causa radio.silence
Significativo: core.global.turnover causa radio.silence (p-value=0.005)

=== Risultati per il progetto: Bitcoin_report ===

Serie costante per prima.donnas. Test ADF non eseguito.
Serie costante per black.cloud. Test ADF non eseguito.
radio.silence: Serie stazionaria (p-value=0.017).
missing.links: Serie non stazionaria, differenziata.
org.silo: Serie non stazionaria, differenziata.
global.turnover: Serie stazionaria (p-value=0.000).
code.turnover: Serie stazionaria (p-value=0.000).
core.global.turnover: Serie stazionaria (p-value=0.009).
core.mail.turnover: Serie non stazionaria, differenziata.
core.code.turnover: Serie stazionaria (p-value=0.001).
ratio.smelly.quitters: Serie stazionaria (p-value=0.000).

Test di Granger: core.mail.turnover causa radio.silence
Significativo: core.mail.turnover causa radio.silence (p-value=0.025)

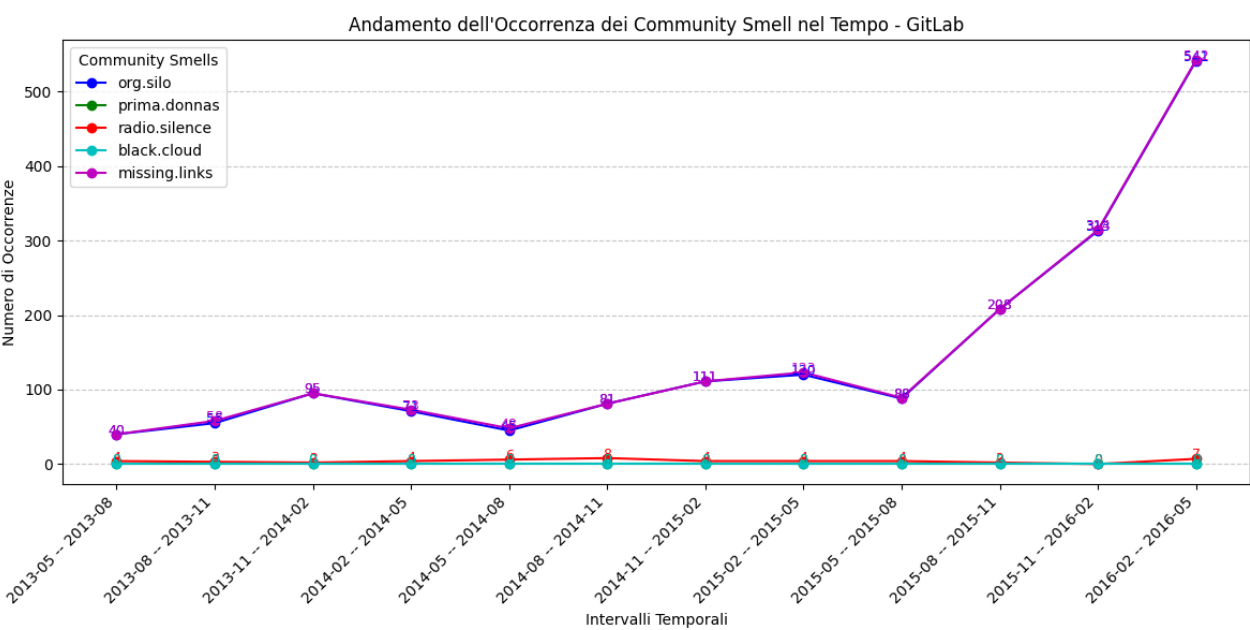
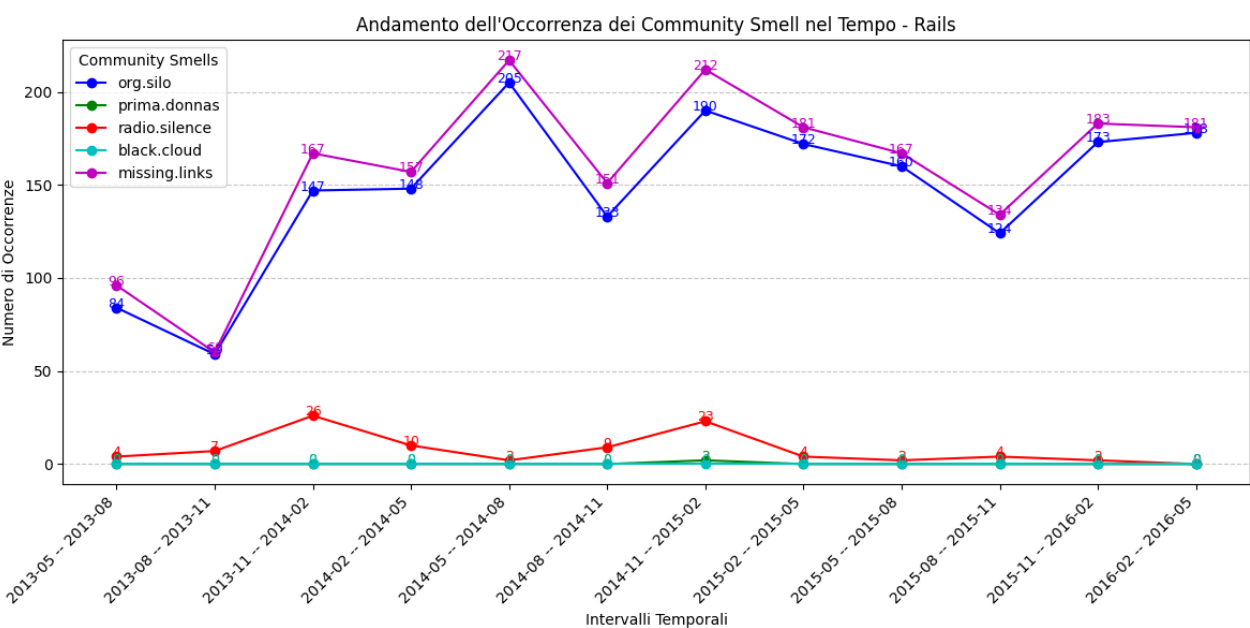
Test di Granger: core.code.turnover causa missing.links
Significativo: core.code.turnover causa missing.links (p-value=0.005)

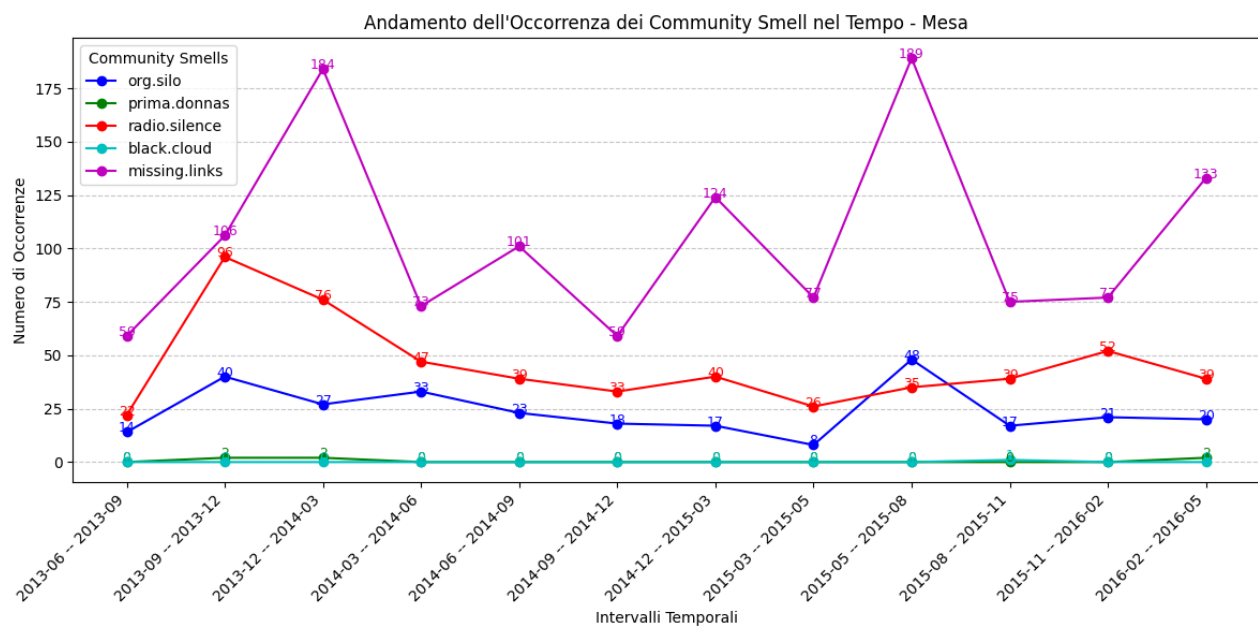
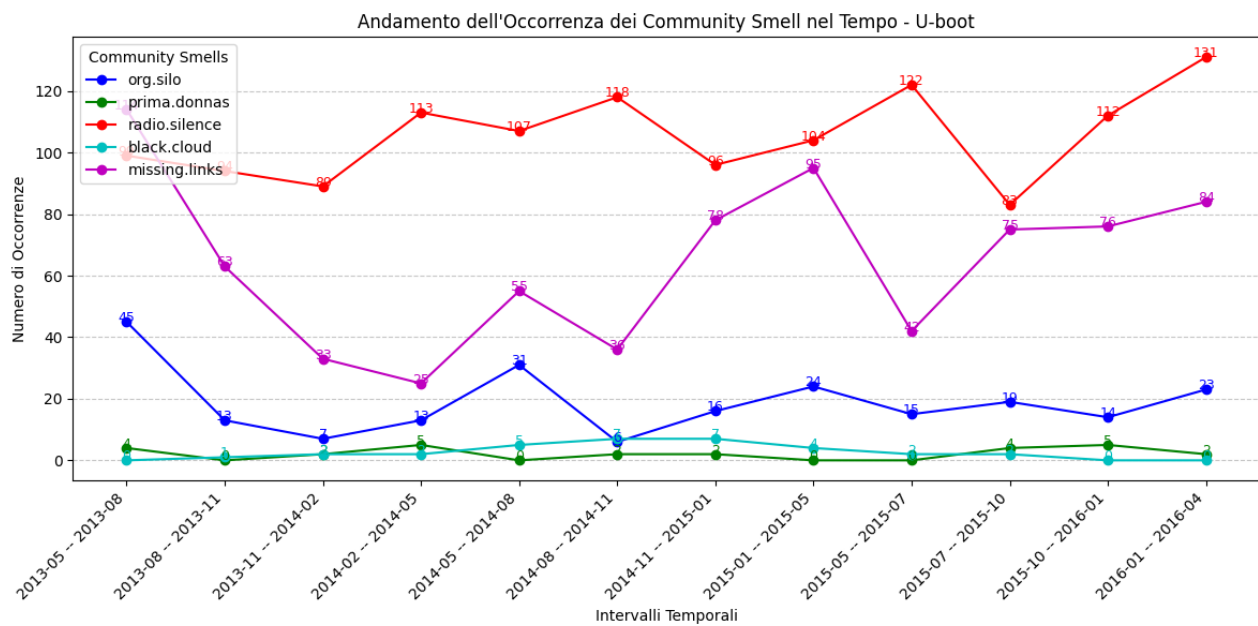
Test di Granger: ratio.smelly.quitters causa missing.links
Significativo: ratio.smelly.quitters causa missing.links (p-value=0.007)

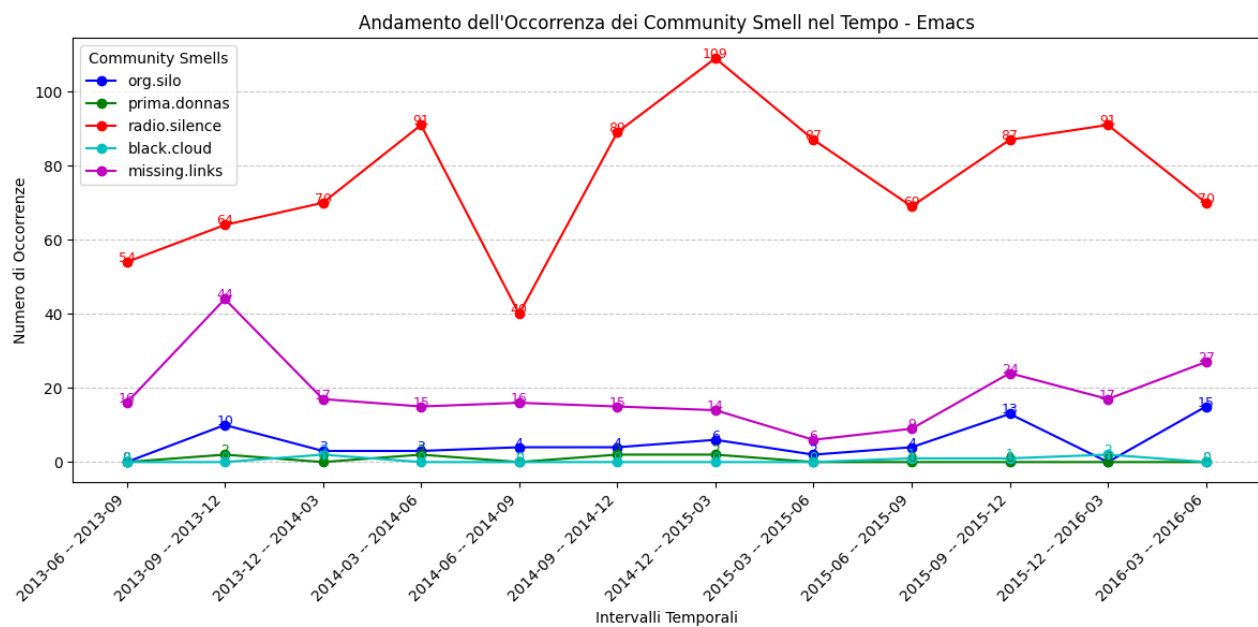
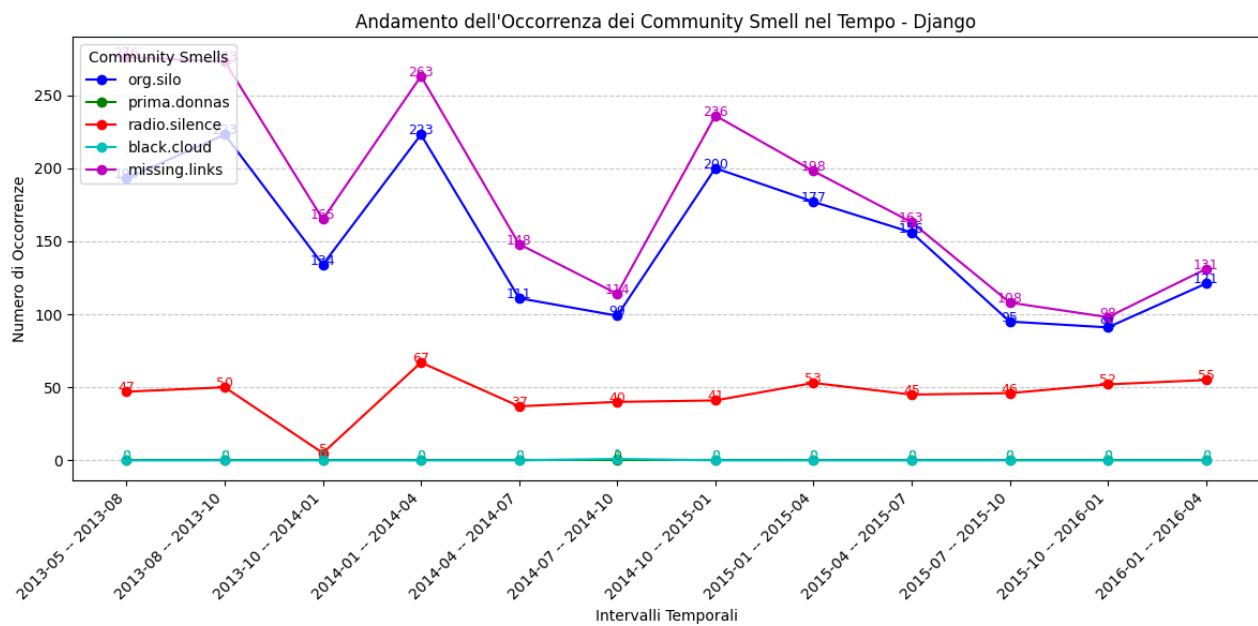
Test di Granger: core.code.turnover causa org.silo
Significativo: core.code.turnover causa org.silo (p-value=0.013)

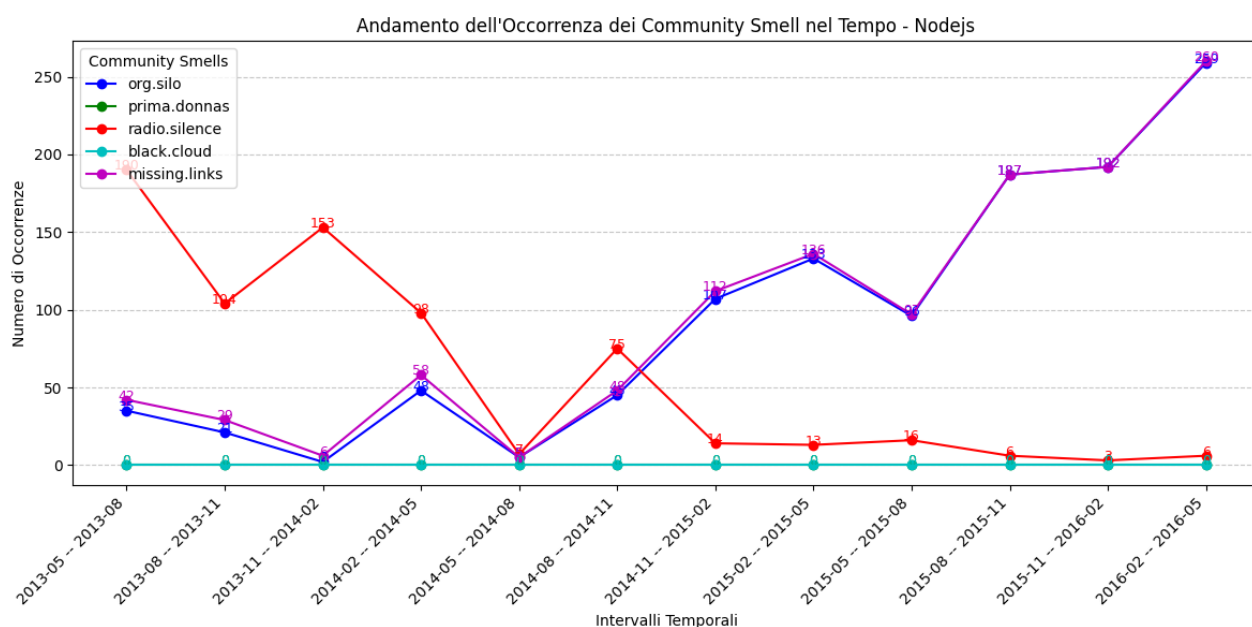
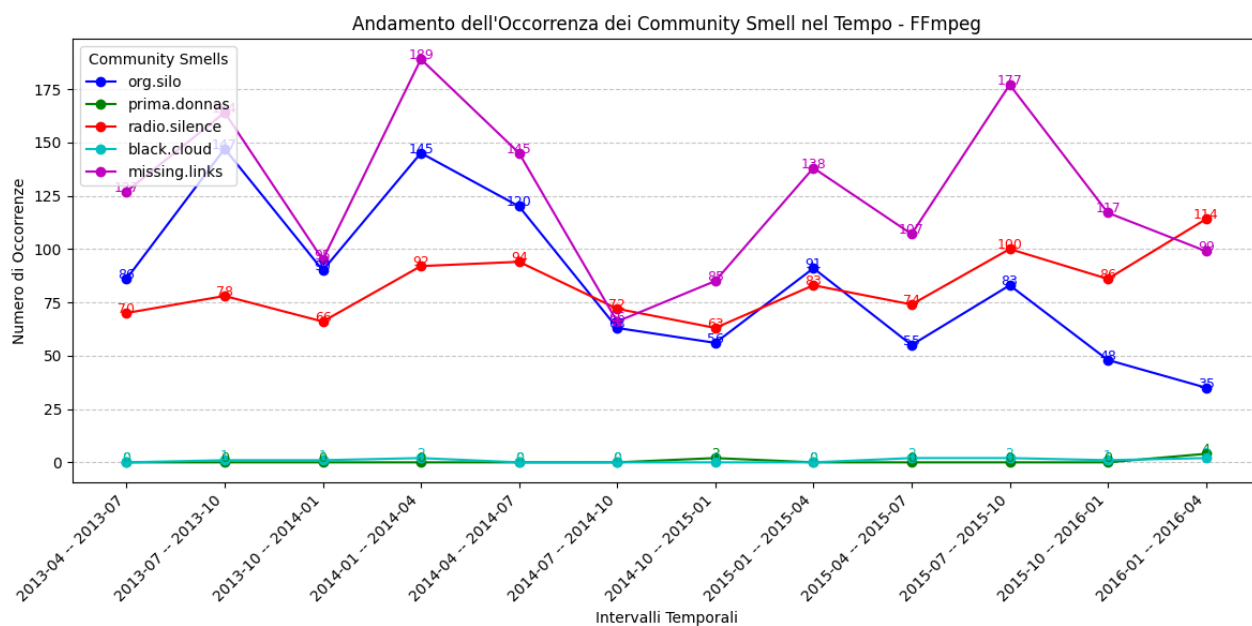
Test di Granger: ratio.smelly.quitters causa org.silo
Significativo: ratio.smelly.quitters causa org.silo (p-value=0.001)

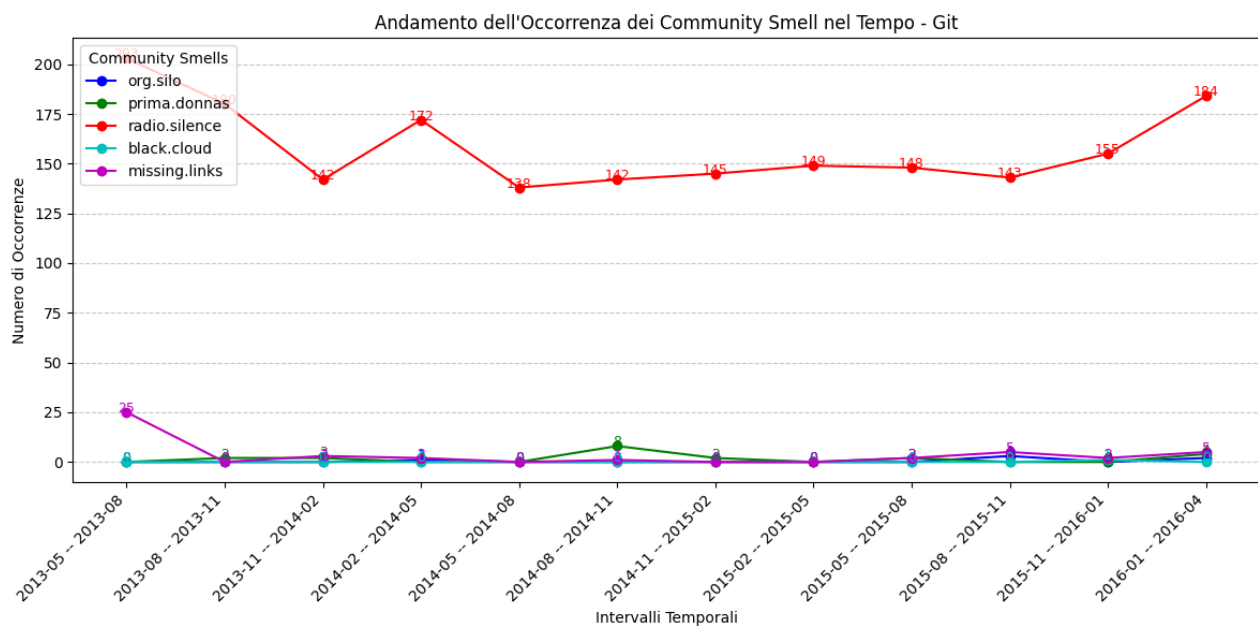
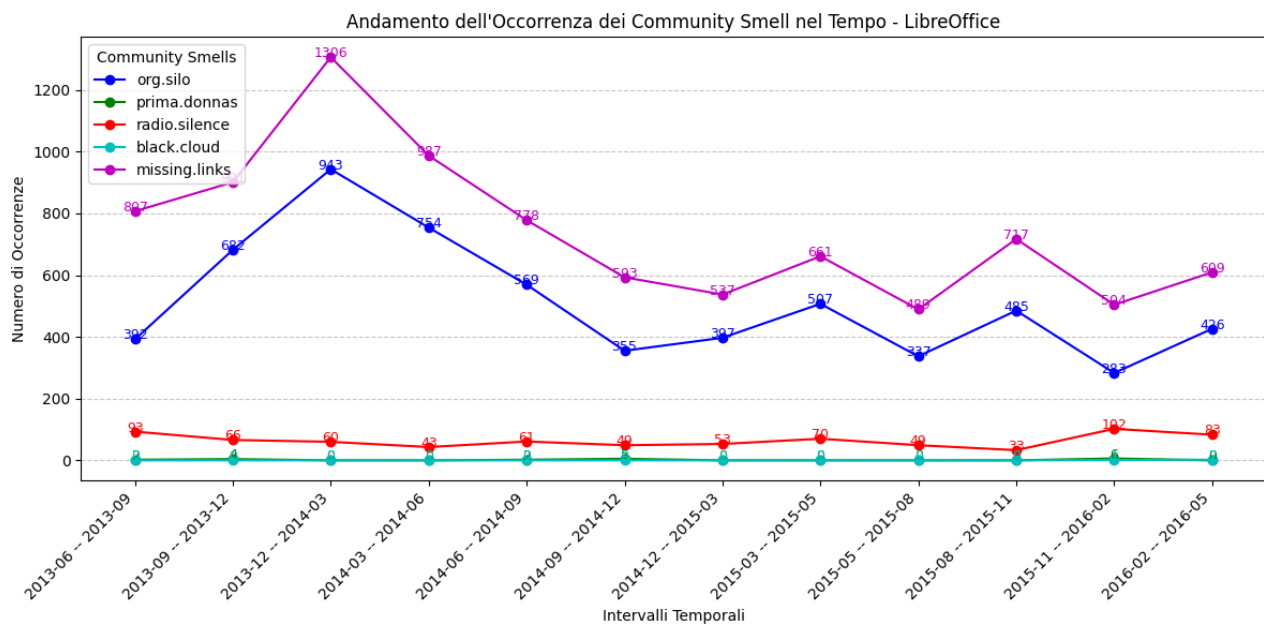
Appendice 3: Grafici andamento temporale occorrenze community smell

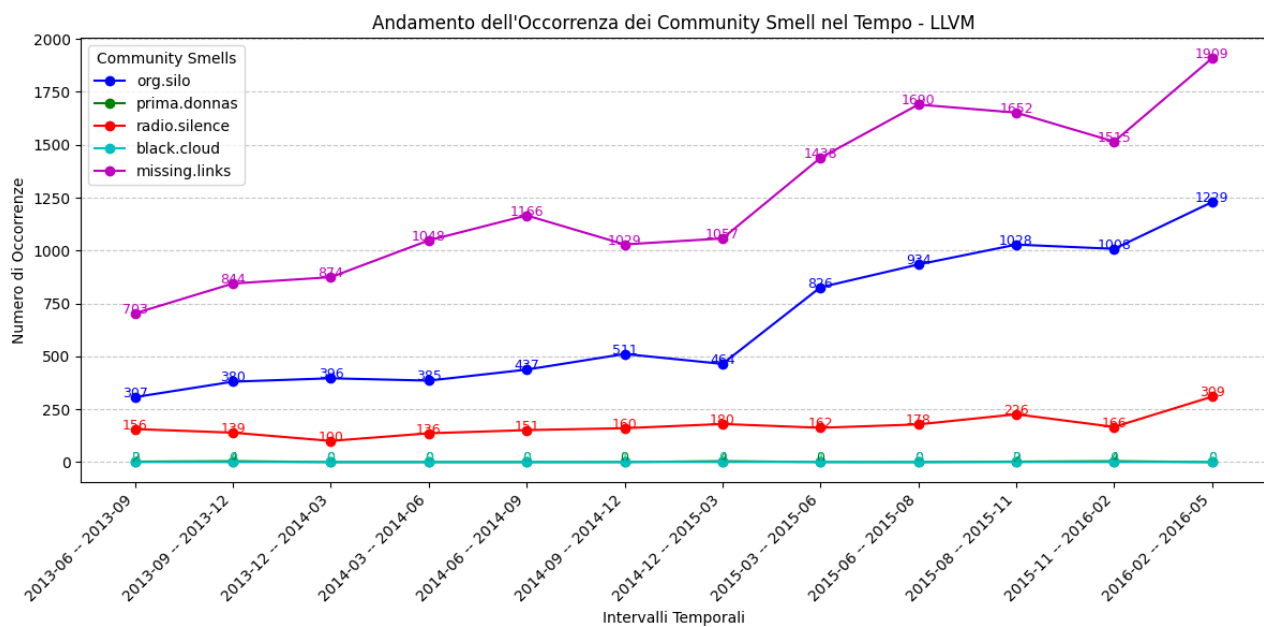
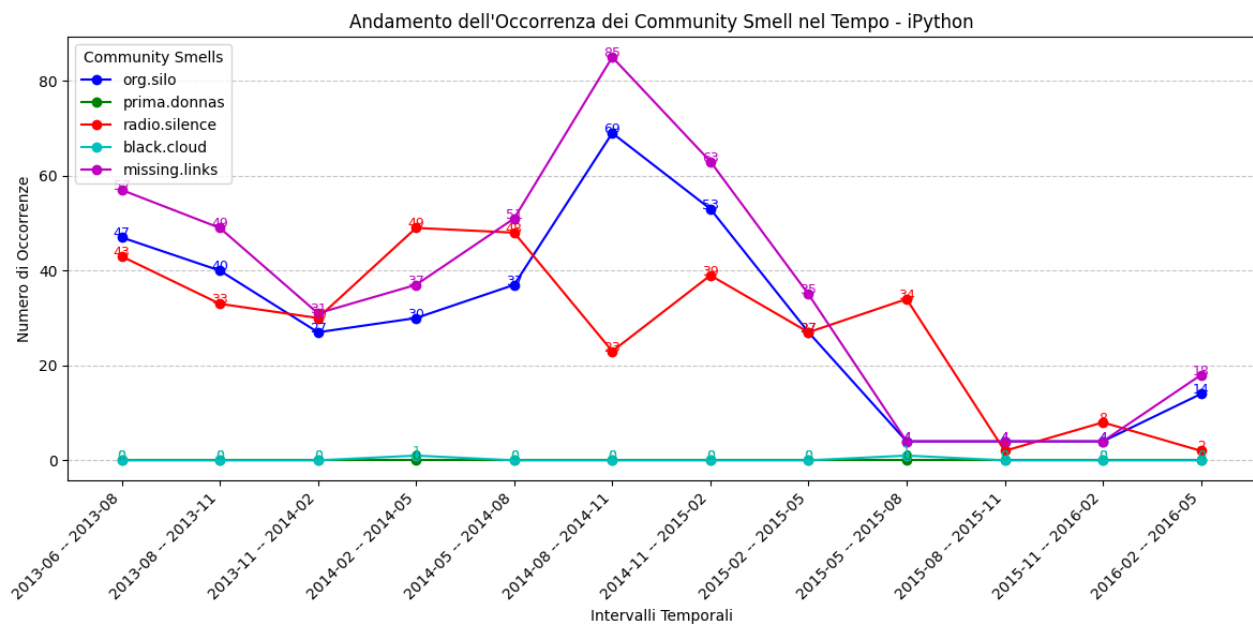


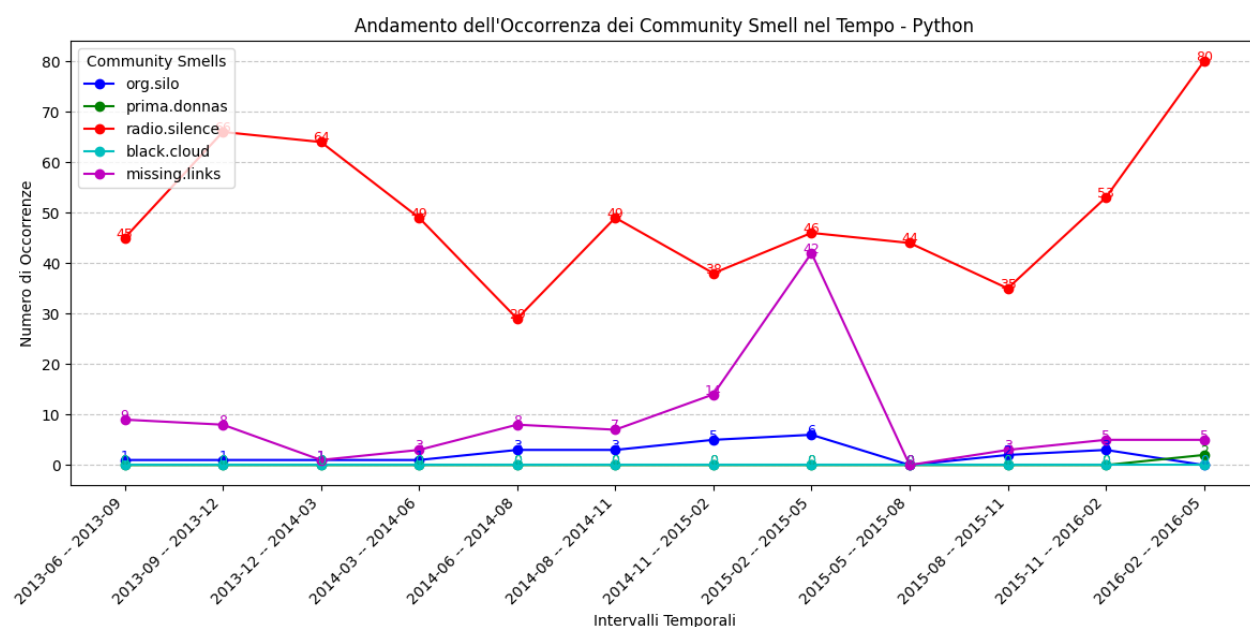
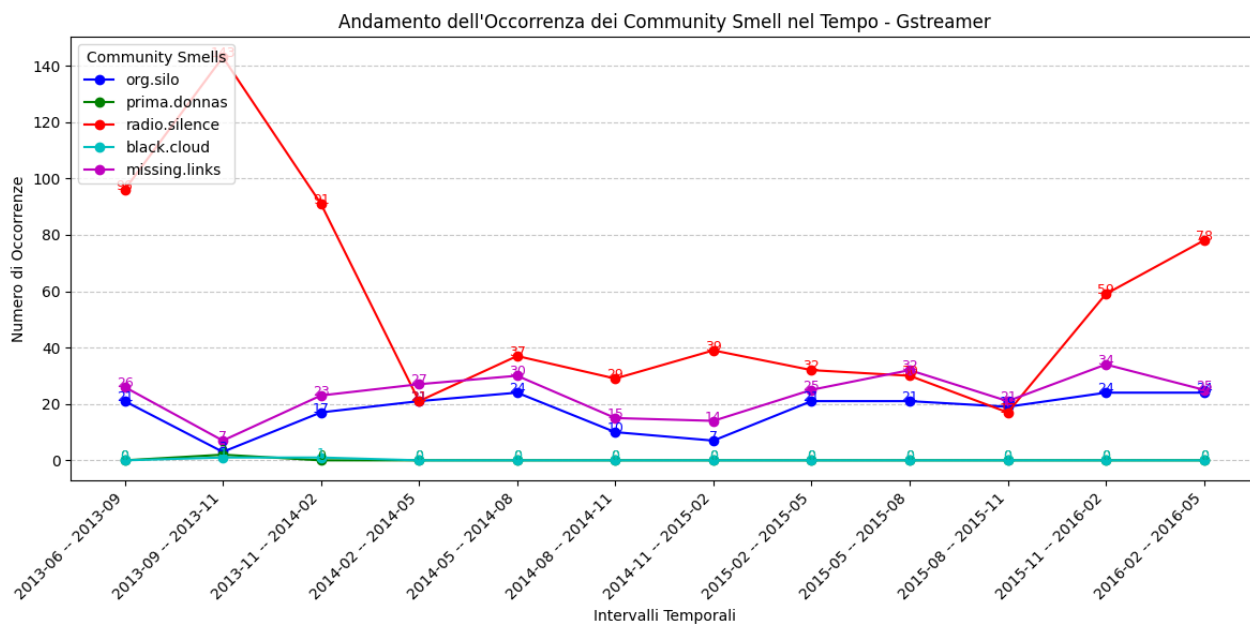


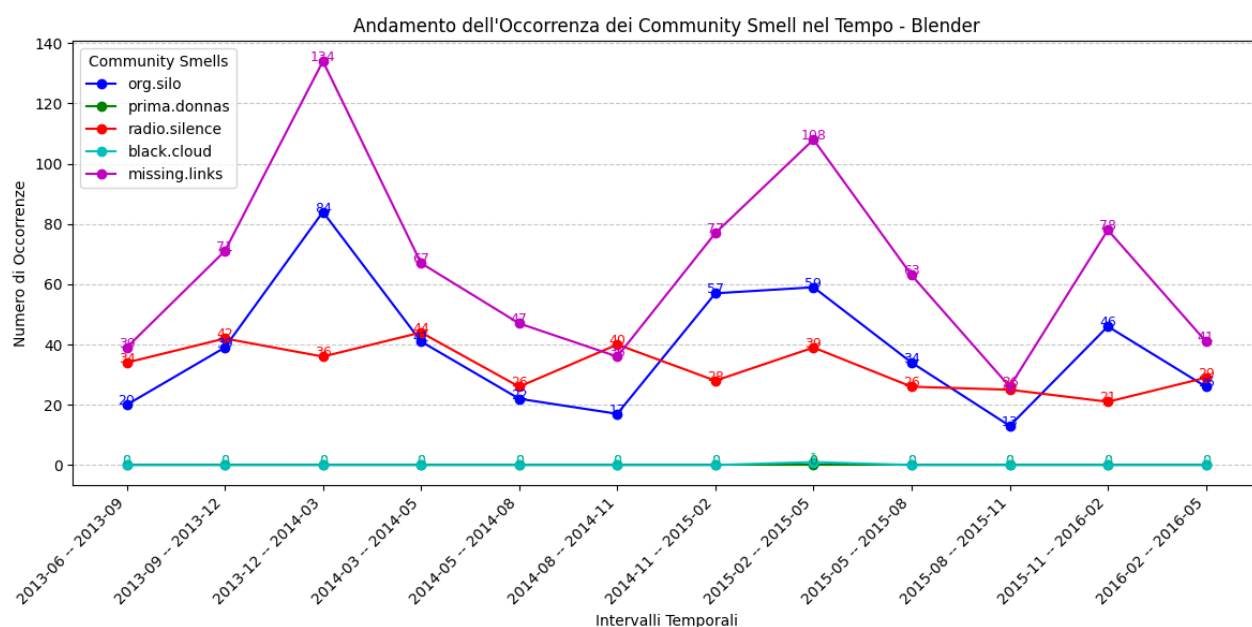
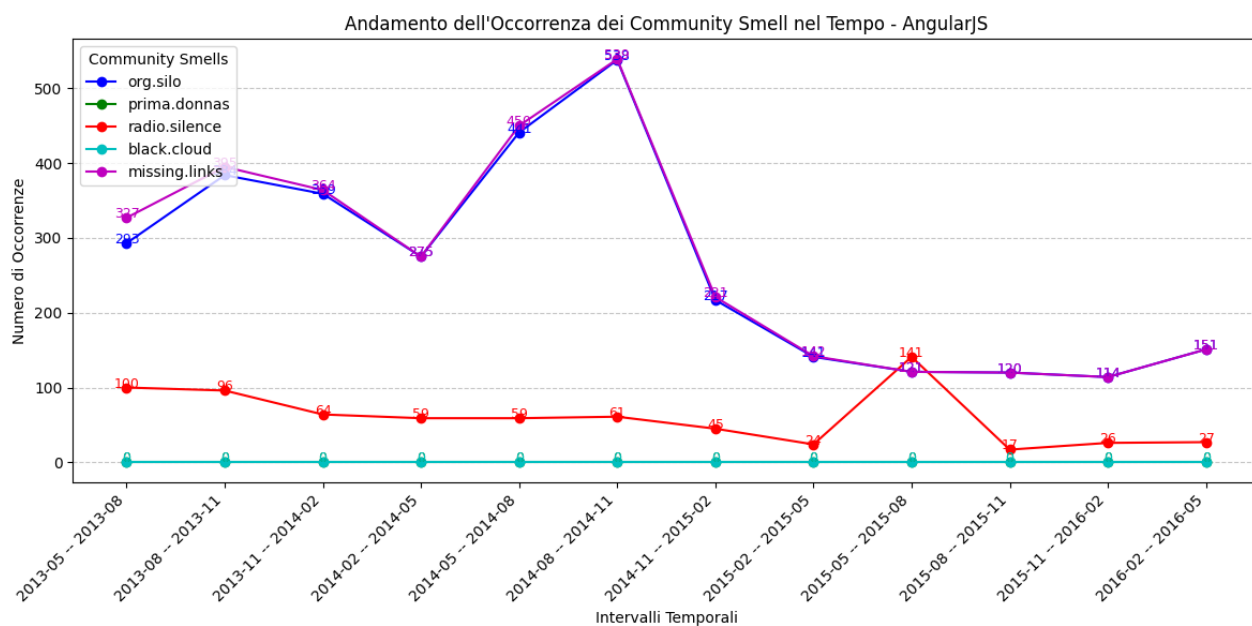




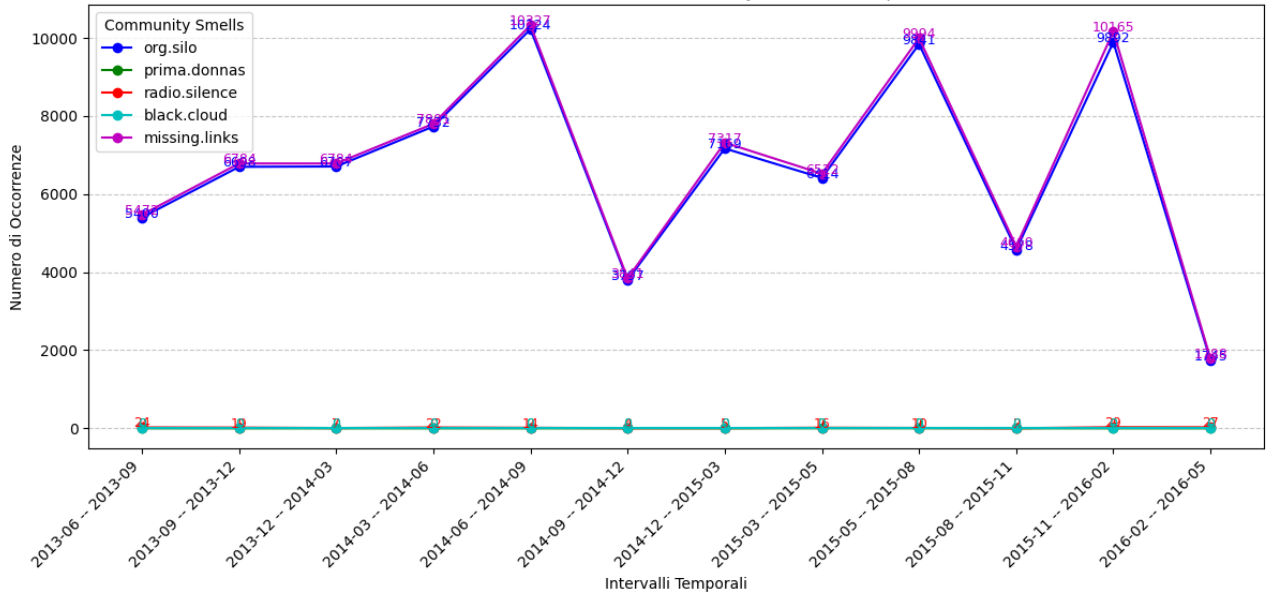




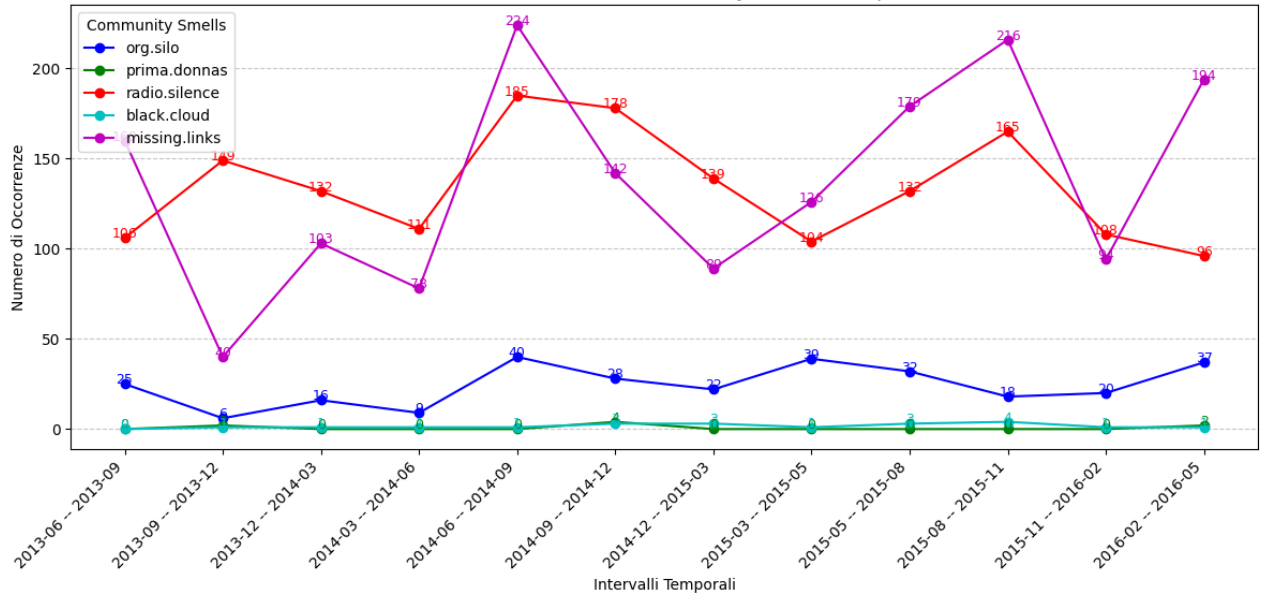


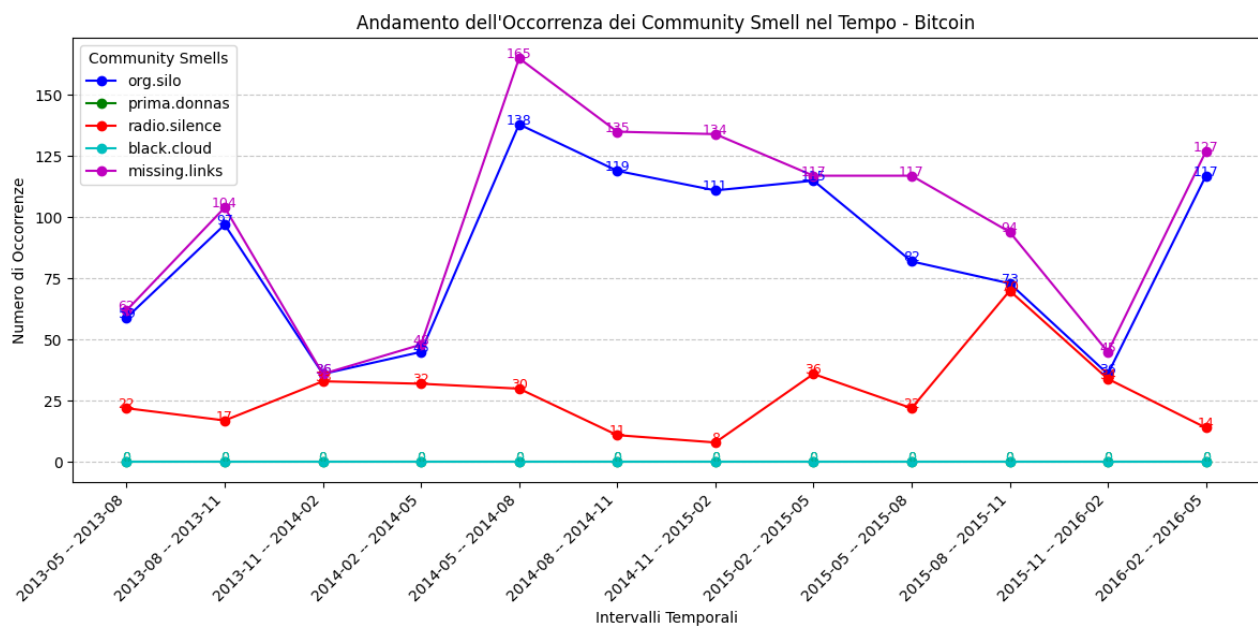
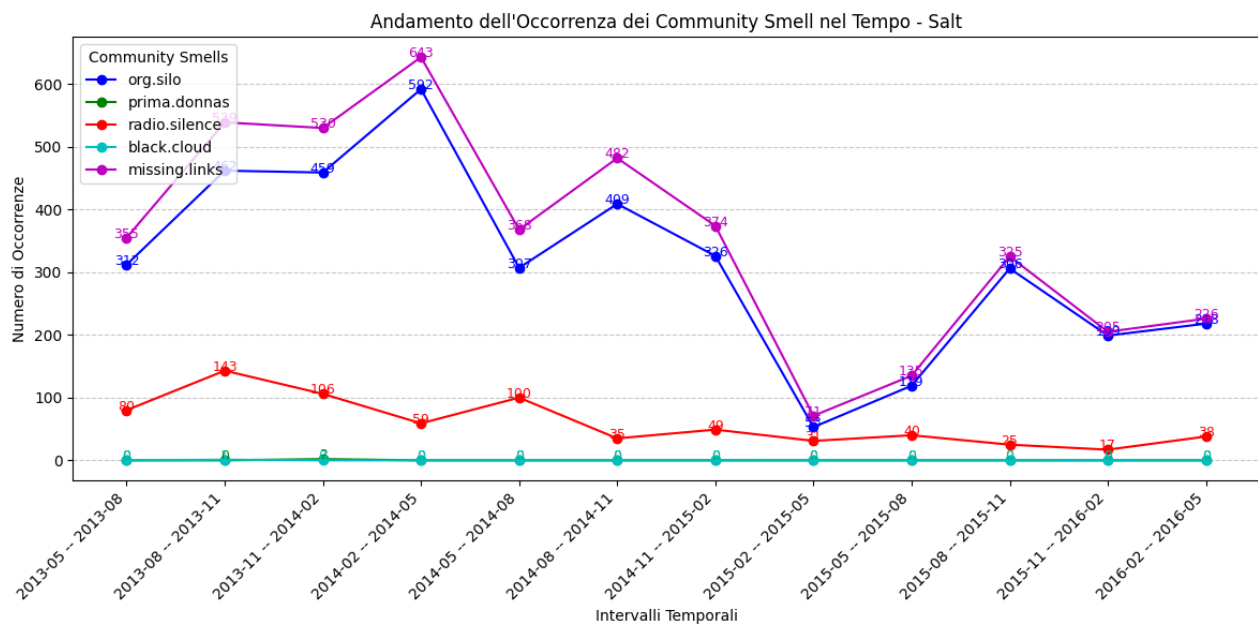


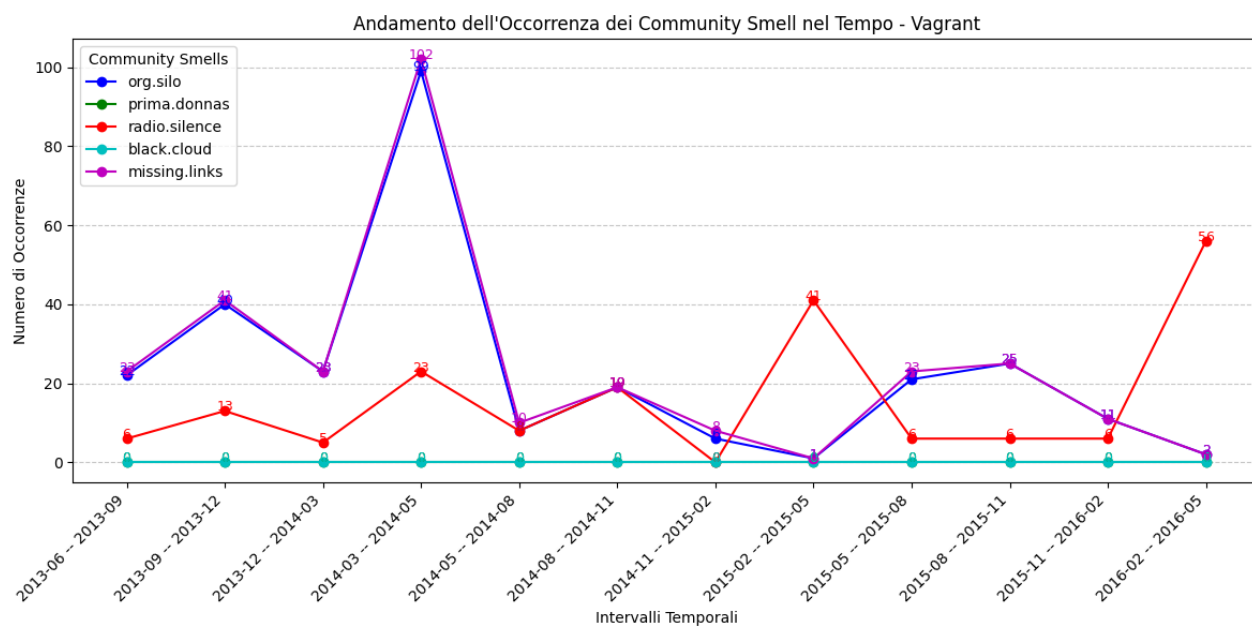
Andamento dell'Occorrenza dei Community Smell nel Tempo - Firefox



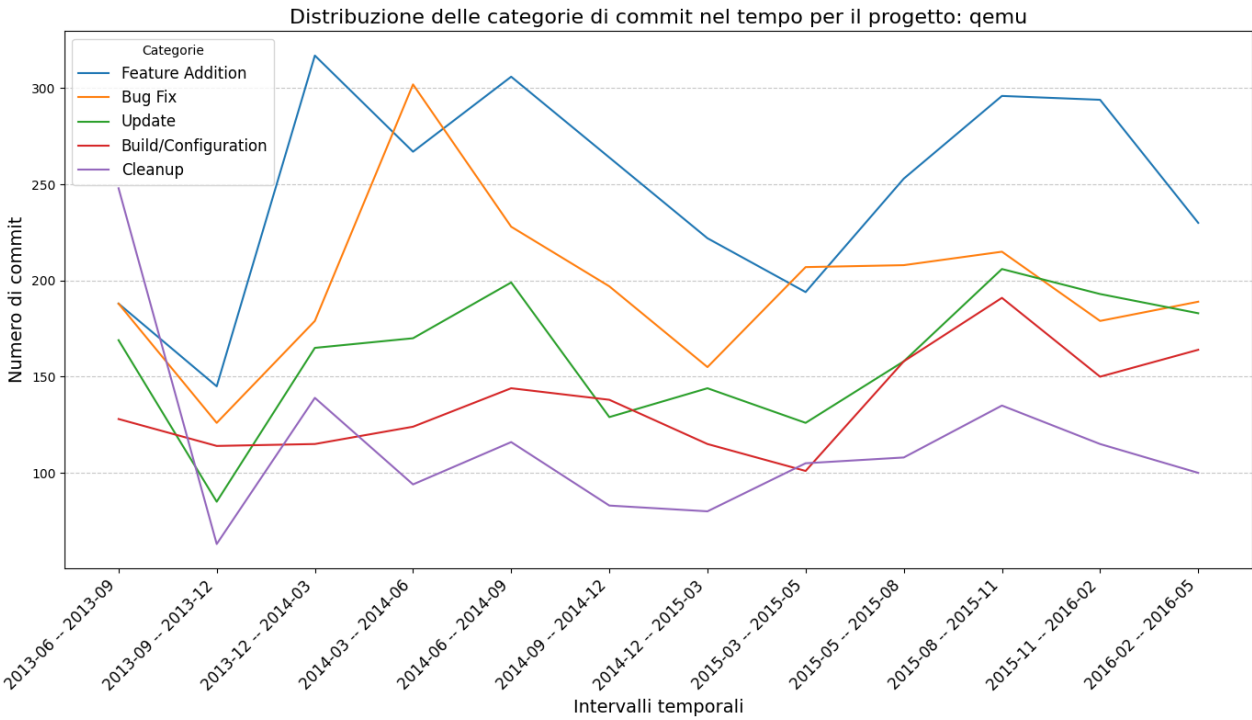
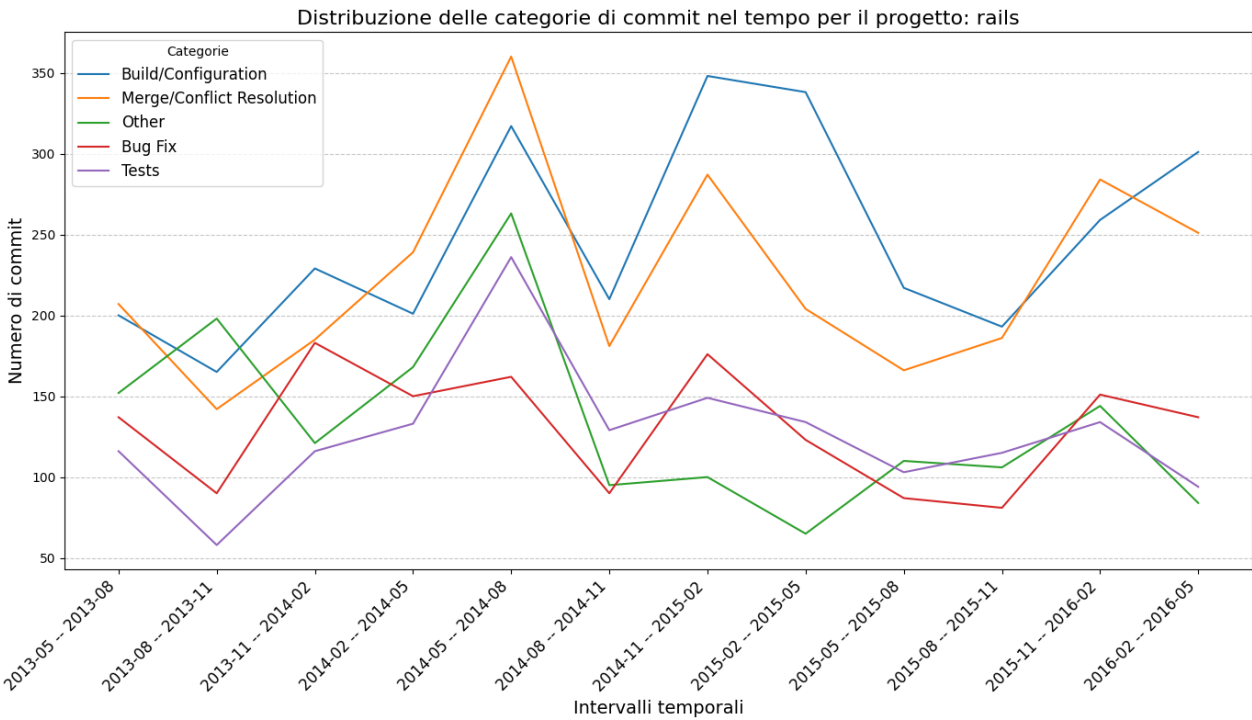
Andamento dell'Occorrenza dei Community Smell nel Tempo - QEMU



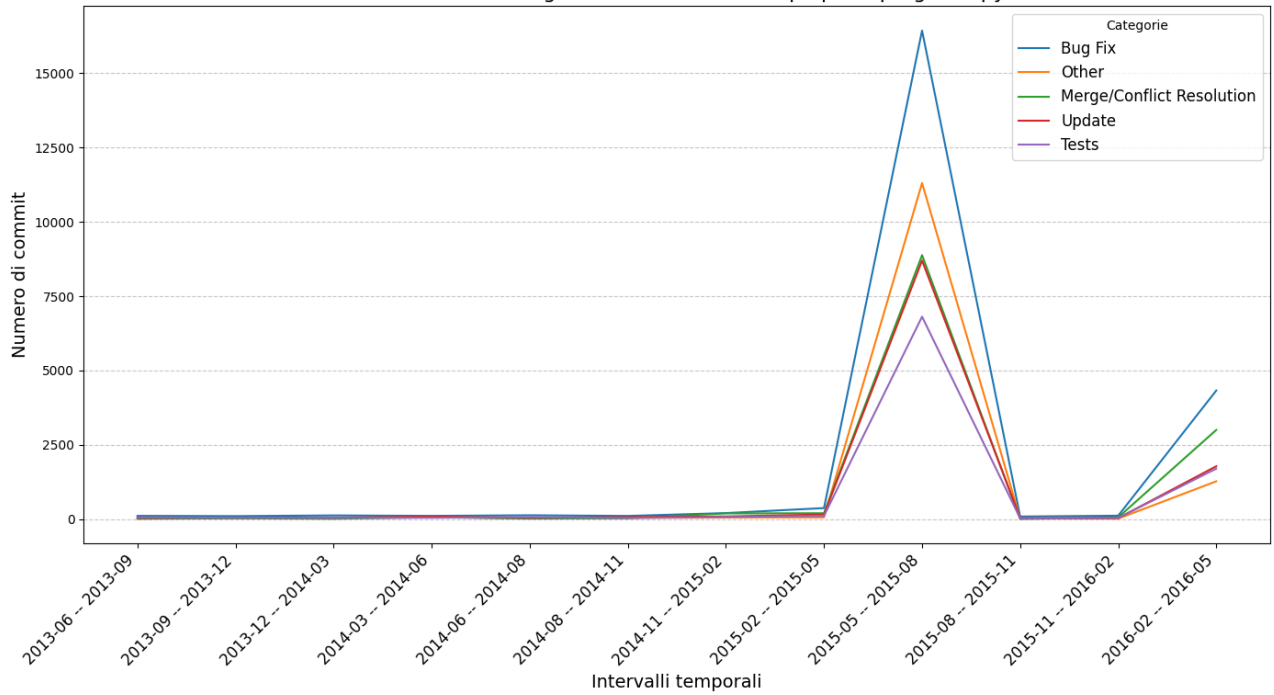




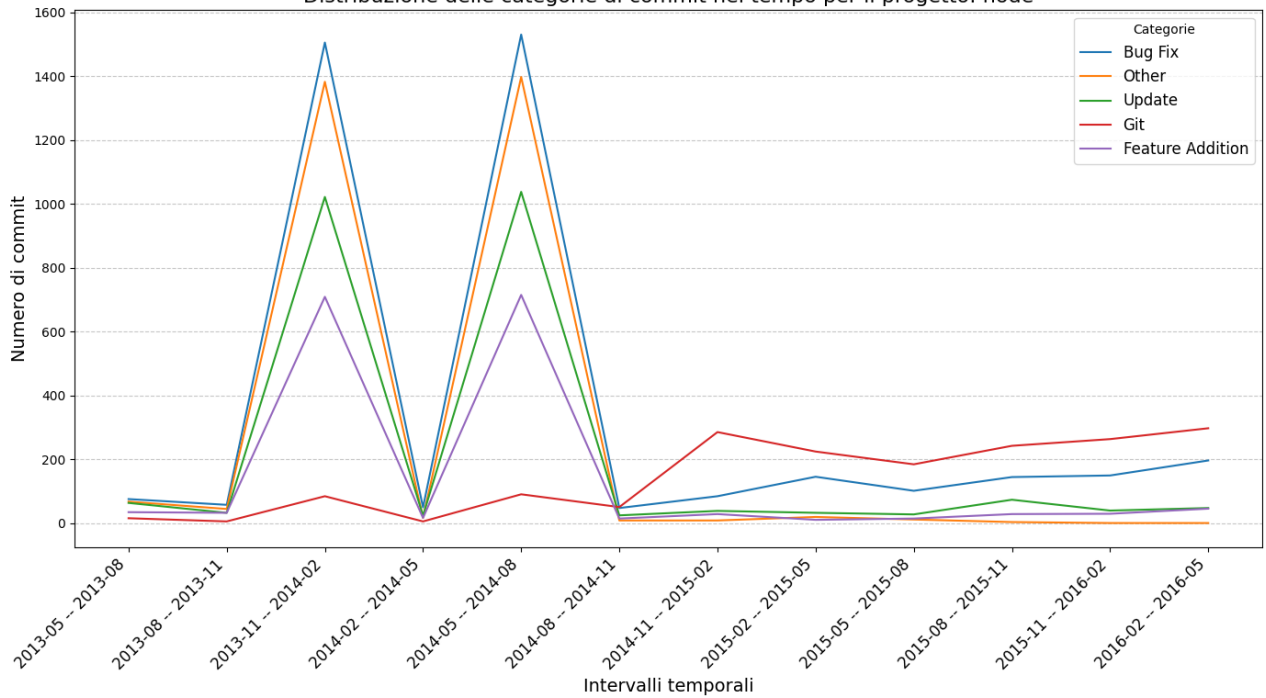
Appendice 4: Andamento temporale delle categorie di commit per progetto



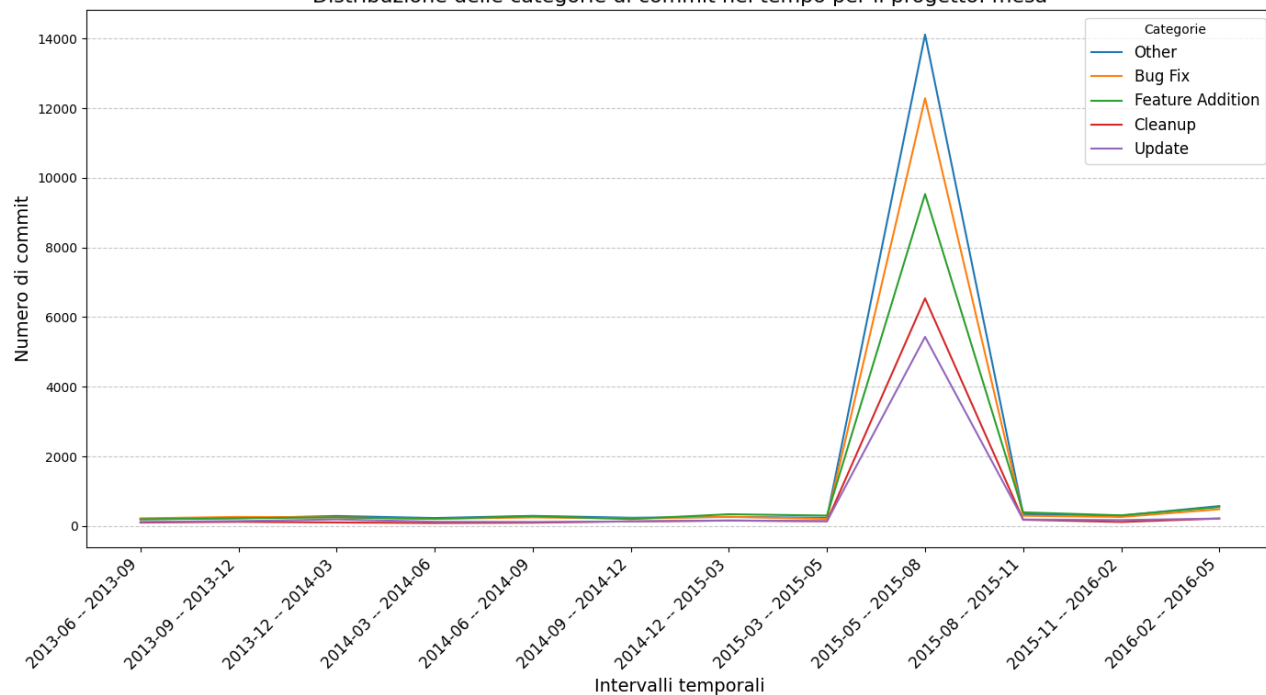
Distribuzione delle categorie di commit nel tempo per il progetto: python



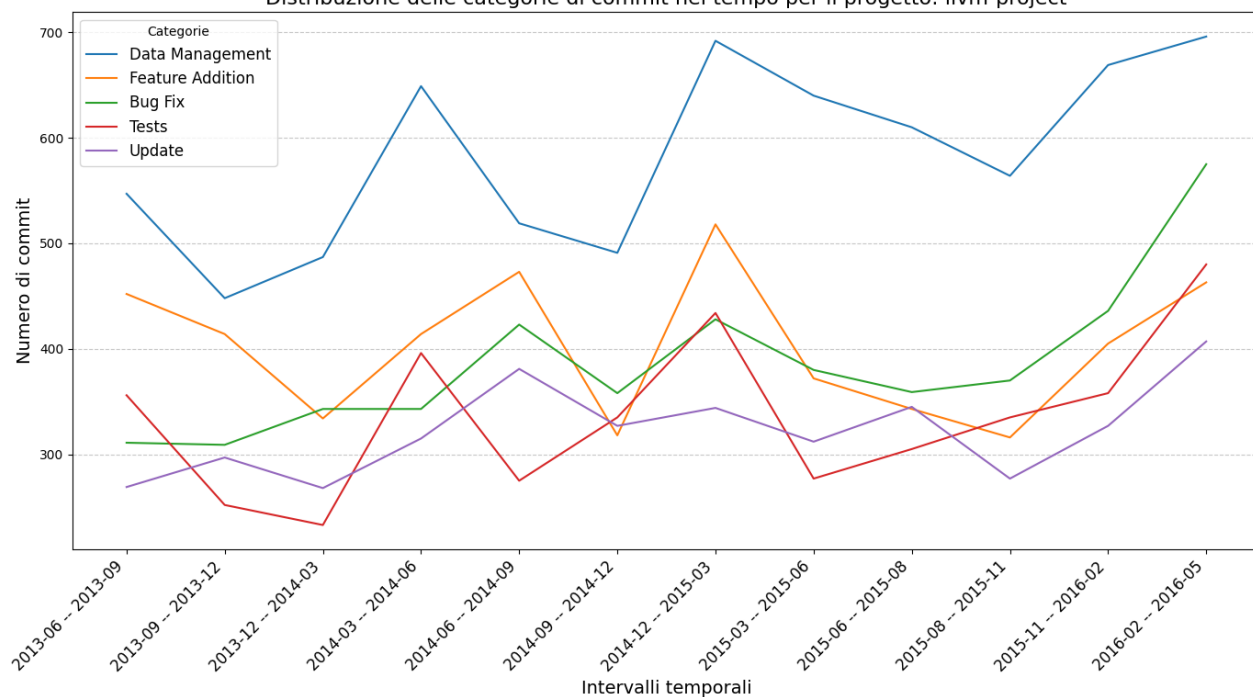
Distribuzione delle categorie di commit nel tempo per il progetto: node

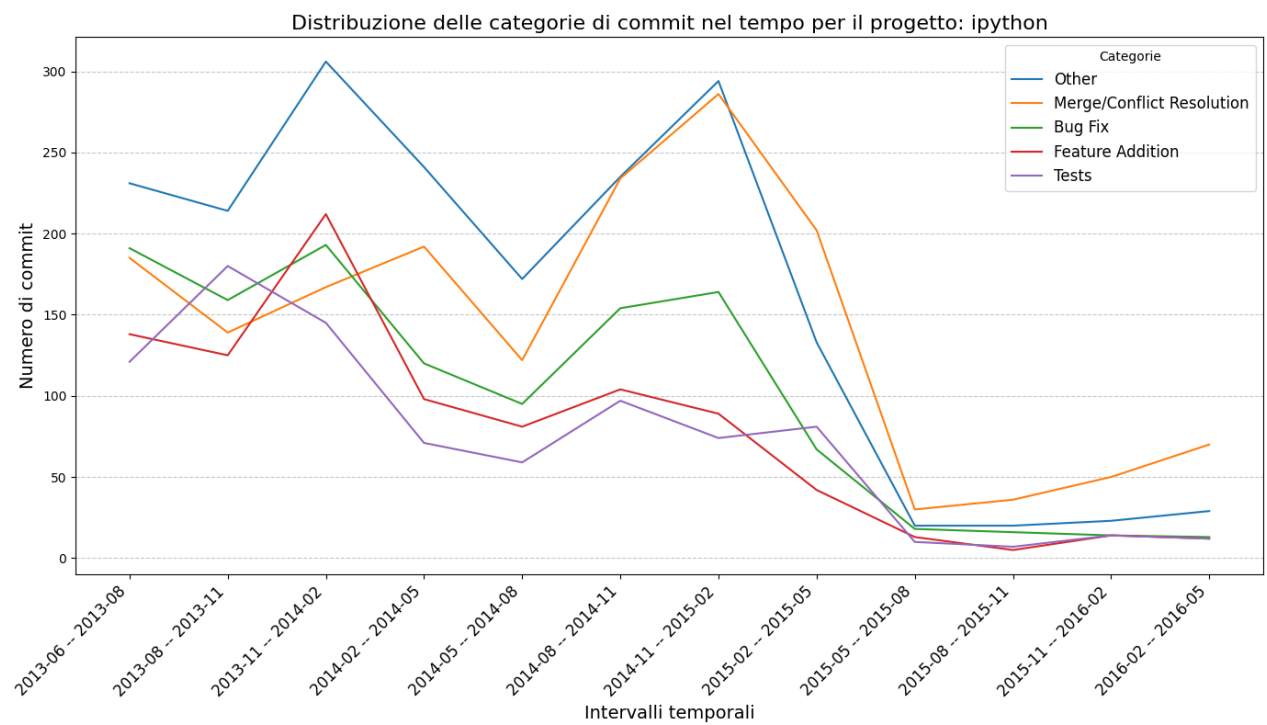
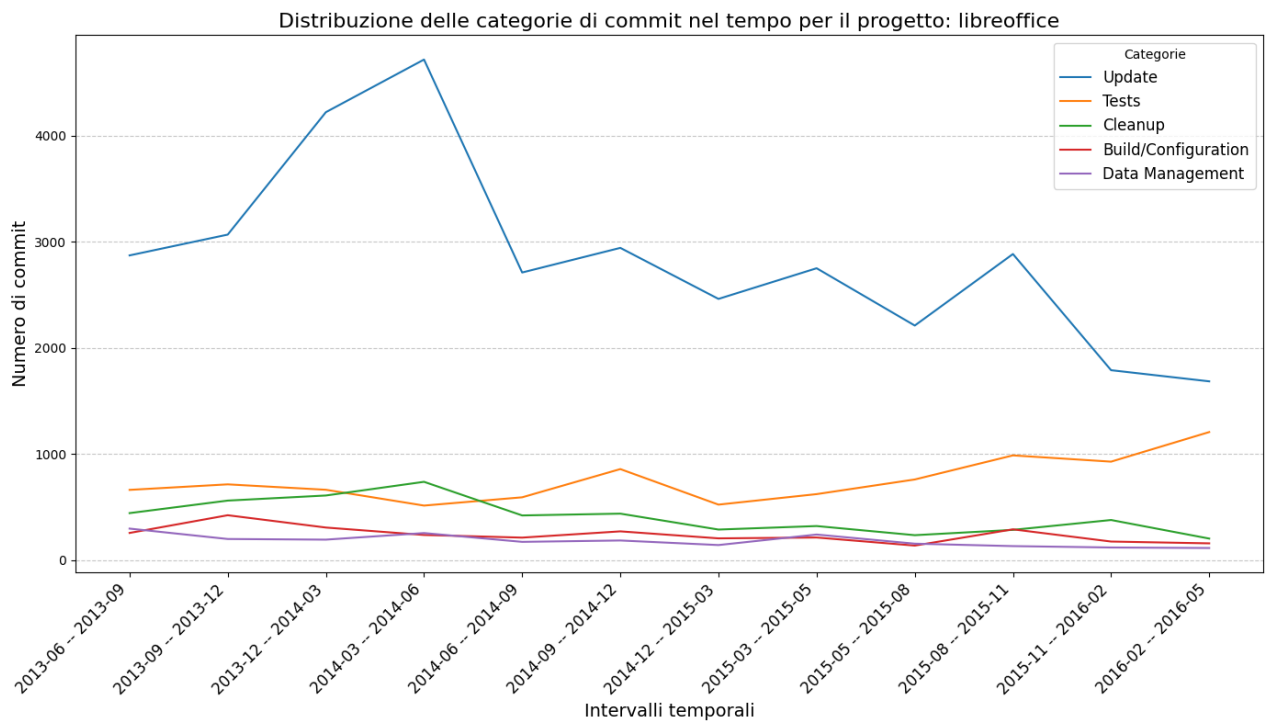


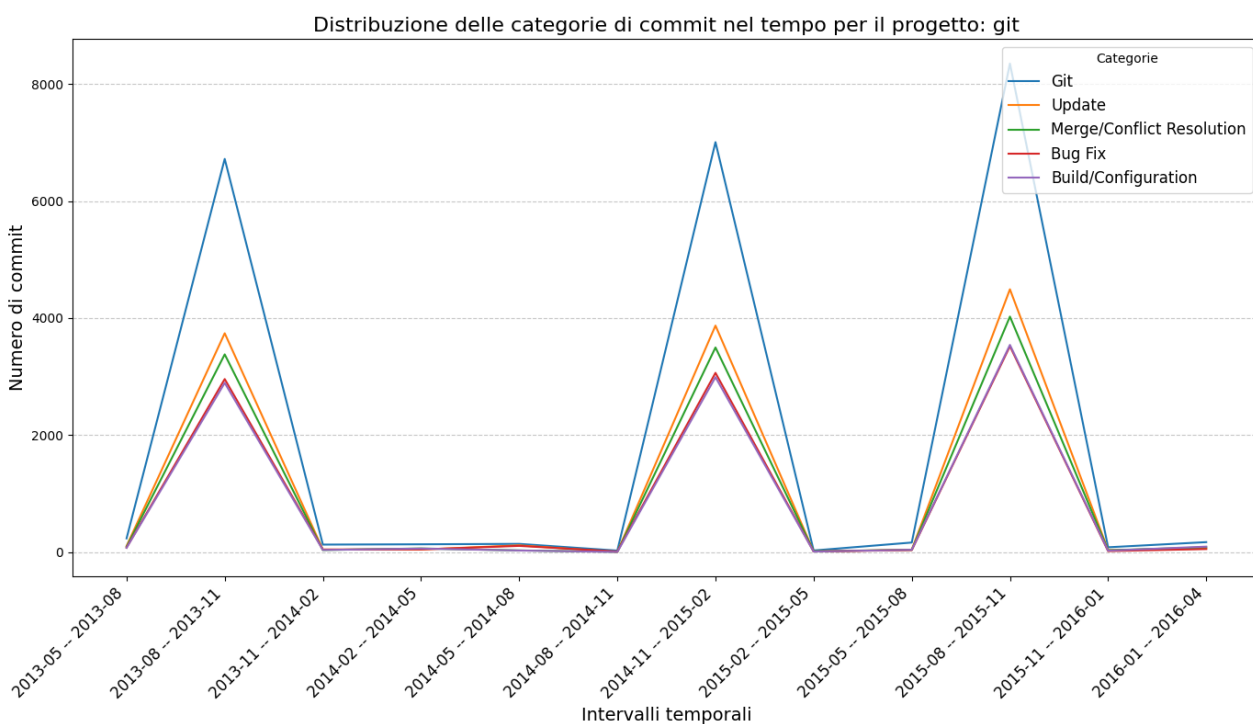
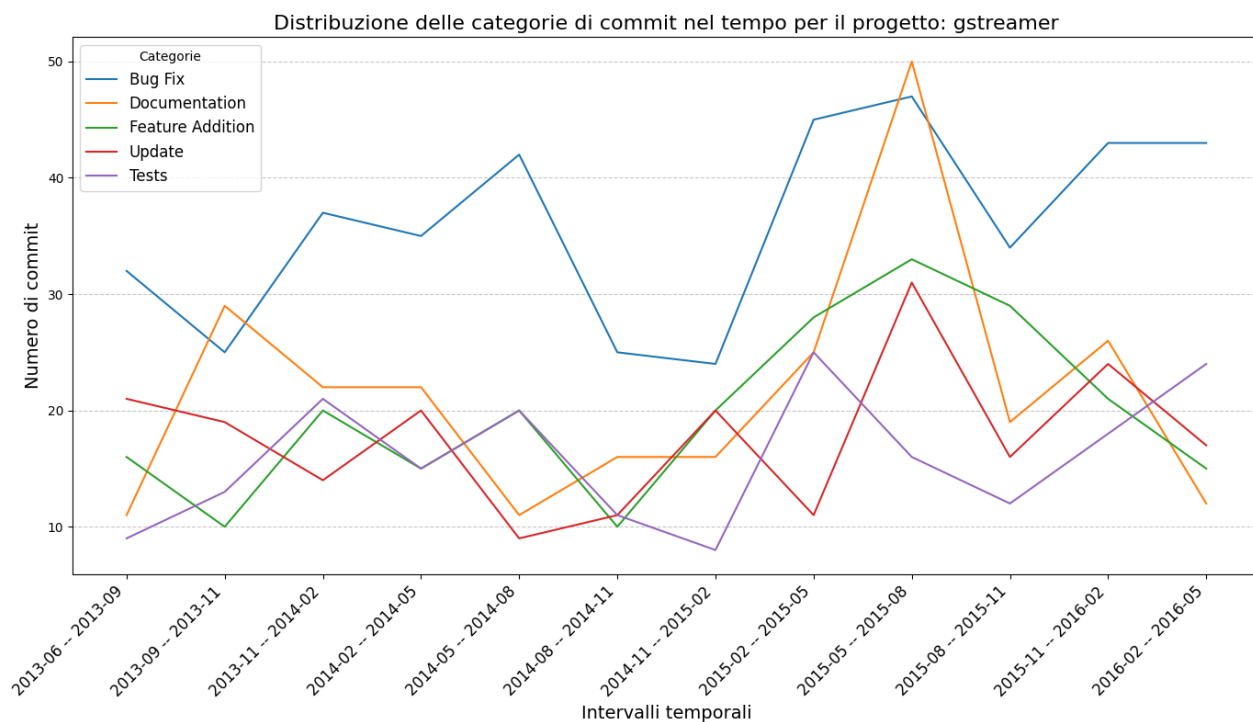
Distribuzione delle categorie di commit nel tempo per il progetto: mesa



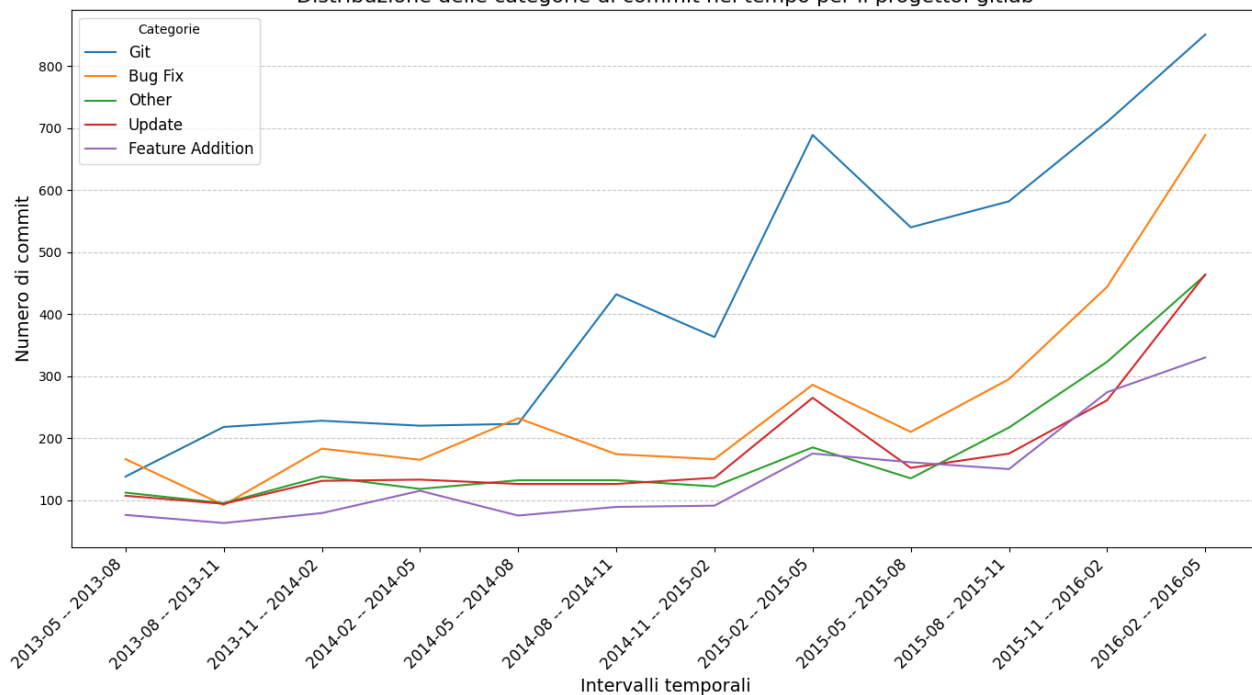
Distribuzione delle categorie di commit nel tempo per il progetto: llvm-project



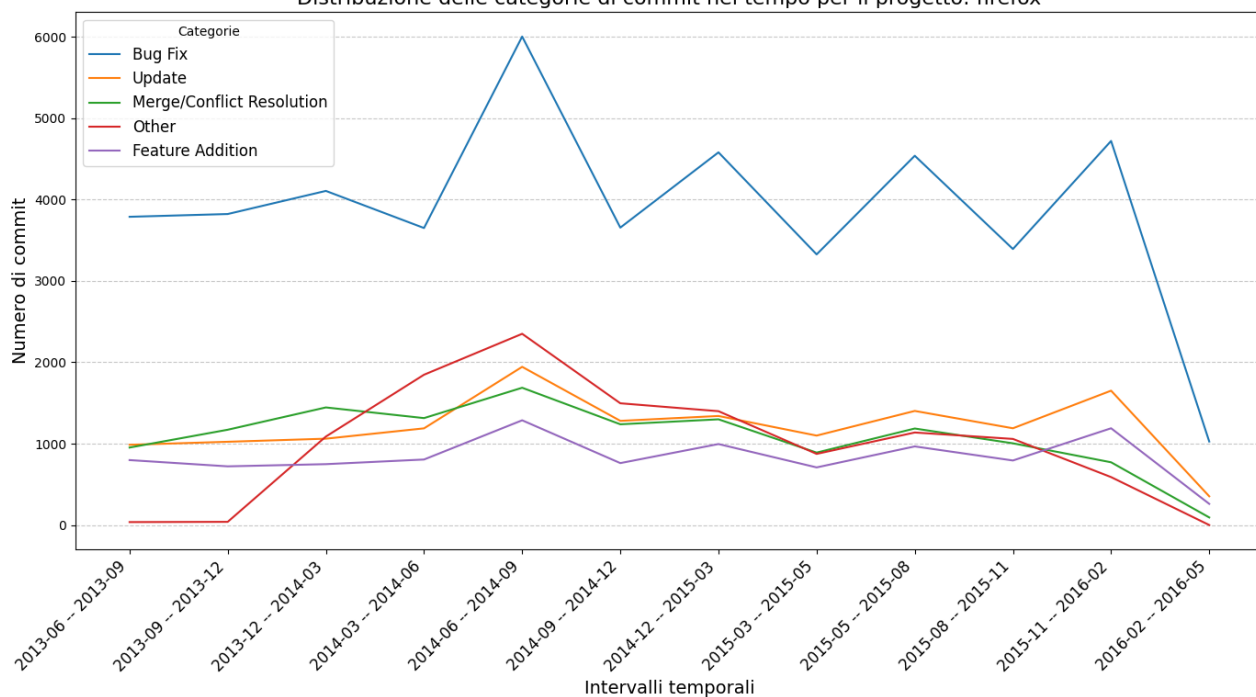


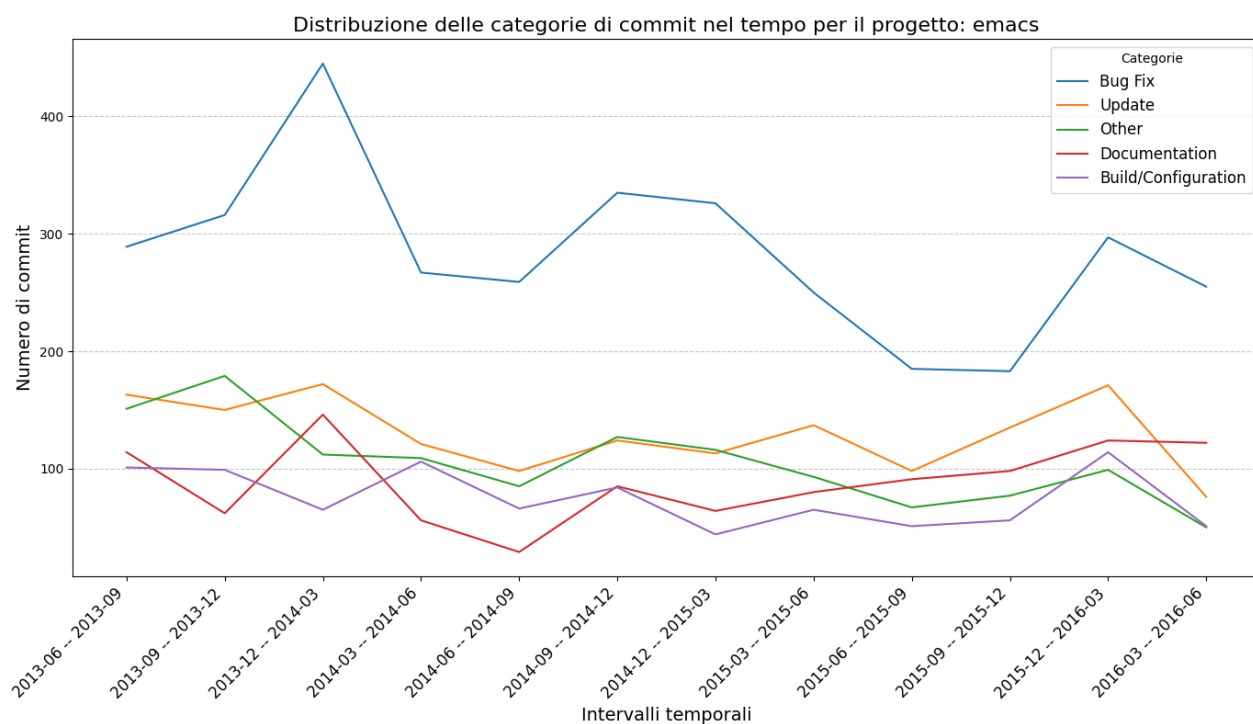
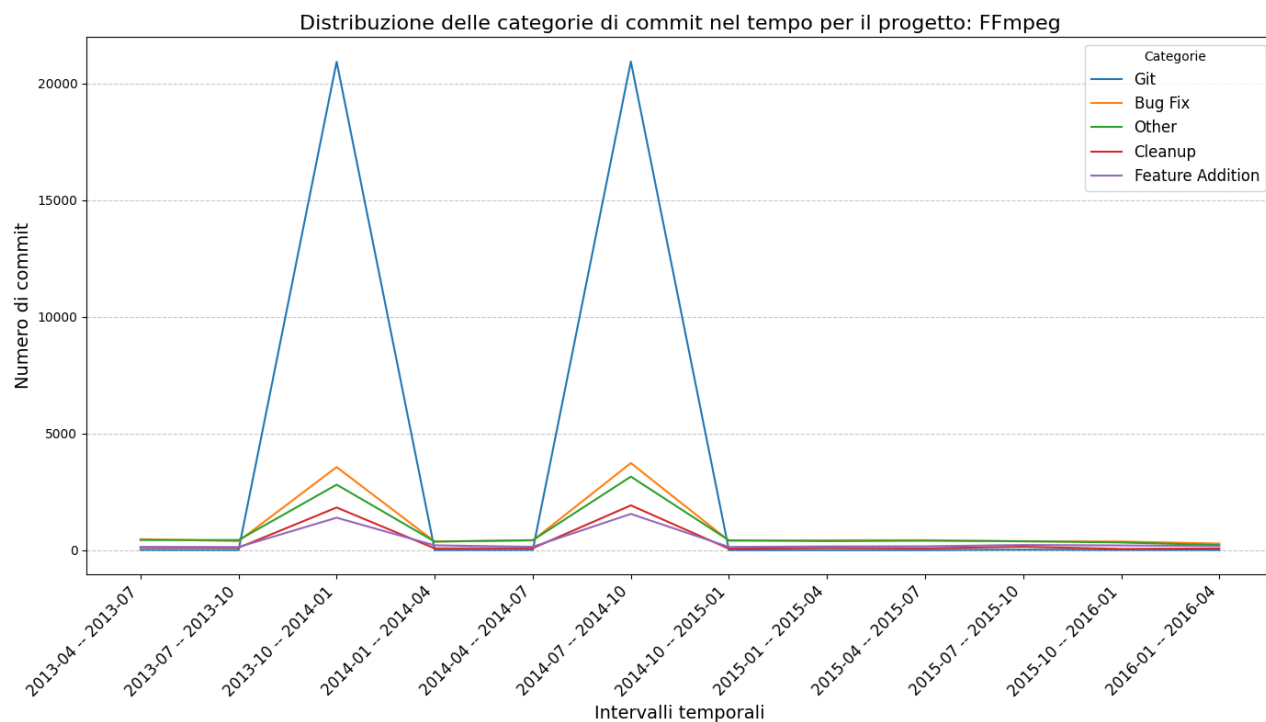


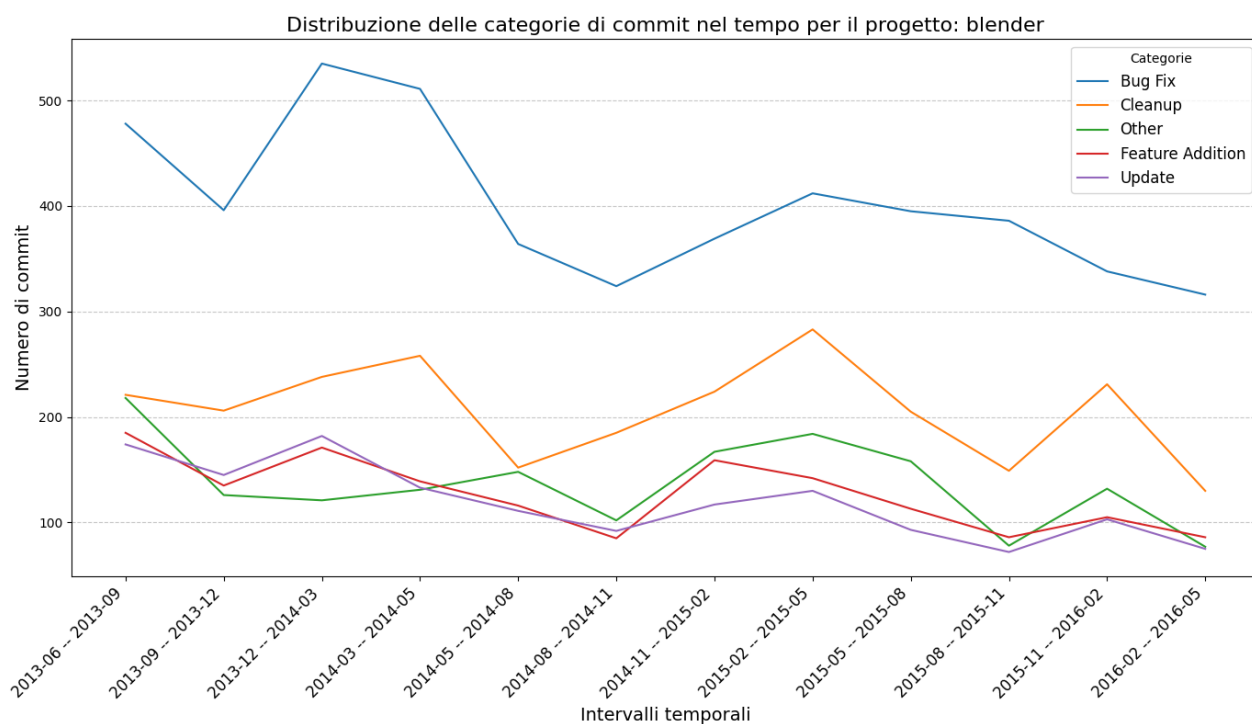
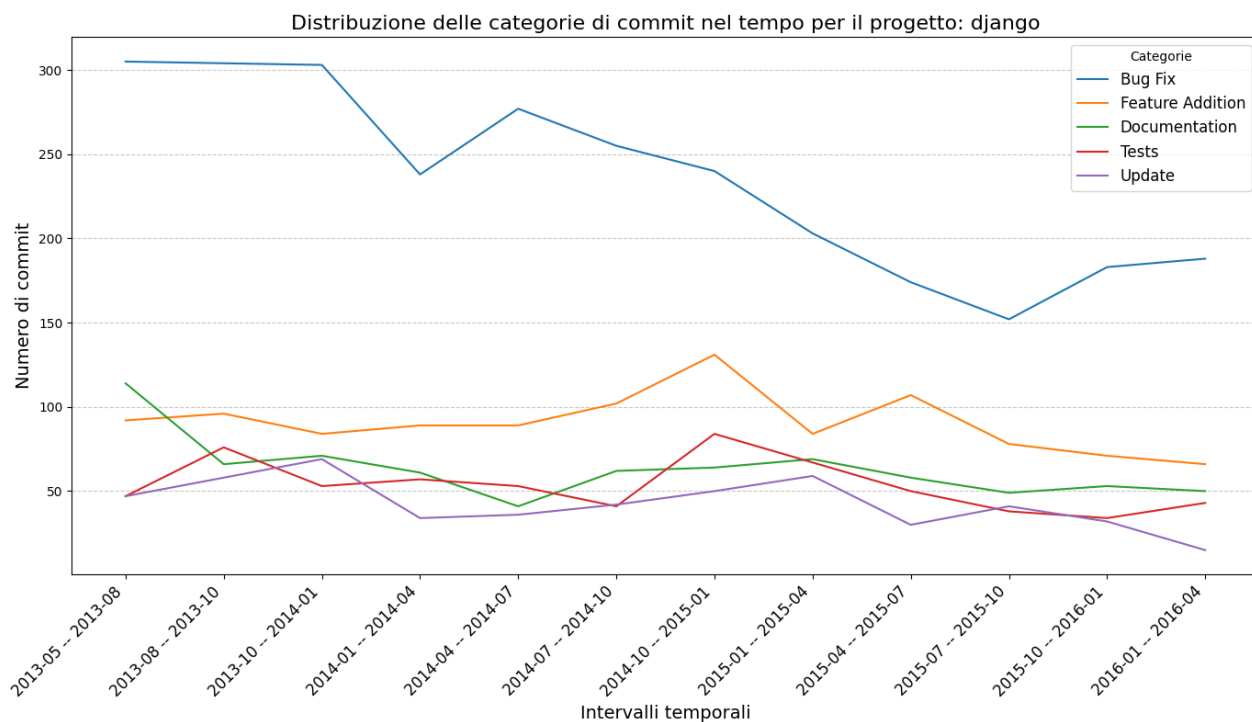
Distribuzione delle categorie di commit nel tempo per il progetto: gitlab

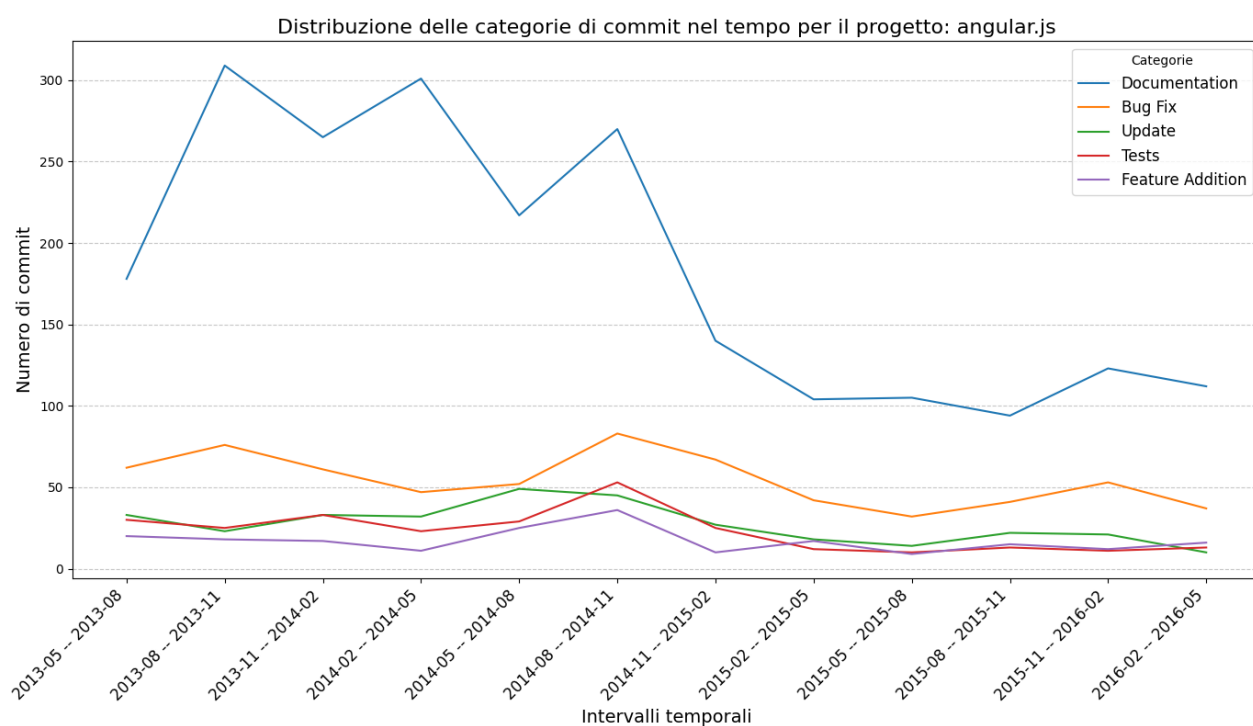
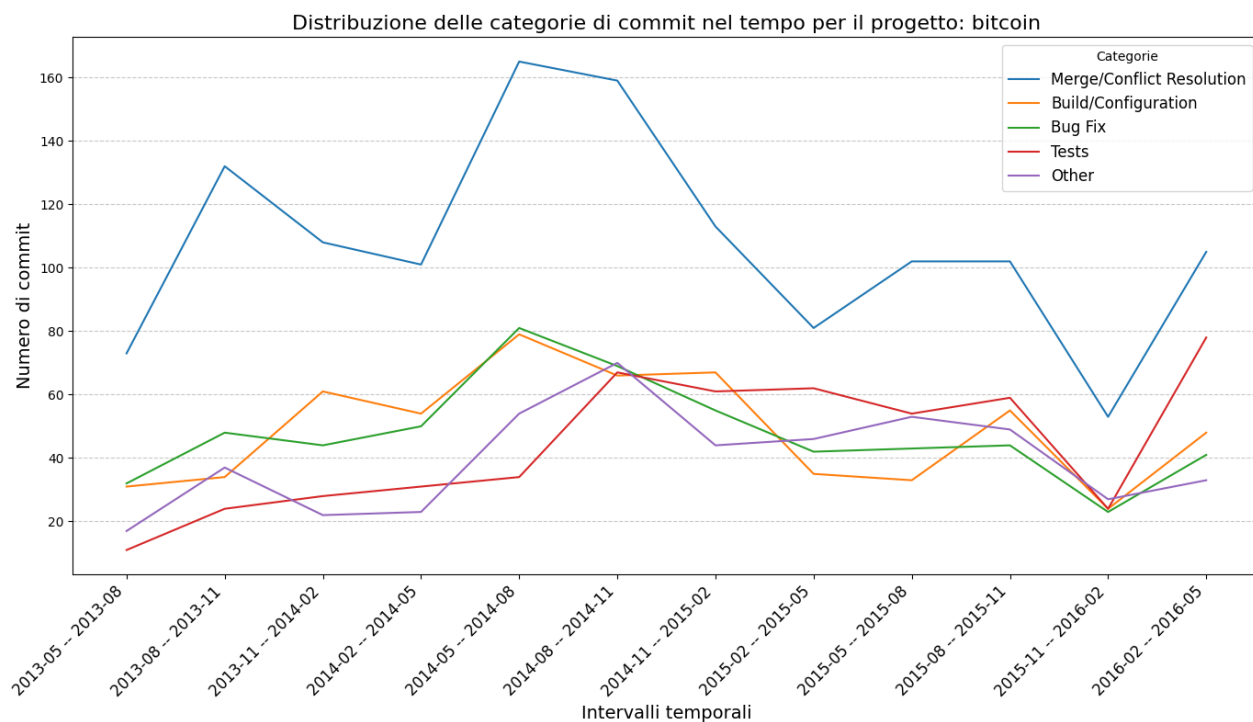


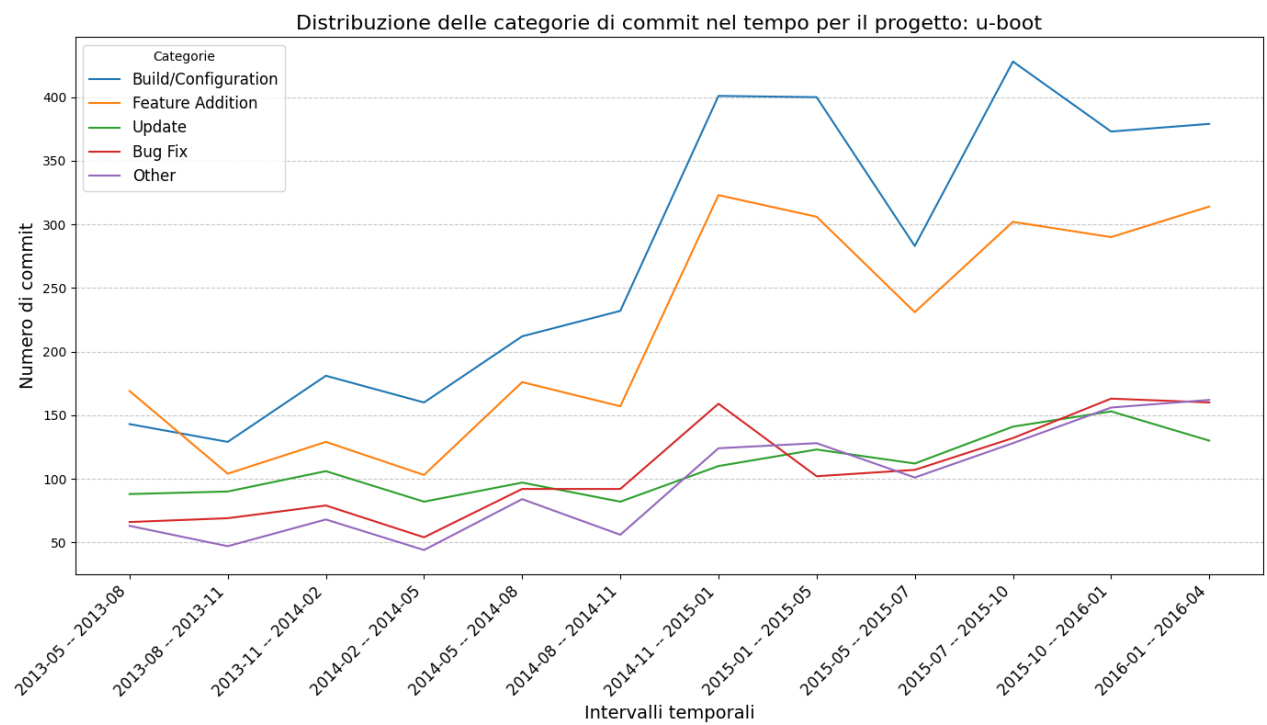
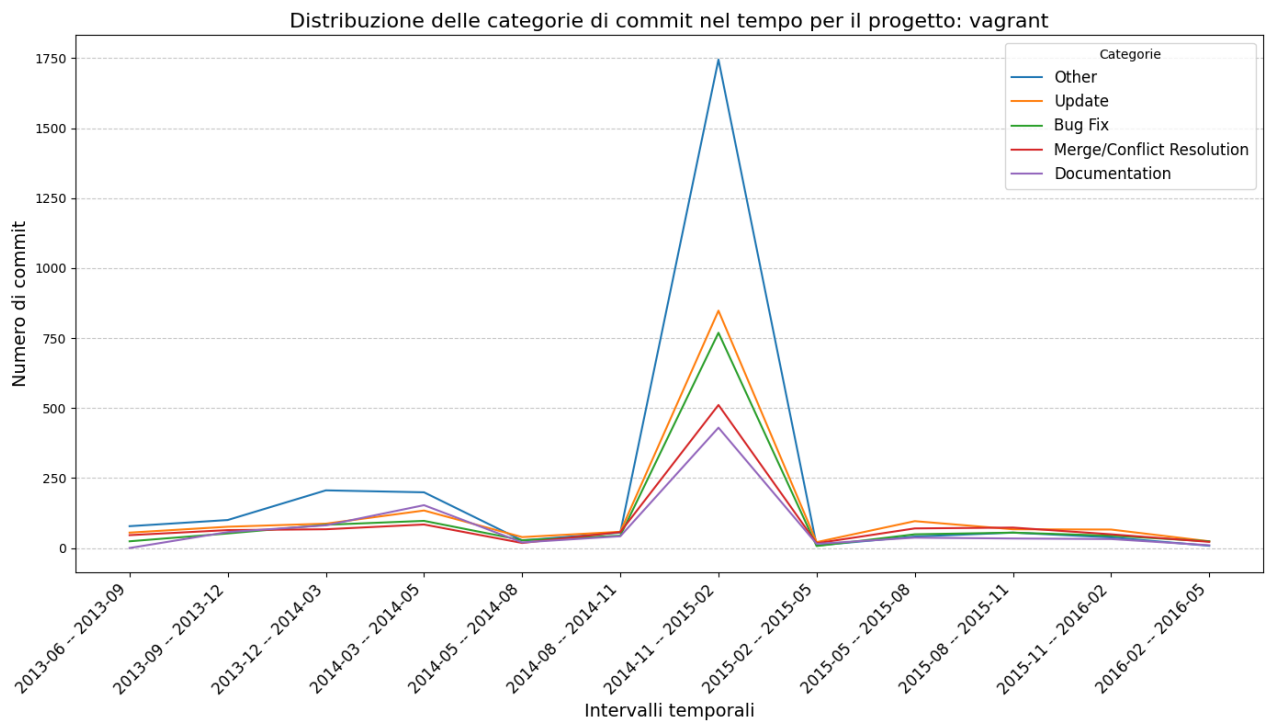
Distribuzione delle categorie di commit nel tempo per il progetto: firefox

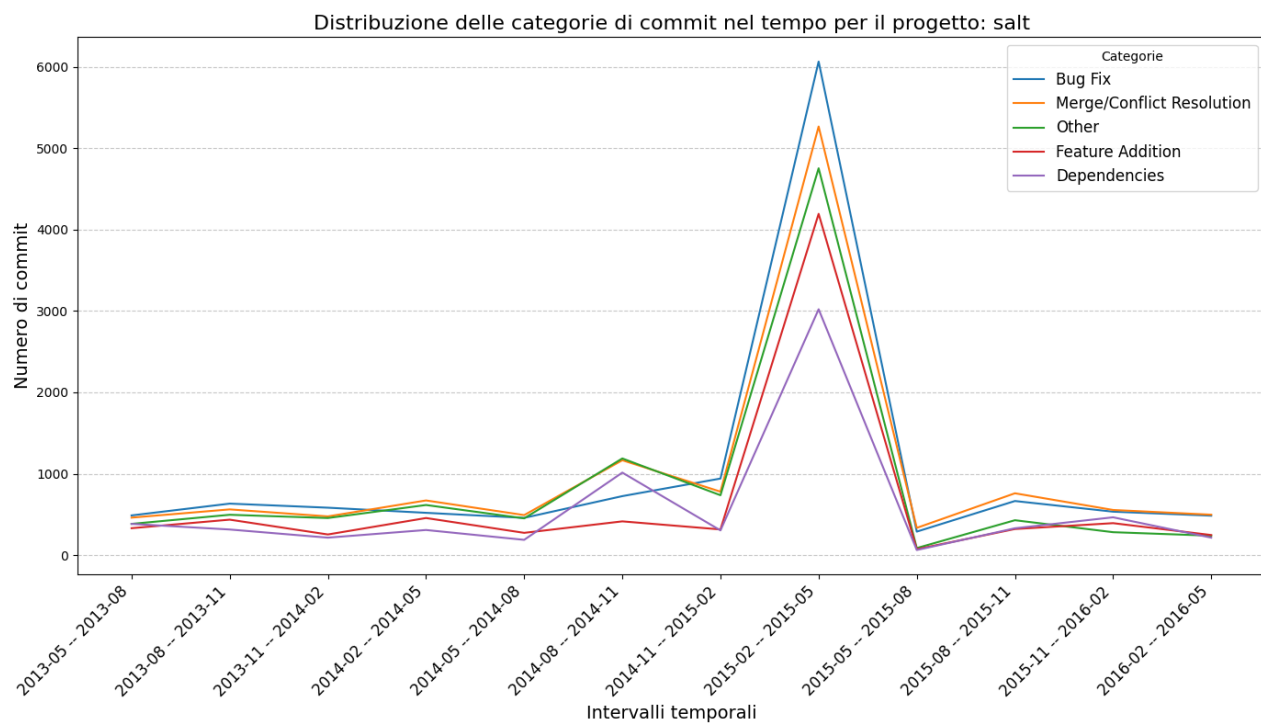












Bibliografia

- Ahammed, T. &. (2020). Understanding the Involvement of Developers in Missing Link Community Smell: An Exploratory Study on Apache Projects.
- G. Catolino, F. P. (2021). Understanding Community Smells Variability: A Statistical Approach. *IEEE/ACM*.
- Magnoni, S. (2016). *An approach to measure Community Smells*.
- Nuri Almarimi, A. O. (2020). Learning to detect community smells in open source software projects. *Knowledge-Based Systems*,.